Otaniemi

March 31, 2026

1 Project Assignment Guidelines

The project applies course techniques to practical wireless applications.

1.1 1. Dataset Creation

Create a spatially consistent dataset. You may consider:

- Single or multiple base stations
- Single or multiple frequencies
- Indoor or outdoor scenarios

You can use any simulator or public dataset, for example:

- Quadriga
- MATLAB Ray Tracing
- RemCom Ray Tracing
- Sionna RT
- DeepMIMO
- University of Stuttgart DICHASUS
- Fraunhofer UWB and 5G dataset
- CAEZ (ETH Zurich CSI Acquisition)

For UE location sampling, choose a suitable sample density. A few samples per m^2 is typically sufficient for many radio resource management applications and can support localization accuracy around $1\,m$.

1.1.1 Quadriga-specific notes

- Set spatial consistency parameter `SC_lambda` based on environment
- Outdoor: `SC_lambda = 30-50 m`
- Indoor: `SC_lambda = 10 m`
- Consider a mobility path with 30 samples

1.1.2 Antenna guidance

- BS antenna: 8-32 elements is sufficient
- Consider sectorized arrays and realistic antenna patterns to improve channel distinguishability
- UE antenna:
  - Sub-6 GHz: single antenna is acceptable
  - mmWave: multi-antenna at both UE and BS (with beamforming)

  - Around 8 UE antennas is sufficient

1.2 2. Fingerprint Localization

Use the dataset for fingerprint localization with both:

- WKNN
- NN-based localization

You are free to choose:

- CSI features
- Distance metric
- Weight function
- NN architecture

Evaluate performance with:

- RMSE
- Error CDF
- Heatmaps of localization error

You may use both classification and regression approaches.

1.3 3. Channel Charting

Create a channel chart for the radio environment using both:

- Conventional DR methods
- NN-based methods

Evaluate using:

- TW
- CT
- KS
- Chart visualizations

When selecting DR parameters, show how they affect performance using a suitable cost function or metric.

At minimum, present channel charting results for two techniques:

- One method covered in class
- One additional method (for example, UMAP)

For resources, see: **What is Channel Charting? | Channel Charting Resources**

1.4 4. Extra Credit Options

You can get extra points by including one or more of the following:

1. Use a customized dataset (for example Aalto campus, Helsinki center, your city) with MATLAB RT, Sionna RT, or RemCom RT
2. In Quadriga, use site-specific simulations (for example Madrid layout)
3. In Quadriga, use spherical wavefront and a near-field scenario

4. Consider multi-point channel charting
5. Use channel charting for an application (for example beam management)
6. Address out-of-sample generalization for channel charting
7. Consider estimated channels with noise and impairments
8. Consider semi-supervised channel charting

1.5 5. If Channel Charting Quality Is Poor

If you cannot obtain a good channel chart from your own dataset:

- First analyze and discuss the dataset limitations
- Then you may use DeepMIMO or measured datasets (for example ETHZ, DICHASUS)

Additional useful datasets:

- MATLAB localization dataset: *Three-Dimensional Indoor Positioning with 802.11az Fingerprinting and Deep Learning*
- Sionna RT dataset: https://zenodo.org/records/14535165

1.6 6. Technical Support

- Channel charting: Pere (`pere.garauburguera@aalto.fi`)
- Fingerprint localization: Xinze (`xinze.li@aalto.fi`)
- Neural networks: Ashvin (`ashvin.1.srinivasan@aalto.fi`)
- You can also contact the course instructor

1.7 Report Requirements

1. Write a **2-5 page** report covering: dataset, fingerprint localization, and channel charting
2. Include figures for UE locations, sample CSI, and summary tables
3. Show channel charting and fingerprint localization results
4. Discuss challenging issues
5. Use proper references and cite all reused figures/block diagrams

1.8 Dataset and Code Submission

Provide a downloadable link to the dataset and selected code.

1.9 Presentation Requirements

- Group presentation length: **10-12 minutes**
- Each group member should present part of the results
- Use clear, representative slides
- You may discuss NN structures, distance metrics, CSI features, DR methods, etc.

Attach presentation slides to the final project submission.

Deadline: 10:00 AM, 17 April 2026

A submission link will be provided for report, slides, and additional materials.

All group members are expected to share responsibility for completing the project.

2 Aalto University private 5G network

https://5gtnf.fi/sites/espoo/ && https://www.6gtnf.fi/aalto-university/

Aalto University has own private 5G/6G mobile network with frequency license for N78 (3640-3700MHz) and N77 (4100-4200MHz) as well as mMwave. Currently completing the installation of 11 mwWave base stations to become one of the largest deployments available for research and test platform. Beside commercial base stations, Aalto site includes cutting edge RAN technologies such as O-RAN, RIC, Backscatter devices as well as core networking technologies such as network slicing, 5G LAN, Time Sensitive Networking, Roaming, Edge Computing (MEC) and eSIM support coming soon.

Each of the 4 corner BS now has a unique azimuth orientation facing outward toward its sector (NE/NW/SE/SW at ±45°, ±135°) with a realistic 18° downward electrical tilt instead of all pointing straight down.

1×1 = 1 element, pattern="iso", polarization="V".

3 ToDo

4 Model explained

Batch RT computation — adds all 200 grid positions as Receiver objects simultaneously, then calls path_solver once. This is far more efficient than 200 individual PathSolver calls (Sionna traces rays from each TX in one GPU-accelerated pass over all RX positions).

Real channel vectors — cir_to_ofdm_channel(frequencies, a_fp[np.newaxis], tau_fp[np.newaxis], normalize=True) produces H_fp of shape [1, n_rx, 1, n_tx, 16, 14, 76] — the true frequency-domain MIMO channel at every grid point.

Feature extraction — |H_fp[0, rx_i, 0, tx_i, 0, 0, :]| extracts the magnitude spectrum for each BS→UE link (first TX antenna, first OFDM symbol). With 4 BSes × 76 subcarriers = 304 physically meaningful features per sample, vs the old 20 geometrical priors.

Solver settings — max_depth=3, LOS + reflection + diffraction, scattering=False. Scattering is disabled for the grid (it dominated compute time); diffraction is kept so NLOS transitions at building corners appear in H.

HDF5 cache — the RT result is saved to fingerprint_rt_dataset.h5. Re-running the cell skips the PathSolver if a matching cache exists (~seconds vs minutes).

Scene restoration — original UE receivers are put back into the scene after computation so downstream visualisation cells still work.

4.1 1. Dataset Creation

[2]:
```
import os # Configure which GPU
import sionna
import tensorflow as tf
```

```
#   GPU configuration
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
    print(f"Using GPU(s): {[g.name for g in gpus]}")
else:
    print("No GPU found - running Sionna on CPU")

if os.getenv("SIONNA_NO_PREVIEW"):
    no_preview = True
else:
    no_preview = False


#resolution = [1920, 1280] # increase for higher quality of renderings
#resolution = [1024, 800] # increase for higher quality of renderings
resolution = [640,400] # low resolution for CPU-only rendering
#resolution = [480,320] # low resolution for CPU-only rendering
#resolution = [320, 128] # low resolution for CPU-only rendering

# Define magic cell command to skip a cell if needed
from IPython.core.magic import register_cell_magic
from IPython import get_ipython

@register_cell_magic
def skip_if(line, cell):
    if eval(line):
        return
    get_ipython().run_cell(cell)

# Set random seed for reproducibility
#sionna.config.seed = 42
```

```
No GPU found - running Sionna on CPU
```

```
2026-03-31 00:19:33.730356: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DEVICE: no
CUDA-capable device is detected
```

[3]:
```
#pip show tensorflow sionna
```

[4]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
from pathlib import Path
from functools import wraps
```

```python
# A robust pictures directory: create if missing and keep prior value if␣
↪already set.
PICTURES_DIR = globals().get("PICTURES_DIR")
if PICTURES_DIR is None:
    PICTURES_DIR = Path.cwd() / "pictures"
    PICTURES_DIR.mkdir(parents=True, exist_ok=True)

def _unwrap(fn):
    """Traverse __wrapped__ chain to reach the true original function."""
    while hasattr(fn, '__wrapped__'):
        fn = fn.__wrapped__
    return fn

#   Stable figure names based on cell ID
# Figures are saved as <cell_id>_fig<n>.png.
# The name is the same across re-runs, so files overwrite rather than
# accumulate numbered copies.

_CURRENT_CELL_ID = globals().get("_CURRENT_CELL_ID", "unknown")
_CURRENT_CELL_FIG_IDX = 0
# Maps full cell URI -> short label like "cell01", "cell02", ...
_CELL_ID_MAP: dict = globals().get("_CELL_ID_MAP", {})
_CELL_ID_COUNTER: list = globals().get("_CELL_ID_COUNTER", [0])  # mutable box

def _cell_short_name(cell_id: str) -> str:
    """Return a stable short label (cell01, cell02, …) for a given cell URI."""
    if cell_id not in _CELL_ID_MAP:
        _CELL_ID_COUNTER[0] += 1
        _CELL_ID_MAP[cell_id] = f"cell{_CELL_ID_COUNTER[0]:02d}"
    return _CELL_ID_MAP[cell_id]

def _pre_run_cell_hook(info):
    global _CURRENT_CELL_ID, _CURRENT_CELL_FIG_IDX
    _CURRENT_CELL_ID = getattr(info, 'cell_id', None) or "unknown"
    _CURRENT_CELL_FIG_IDX = 0   # reset per-cell figure index each run

def save_open_figures(dpi=300):
    global _CURRENT_CELL_FIG_IDX
    saved_paths = []
    for figure_number in plt.get_fignums():
        fig = plt.figure(figure_number)
        if getattr(fig, "_auto_saved", False) and not fig.stale:
            continue
        _CURRENT_CELL_FIG_IDX += 1
        short = _cell_short_name(_CURRENT_CELL_ID)
        filename = f"{short}_figure{_CURRENT_CELL_FIG_IDX:03d}.png"
```

```
        path = PICTURES_DIR / filename
        # Normalize dpi to plain float - Sionna's renderer leaves figure.dpi and
        # figure._original_dpi as numpy.float64. matplotlib's savefig␣
↪unconditionally
        # reads _original_dpi and tries to restore it via setattr, which calls
        # Affine2D.scale(numpy.float64) and fails. Both attributes must be␣
↪patched.
        fig.dpi = float(fig.dpi)
        if hasattr(fig, '_original_dpi'):
            fig._original_dpi = float(fig._original_dpi)
        fig.savefig(path, bbox_inches="tight", dpi=dpi)
        fig._auto_saved = True
        fig.stale = False
        saved_paths.append(path)
    return saved_paths

def _report_saved_figures(saved_paths):
    if saved_paths:
        print("Saved figures:", ", ".join(path.name for path in saved_paths))

#   Register / re-register IPython event hooks
_ipython = get_ipython()
if _ipython is not None and hasattr(_ipython, "events"):
    # Unregister old hooks when this cell is re-run
    _old_pre = globals().get("_pre_run_cell_hook_ref")
    if _old_pre is not None:
        try:
            _ipython.events.unregister("pre_run_cell", _old_pre)
        except ValueError:
            pass
    _old_post = globals().get("_autosave_figures_after_cell")
    if _old_post is not None:
        try:
            _ipython.events.unregister("post_run_cell", _old_post)
        except ValueError:
            pass

    _ipython.events.register("pre_run_cell", _pre_run_cell_hook)
    _pre_run_cell_hook_ref = _pre_run_cell_hook   # keep reference for future␣
↪unregister

    def _autosave_figures_after_cell(*_):
        _report_saved_figures(save_open_figures())

    _ipython.events.register("post_run_cell", _autosave_figures_after_cell)

#   Wrap plt.show to auto-save when called explicitly
```

```
_original_plt_show = _unwrap(globals().get("_original_plt_show", plt.show))

@wraps(_original_plt_show)
def _autosaving_plt_show(*args, **kwargs):
    result = _original_plt_show(*args, **kwargs)
    _report_saved_figures(save_open_figures())
    return result

plt.show = _autosaving_plt_show

from sionna.rt import load_scene, PlanarArray, Transmitter, Receiver, Camera,\
                      Paths, RadioMaterial, SceneObject

from sionna.phy.ofdm import ResourceGrid
from sionna.phy.channel import subcarrier_frequencies, cir_to_ofdm_channel

# Always re-wrap using the true original to avoid stacking wrappers across␣
↪re-runs.
_original_planar_array_show = _unwrap(PlanarArray.show)

@wraps(_original_planar_array_show)
def _autosaving_planar_array_show(self, *args, **kwargs):
    result = _original_planar_array_show(self, *args, **kwargs)
    _report_saved_figures(save_open_figures())
    return result

_autosaving_planar_array_show._autosaves_figures = True
PlanarArray.show = _autosaving_planar_array_show

print(f"Auto-saving notebook figures to {PICTURES_DIR.resolve()}")
```

```
Auto-saving notebook figures to
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/pictures
```

[5]:
```
SPEED_OF_LIGHT = 299_792_458.0
fc = 3.5e9
# TX_POWER_DBM = 23.0   # Transmit power in dBm (23 dBm  200 mW, typical␣
↪small-cell BS)
TX_POWER_DBM = 40.0   # Transmit power in dBm (40 dBm = 10 W, outdoor radio)

#   Simulation parameters - edit here, used throughout the notebook
BS_H               = 16.0   # rooftop BS height (m)
UE_H               = 1.5    # UE height (m) - typical pedestrian
tx_positions       = [      # 8 BS positions in Otaniemi local frame [x, y, z]
    # Inner ring - all face inward toward scene centre
    [ 100,  120, BS_H],     # BS1: NE inner
```

```
    [-100,  100, BS_H],      # BS2: NW inner
    [ 100, -100, BS_H],      # BS3: SE inner
    [-100, -100, BS_H],      # BS4: SW inner
    # Outer corners - all face inward toward scene centre
    [ 250,  200, BS_H],      # BS5: far NE corner
    [-250,  200, BS_H],      # BS6: far NW corner
    [ 250, -150, BS_H],      # BS7: far SE corner
    [-250, -150, BS_H],      # BS8: far SW corner
]
fft_size           = 3168   # OFDM subcarriers (5G NR n78, 100 MHz @ 30 kHz SCS)
subcarrier_spacing = 30e3   # 5G NR numerology 1 (Hz)
```

[6]:
```
import struct, os
from pathlib import Path

def write_ply(filename, vertices, faces):
    """Write a triangle mesh as a binary PLY file (with UV coords for Mitsuba/
 ↪Sionna)."""
    with open(filename, 'wb') as f:
        header = (
            "ply\n"
            "format binary_little_endian 1.0\n"
            f"element vertex {len(vertices)}\n"
            "property float x\nproperty float y\nproperty float z\n"
            "property float u\nproperty float v\n"
            f"element face {len(faces)}\n"
            "property list uchar int vertex_indices\n"
            "end_header\n"
        )
        f.write(header.encode())
        for v in vertices:
            f.write(struct.pack('<fff', *v))
            f.write(struct.pack('<ff', 0.0, 0.0))  # dummy UV coords required␣
 ↪by Mitsuba
        for face in faces:
            f.write(struct.pack('<B', 3))
            f.write(struct.pack('<iii', *face))

def write_box_ply(filename, x0, x1, y0, y1, z0, z1):
    """Write a closed axis-aligned box as binary PLY (8 vertices, 12 triangles).
 ↪"""
    verts = [
        (x0,y0,z0),(x1,y0,z0),(x1,y1,z0),(x0,y1,z0),
        (x0,y0,z1),(x1,y0,z1),(x1,y1,z1),(x0,y1,z1),
    ]
    faces = [
        (0,2,1),(0,3,2),
```

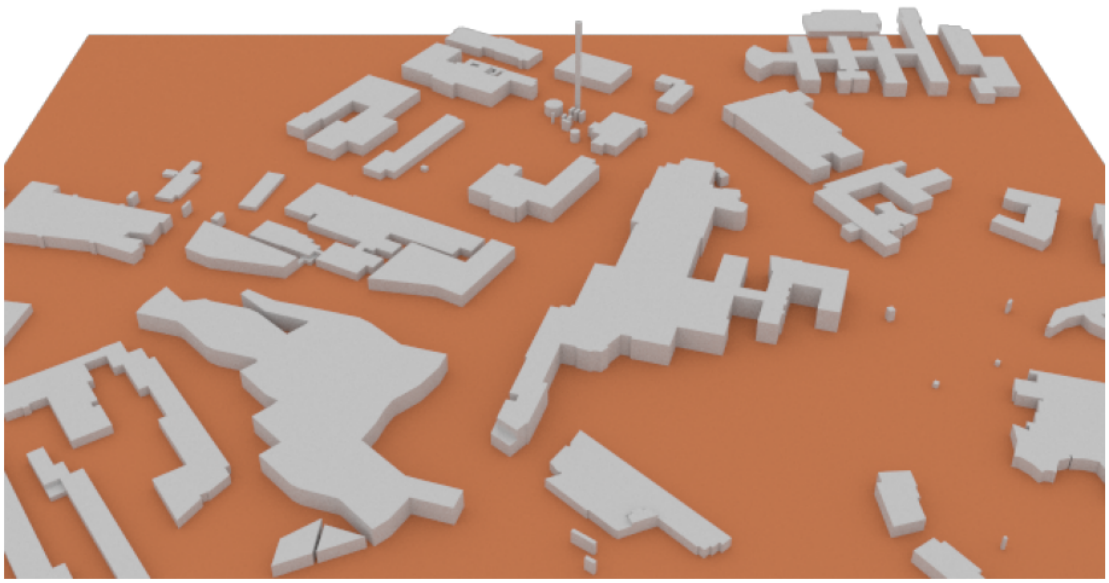
```
        (4,5,6),(4,6,7),
        (0,1,5),(0,5,4),
        (2,3,7),(2,7,6),
        (0,4,7),(0,7,3),
        (1,2,6),(1,6,5),
    ]
    write_ply(filename, verts, faces)

scene_xml_path = os.path.join(os.getcwd(), "OtaniemiScene.xml")
scene_dir = os.path.dirname(scene_xml_path)
ply_dir = os.path.join(scene_dir, "ply")
print("scene_dir:", scene_dir)
print("ply_dir:  ", ply_dir)
print("Using pre-generated Otaniemi scene from OtaniemiScene.xml")
```

```
scene_dir: /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for
Wireless Comunications E7340/Project
ply_dir:   /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for
Wireless Comunications E7340/Project/ply
Using pre-generated Otaniemi scene from OtaniemiScene.xml
```

[7]:
```
scene = load_scene("OtaniemiScene.xml")
scene.frequency = fc

_scene_cls = type(scene)

# Fully unwrap any previously installed wrapper layers before re-wrapping.
# functools.wraps() sets __wrapped__ on each wrapper, so we can traverse the
# chain to reach the true original render method.
def _unwrap(fn):
    while hasattr(fn, '__wrapped__'):
        fn = fn.__wrapped__
    return fn

_original_scene_render = _unwrap(_scene_cls.render)

@wraps(_original_scene_render)
def _autosaving_scene_render(self, *args, **kwargs):
    result = _original_scene_render(self, *args, **kwargs)
    _report_saved_figures(save_open_figures())
    return result

_autosaving_scene_render._autosaves_figures = True
_scene_cls.render = _autosaving_scene_render

# Camera positioned to frame all 8 BSes (corners at ±250 m / ±200 m).
# Pull back to y=-700, raise to z=500 so the full ±300 m grid is visible.
```

```
cam = Camera(position=[0, -700, 500], look_at=[0, 25, 0])

scene.render(camera=cam, num_samples=512, resolution=resolution,␣
↪show_devices=False);
```

Saved figures: cell01_figure001.png



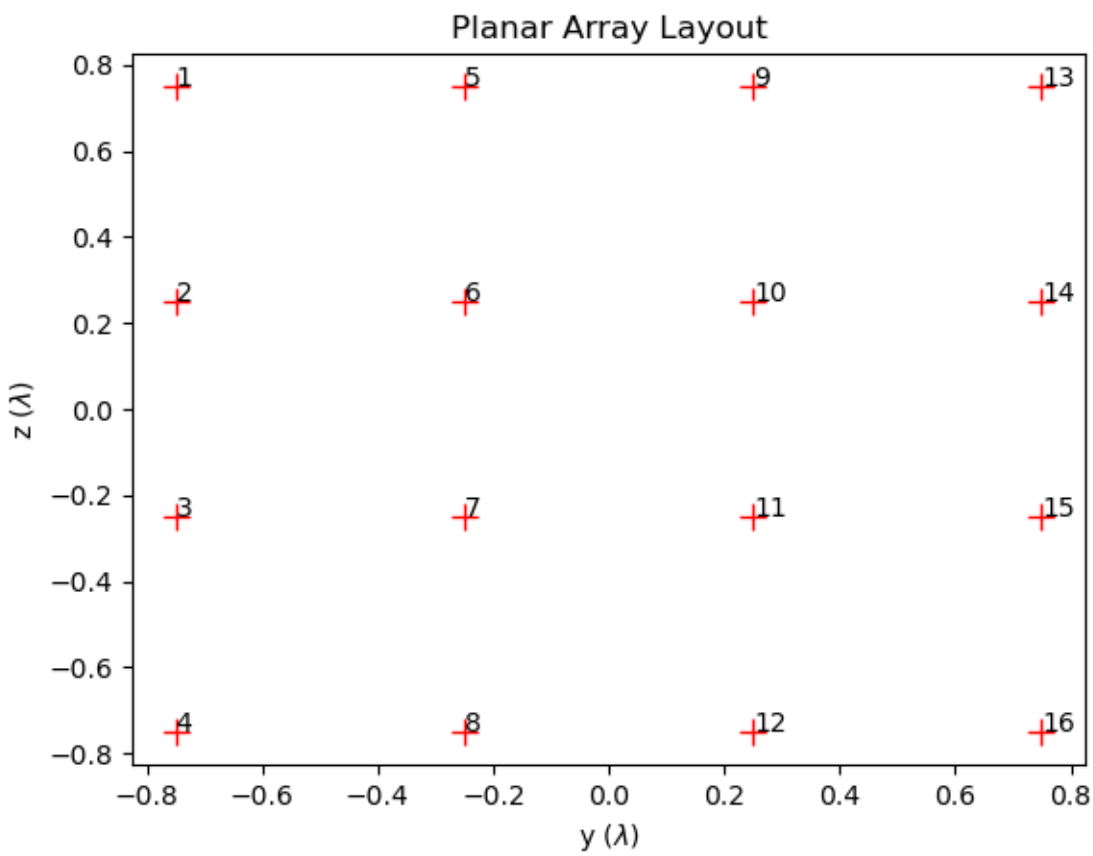
[8]:
```
scene.objects
```

[8]:
```
{'no-name-1': <sionna.rt.scene_object.SceneObject at 0x7f04272f2190>,
 'ground': <sionna.rt.scene_object.SceneObject at 0x7f040e5fd010>}
```

[9]:
```
# Antennas (Sub-6 GHz / n78 style):
# BS: 4x4 = 16 elements with tr38901 realistic pattern + dual-pol (within␣
↪recommended 8-32 range)
# UE: single antenna - task guidelines state single antenna is acceptable for␣
↪sub-6 GHz
tx_array = PlanarArray(num_rows=4, num_cols=4,
                       vertical_spacing=0.5, horizontal_spacing=0.5,
                       pattern="tr38901", polarization="VH")
rx_array = PlanarArray(num_rows=1, num_cols=1,
                       vertical_spacing=0.5, horizontal_spacing=0.5,
                       pattern="iso", polarization="V")
```

```
print('TX positions:', tx_array.positions)
print('RX positions:', rx_array.positions)
tx_array.show()

scene.tx_array = tx_array
scene.rx_array = rx_array
```

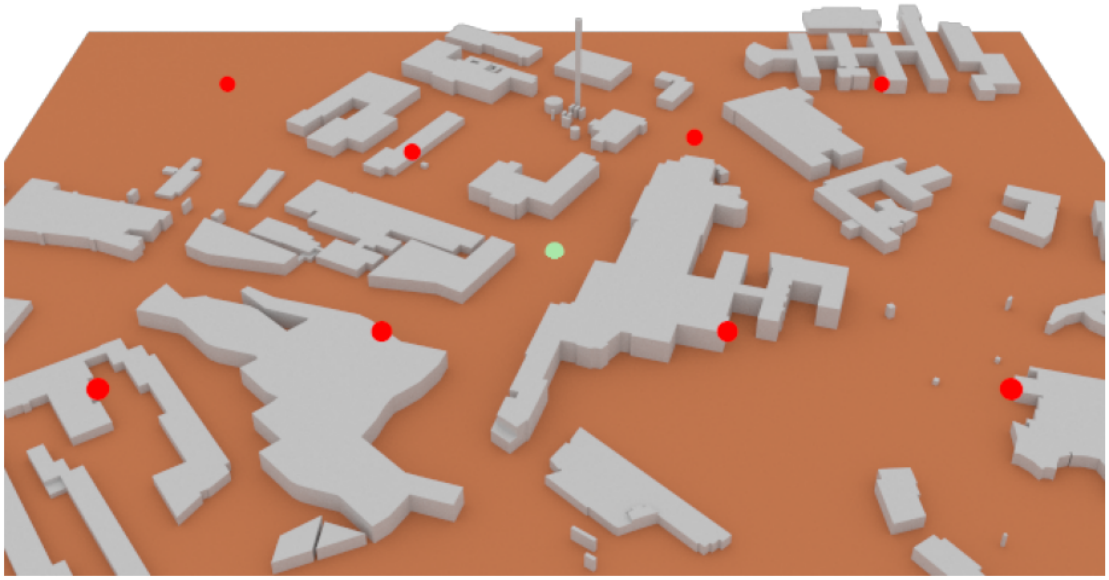
```
TX positions: <bound method AntennaArray.positions of
<sionna.rt.antenna_array.PlanarArray object at 0x7f04272fc850>>
RX positions: <bound method AntennaArray.positions of
<sionna.rt.antenna_array.PlanarArray object at 0x7f040e2203d0>>
Saved figures: cell02_figure001.png
```



[10]:
```
# Eight rooftop base stations: 4 inner-sector + 4 outer-corner.
# BS_H, tx_positions, UE_H are defined once in the constants cell above.

# Sionna orientation = [yaw, pitch, roll] in radians.
# Yaw rotates around Z (azimuth, CCW from +X).
# Pitch of pi/10 (~18°) gives a realistic downward electrical tilt.
#
```

```python
# All BSes face toward the scene centre (origin).
# Yaw is computed as arctan2(-y, -x) so the beam points from each BS toward␣
↪[0,0].
tx_orientations = [
    [float(np.arctan2(-p[1], -p[0])), float(np.pi/10), 0.0]   # yaw toward␣
↪scene centre
    for p in tx_positions
]

def _f3(v):
    """Convert a 3-vector to plain Python floats for Mitsuba/Sionna."""
    return [float(v[0]), float(v[1]), float(v[2])]

# Remove existing TX/RX if re-running
for name in list(scene.transmitters.keys()):
    scene.remove(name)
for name in list(scene.receivers.keys()):
    scene.remove(name)

for i, (pos, ori) in enumerate(zip(tx_positions, tx_orientations), start=1):
    name = f"BS{i}"
    tx = Transmitter(name=name, position=_f3(pos), orientation=_f3(ori))
    tx.power_dbm = float(TX_POWER_DBM)
    scene.add(tx)

#--------------------------------------
# Two representative UEs for visual/path validation (actual dataset uses grid␣
↪sampling)
ue_positions = [
    [0.0,  0.0, UE_H],      # scene center
    [80.0, 60.0, UE_H],     # NE quadrant pedestrian
]
rx_list = []
for i, pos in enumerate(ue_positions):
    name = f"UE{i}"
    rx = Receiver(name=name, position=_f3(pos))
    scene.add(rx)
    rx_list.append(name)

scene.render(camera=cam, num_samples=1024, resolution=resolution);
```

```
Saved figures: cell03_figure001.png
```

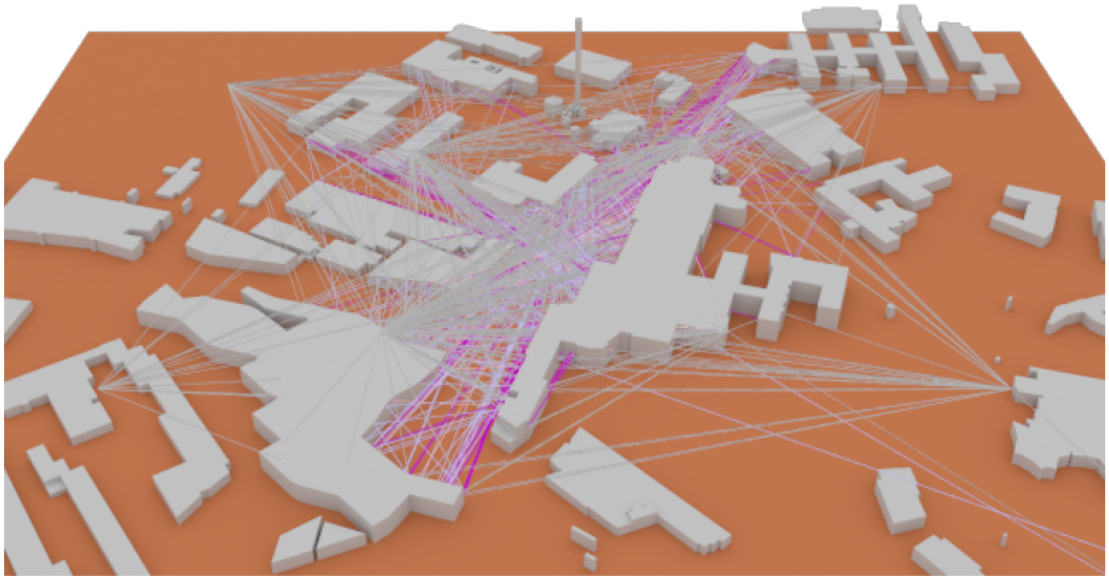


[11]:
```
from sionna.rt import PlanarArray, RadioMapSolver

# Reuse arrays if they already exist in the notebook; create only if missing.
if "tx_array" not in globals():
  print("Creating tx_array...")
  tx_array = PlanarArray(
    num_rows=4, num_cols=4,
    vertical_spacing=0.5, horizontal_spacing=0.5,
    pattern="tr38901", polarization="VH"
  )

if "rx_array" not in globals():
  print("Creating rx_array...")
  rx_array = PlanarArray(
    num_rows=1, num_cols=1,
    vertical_spacing=0.5, horizontal_spacing=0.5,
    pattern="iso", polarization="V"
  )

scene.tx_array = tx_array
scene.rx_array = rx_array
```

[12]:
```
from sionna.rt import PathSolver
import inspect
```

```python
#   Path generation settings
# Settings are intentionally aligned with the fingerprint PathSolver (cell␣
↪below)
# so the visualisation CIR accurately reflects what goes into the fingerprint␣
↪DB.
# max_depth=3: up to 3-bounce specular reflections.
# scattering=False: disabled to match fingerprint grid settings.
# Diffraction is kept so NLOS transitions at building corners appear in H.
#
# num_samples controls how many diffracted rays are traced.
# Increase it for denser coverage at the cost of longer compute time.
path_solver = PathSolver()

solver_kwargs = dict(
    max_depth=3,          # 3-bounce reflections - aligned with fingerprint␣
↪settings
    los=True,             # include line-of-sight
    reflection=True,      # include specular reflections
    diffraction=True,     # include edge diffraction (adds ~2-5× paths)
    scattering=False,     # disabled - aligned with fingerprint PathSolver␣
↪settings
    num_samples=int(1e6), # rays for diffraction sampling
)

try:
    supported_kwargs = set(inspect.signature(path_solver.__call__).parameters)
    solver_kwargs = {k: v for k, v in solver_kwargs.items() if k in␣
↪supported_kwargs}

    paths = path_solver(scene=scene, **solver_kwargs)
except TypeError:
    # Compatibility fallback for versions expecting the scene as a positional␣
↪arg
    paths = path_solver(scene, **solver_kwargs)

print(f"Number of paths found: {int(np.asarray(paths.tau).shape[-1])}")

# Render scene with paths
scene.render(camera=cam, paths=paths, num_samples=8192, resolution=resolution,␣
↪show_devices=False);
```

```
Number of paths found: 139
Saved figures: cell04_figure001.png
```

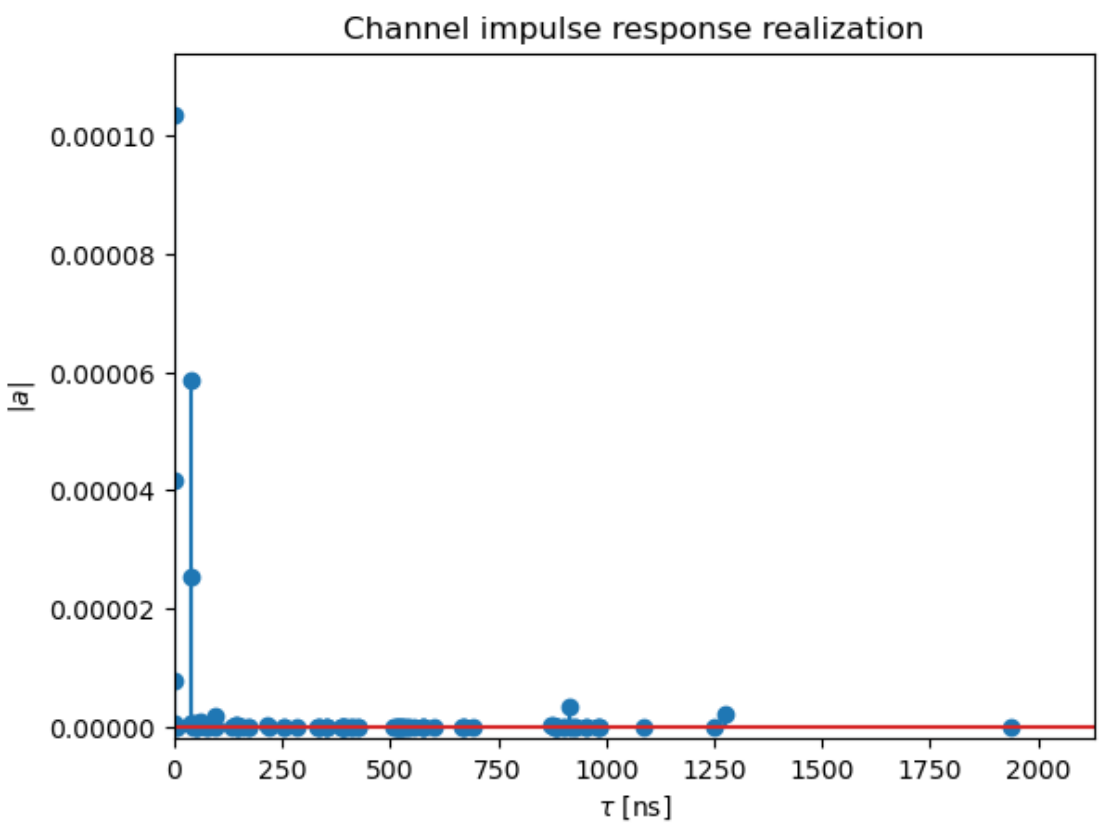


[13]:
```
a, tau = paths.cir(out_type='numpy')

# Shape: [num_rx, num_rx_ant, num_tx, num_tx_ant, num_paths, num_time_steps]
print("Shape of a: ", a.shape)

# Shape: [num_rx, num_rx_ant, num_tx, num_tx_ant, num_paths] or [num_rx,␣
↪num_tx, num_paths]
print("Shape of tau: ", tau.shape)
```

```
Shape of a:  (2, 1, 8, 32, 139, 1)
Shape of tau:  (2, 8, 139)
```

[14]:
```
# tau: [rx, rx_ant, tx, tx_ant, paths] (5D) or [rx, tx, paths] (3D) - no batch␣
↪dim in Sionna 1.2
if tau.ndim == 5:
    t = tau[0, 0, 0, 0, :] / 1e-9
elif tau.ndim == 3:
    t = tau[0, 0, :] / 1e-9
else:
    t = tau.reshape(-1) / 1e-9

# a: [rx, rx_ant, tx, tx_ant, paths, time_steps] (6D) - select rx=0, rx_ant=0,␣
↪tx=0, tx_ant=0
if a.ndim == 6:
```

```
    a_abs = np.abs(a[0, 0, 0, 0, :, 0])
else:
    a_abs = np.abs(a.reshape(-1))

a_max = np.max(a_abs)
# Add dummy entry at start/end for nicer figure
t = np.concatenate([(0.,), t, (np.max(t)*1.1,)])
a_abs = np.concatenate([(np.nan,), a_abs, (np.nan,)])

# And plot the CIR
plt.figure()
plt.title("Channel impulse response realization")

plt.stem(t, a_abs)
plt.xlim([0, np.max(t)])
plt.ylim([-2e-6, a_max*1.1])
plt.xlabel(r"$\tau$ [ns]")
plt.ylabel(r"$|a|$")
# Figure needs to be saved before plt.show() to avoid blank output in some␣
↪environments (e.g. Sionna Playground)
plt.savefig(PICTURES_DIR / "sionna_cir.png", dpi=300)
plt.show()
```



[15]:
```
import mitsuba as mi

# Note: do NOT call mi.set_variant() here - Sionna already initialised Mitsuba
# (llvm_ad_rgb) via load_scene() in cell 6. Calling set_variant() after Dr.JIT
# has been used raises RuntimeError: "Cannot change variant after␣
↪initialisation".

scene_mi = mi.load_file("OtaniemiScene.xml")
for shape in scene_mi.shapes():
    print(type(shape).__name__, shape.id())
```

```
Mesh
Mesh ground
```

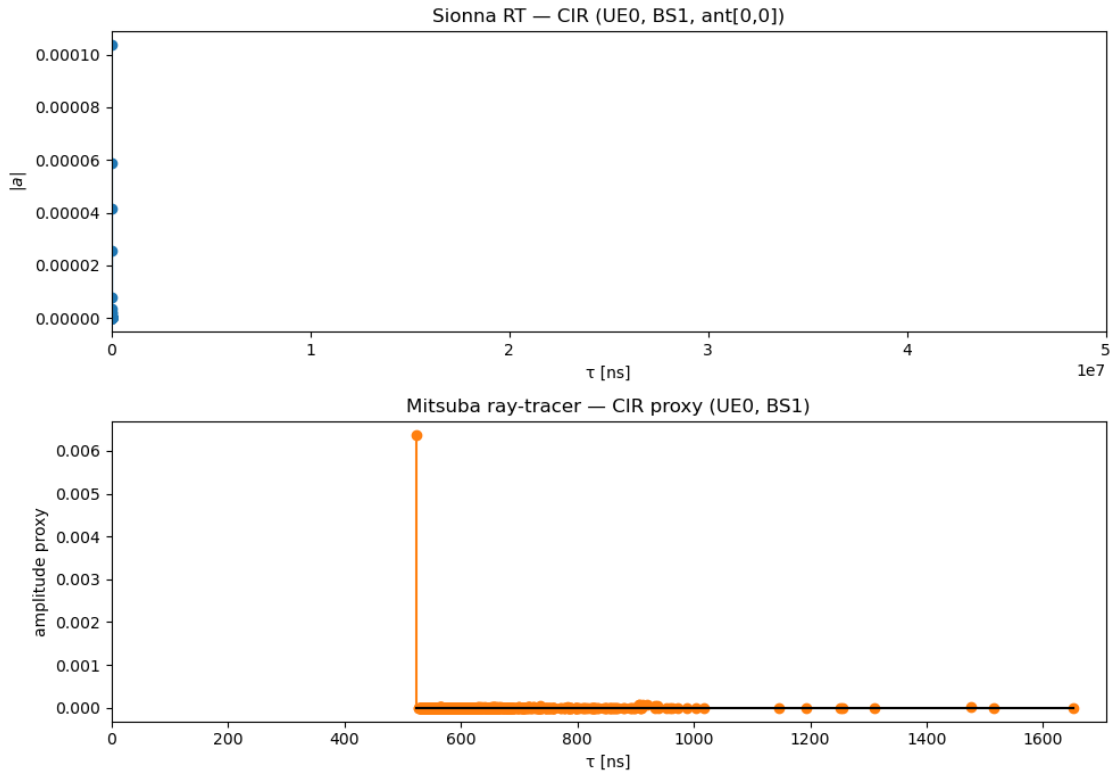
[16]:
```
import mitsuba as mi
import drjit as dr
import numpy as np
import matplotlib.pyplot as plt
import os
import h5py

# Use Mitsuba's built-in ray tracer to compute a CIR and compare against Sionna␣
↪RT.

#   Constants (re-declared so this cell runs standalone after prior cells)
SPEED_OF_LIGHT = 299_792_458.0
fc = 3.5e9

#   1. Geometry: match Sionna RT BS/UE placement
# Use actual BS1 position (tx_positions[0]) so indices [rx=0, tx=0] in paths.
↪cir()
# reference the same spatial link as the Mitsuba comparison.
BS_H = globals().get("BS_H", 16.0)
UE_H = globals().get("UE_H", 1.5)

# Reuse positions from earlier cells if available
if "tx_positions" in globals() and len(tx_positions) > 0:
    _tp = tx_positions
else:
    _tp = [[100.0, 120.0, BS_H]]
    tx_positions = _tp

if "ue_positions" in globals() and len(ue_positions) > 0:
    _up = ue_positions
else:
```

```
    _up = [
        [0.0,  0.0, UE_H],   # UE0
        [80.0, 60.0, UE_H],  # UE1
    ]
    ue_positions = _up

bs_pos_np = np.array(_tp[0], dtype=np.float64)   # BS1: tx_positions[0]
ue_pos_np_list = [np.array(p, dtype=np.float64) for p in ue_positions]

#   2. Load Mitsuba scene if not already in kernel
if 'scene_mi' not in globals():
    scene_mi = mi.load_file("OtaniemiScene.xml")
    print("Loaded Mitsuba scene from OtaniemiScene.xml")

#   3. Load Sionna a / tau from HDF5 if PathSolver cells have not been run
if 'a' not in globals() or 'tau' not in globals():
    _h5 = "sionna_dataset.h5"
    if not os.path.exists(_h5):
        raise RuntimeError(
            f"'{_h5}' not found - run the Sionna path-solver cells first."
        )
    with h5py.File(_h5, "r") as _f:
        a   = _f["CIR/a_real"][:] + 1j * _f["CIR/a_imag"][:]
        tau = _f["CIR/tau_s"][:]
    print(f"Loaded a {a.shape} and tau {tau.shape} from {_h5}")

#   4. Mitsuba path-tracer helper
def trace_paths(scene_mi, origin_np, target_np, max_depth=3):
    """
    Shoot rays from origin to target and collect (delay_ns, amplitude) per path.
    LOS: amplitude = 1/r (free-space).
    Single-bounce: amplitude = cos_i * cos_r / (d_tx * d_rx).
    """
    results = []
    origin = mi.Point3f(float(origin_np[0]), float(origin_np[1]),␣
 ↪float(origin_np[2]))

    # Direct LOS path
    d_vec = target_np - origin_np
    dist  = float(np.linalg.norm(d_vec))
    d_hat = d_vec / dist
    ray_d = mi.Ray3f(origin,
                     mi.Vector3f(float(d_hat[0]), float(d_hat[1]),␣
 ↪float(d_hat[2])))
    ray_d.maxt = float(dist * 0.9999)
    si_los = scene_mi.ray_intersect(ray_d)
    if dr.none(si_los.is_valid()):
```

```
        results.append((dist / SPEED_OF_LIGHT / 1e-9, 1.0 / dist))

    # Single-bounce reflections
    if max_depth >= 1:
        n_probe = 4096
        rng  = np.random.default_rng(42)
        dirs = rng.standard_normal((n_probe, 3)).astype(np.float32)
        dirs /= np.linalg.norm(dirs, axis=1, keepdims=True)
        rays = mi.Ray3f(
            o=mi.Point3f(np.tile(origin_np.astype(np.float32), (n_probe, 1)).T.
 ↪tolist()),
            d=mi.Vector3f(dirs.T.tolist())
        )
        si = scene_mi.ray_intersect(rays)
        valid = np.array(si.is_valid(), dtype=bool)

        hit_p = np.array(si.p).T[valid, :]
        hit_n = np.array(si.n).T[valid, :]
        dirs_valid = dirs[valid]

        for k in range(hit_p.shape[0]):
            p      = hit_p[k]
            n      = hit_n[k]
            to_rx  = target_np - p
            d_rx   = float(np.linalg.norm(to_rx))
            ray_rx = mi.Ray3f(
                mi.Point3f(float(p[0]), float(p[1]), float(p[2])),
                mi.Vector3f(float(to_rx[0]/d_rx), float(to_rx[1]/d_rx),␣
 ↪float(to_rx[2]/d_rx))
            )
            ray_rx.maxt = float(d_rx * 0.9999)
            if dr.any(scene_mi.ray_intersect(ray_rx).is_valid()):
                continue   # blocked
            d_tx   = float(np.linalg.norm(p - origin_np))
            cos_i  = abs(float(np.dot(n, -dirs_valid[k])))
            cos_r  = abs(float(np.dot(n,  to_rx / d_rx)))
            amp    = cos_i * cos_r / (d_tx * d_rx)
            results.append(((d_tx + d_rx) / SPEED_OF_LIGHT / 1e-9, amp))

    return results

#   5. Compute Mitsuba CIR for UE0
print("Tracing Mitsuba paths (this may take ~30 s) …")
mi_paths  = trace_paths(scene_mi, bs_pos_np, ue_pos_np_list[0], max_depth=3)
mi_delays = np.array([p[0] for p in mi_paths])
mi_amps   = np.array([p[1] for p in mi_paths])
print(f"  → {len(mi_delays)} paths found")
```

```python
#   6. Sionna CIR for BS1 → UE0 (first antenna pair)
if tau.ndim == 5:
    sionna_tau_ns = tau[0, 0, 0, 0, :] / 1e-9   # [rx, rx_ant, tx, tx_ant,␣
 ↪paths]
elif tau.ndim == 3:
    sionna_tau_ns = tau[0, 0, :] / 1e-9
else:
    sionna_tau_ns = np.asarray(tau).reshape(-1) / 1e-9

if a.ndim == 6:
    sionna_amp = np.abs(a[0, 0, 0, 0, :, 0])
else:
    sionna_amp = np.abs(np.asarray(a).reshape(-1))

n = min(len(sionna_tau_ns), len(sionna_amp))
sionna_tau_ns = sionna_tau_ns[:n]
sionna_amp    = sionna_amp[:n]

#   7. Plot comparison
fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=False)

axes[0].set_title("Sionna RT - CIR (UE0, BS1, ant[0,0])")
axes[0].stem(sionna_tau_ns, sionna_amp, linefmt='C0-', markerfmt='C0o',␣
 ↪basefmt='k-')
axes[0].set_xlabel("  [ns]"); axes[0].set_ylabel(r"$|a|$")
axes[0].set_xlim(left=0)

axes[1].set_title("Mitsuba ray-tracer - CIR proxy (UE0, BS1)")
axes[1].stem(mi_delays, mi_amps, linefmt='C1-', markerfmt='C1o', basefmt='k-')
axes[1].set_xlabel("  [ns]"); axes[1].set_ylabel("amplitude proxy")
axes[1].set_xlim(left=0)

plt.tight_layout()
plt.savefig(PICTURES_DIR / "sionna_vs_mitsuba_cir_comparison.png", dpi=300)
plt.show()

#   8. Numeric summary
print(f"\nSionna  : {len(sionna_tau_ns)} paths | "
      f"min  ={sionna_tau_ns.min():.2f} ns | max  ={sionna_tau_ns.max():.2f} ns")
print(f"Mitsuba : {len(mi_delays)} paths | "
      f"min  ={mi_delays.min():.2f} ns | max  ={mi_delays.max():.2f} ns")

los_dist = np.linalg.norm(bs_pos_np - ue_pos_np_list[0])
print(f"Expected LOS delay (BS1→UE0): {los_dist / SPEED_OF_LIGHT / 1e-9:.2f}␣
 ↪ns")
```

```
Tracing Mitsuba paths (this may take ~30 s) …
  → 852 paths found
```




```
Sionna  : 139 paths | min  =-1000000000.00 ns | max  =1936.51 ns
Mitsuba : 852 paths | min  =523.28 ns | max  =1651.41 ns
Expected LOS delay (BS1→UE0): 523.28 ns
```

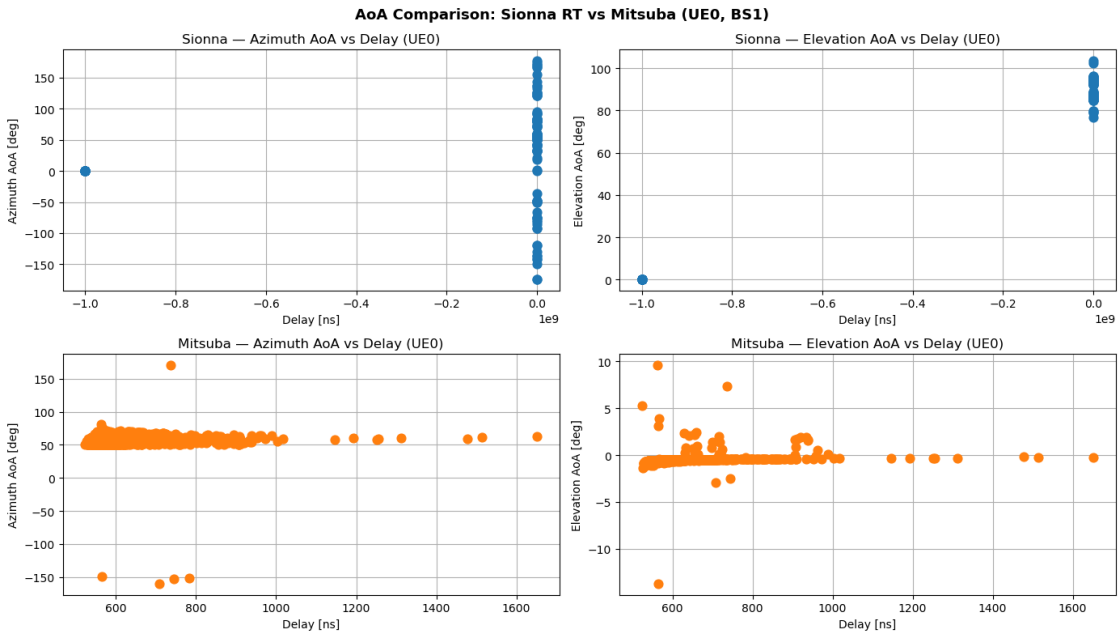
[17]:
```
#   AoA Calculation and Comparison

#   1. Sionna AoA
# Retrieve angular information from Sionna paths object
# paths.phi_r / paths.theta_r are drjit TensorXf in Sionna 1.2 - use np.array()␣
 ↪to convert.
phi_r = np.array(paths.phi_r)       # azimuth AoA (radians)
theta_r = np.array(paths.theta_r)   # elevation AoA (radians)

# Select UE0 and reduce to a 1D path vector robustly
# Shape: [rx, rx_ant, tx, tx_ant, paths] (5D) or [rx, tx, paths] (3D)
if phi_r.ndim == 5:
  phi_link = phi_r[0, 0, 0, 0, :]
  theta_link = theta_r[0, 0, 0, 0, :]
elif phi_r.ndim == 3:
```

```
  phi_link = phi_r[0, 0, :]
  theta_link = theta_r[0, 0, :]
else:
  phi_link = np.asarray(phi_r).reshape(-1)
  theta_link = np.asarray(theta_r).reshape(-1)

sionna_az_aoa = np.degrees(np.asarray(phi_link).reshape(-1))
sionna_el_aoa = np.degrees(np.asarray(theta_link).reshape(-1))

# Align Sionna delay and AoA vectors
n_sionna = min(len(sionna_tau_ns), len(sionna_az_aoa), len(sionna_el_aoa))
sionna_tau_ns_aoa = sionna_tau_ns[:n_sionna]
sionna_az_aoa = sionna_az_aoa[:n_sionna]
sionna_el_aoa = sionna_el_aoa[:n_sionna]

#   2. Mitsuba AoA
# Recompute Mitsuba paths with arrival direction at UE0
def compute_aoa_mitsuba(scene_mi, origin_np, target_np, max_depth=3):
  """
  Returns list of (delay_ns, amplitude, az_aoa_deg, el_aoa_deg) tuples.
  AoA is the direction of the incoming ray at the receiver.
  """
  results = []
  origin = mi.Point3f(float(origin_np[0]), float(origin_np[1]),␣
 ↪float(origin_np[2]))

  #   LOS
  d_vec = target_np - origin_np
  dist = float(np.linalg.norm(d_vec))
  d_hat = d_vec / dist
  ray_d = mi.Ray3f(origin,
                   mi.Vector3f(float(d_hat[0]), float(d_hat[1]),␣
 ↪float(d_hat[2])))
  ray_d.maxt = float(dist * 0.9999)
  si_los = scene_mi.ray_intersect(ray_d)
  if dr.none(si_los.is_valid()):
    inc = -d_hat
    az = np.degrees(np.arctan2(inc[1], inc[0]))
    el = np.degrees(np.arcsin(np.clip(inc[2], -1, 1)))
    results.append((dist / SPEED_OF_LIGHT / 1e-9, 1.0 / dist, az, el))

  #   Single-bounce reflections
  if max_depth >= 1:
    n_probe = 4096
    rng = np.random.default_rng(42)
    dirs = rng.standard_normal((n_probe, 3)).astype(np.float32)
    dirs /= np.linalg.norm(dirs, axis=1, keepdims=True)
```

```
    rays = mi.Ray3f(
      o=mi.Point3f(np.tile(origin_np.astype(np.float32), (n_probe, 1)).T.
 ↪tolist()),
      d=mi.Vector3f(dirs.T.tolist())
    )
    si = scene_mi.ray_intersect(rays)
    valid = np.array(si.is_valid(), dtype=bool)

    hit_p = np.array(si.p).T[valid, :]
    hit_n = np.array(si.n).T[valid, :]
    dirs_valid = dirs[valid]

    for k in range(hit_p.shape[0]):
      p = hit_p[k]
      n = hit_n[k]
      to_rx = target_np - p
      dist_rx = float(np.linalg.norm(to_rx))
      to_rx_hat = to_rx / dist_rx

      ray_rx = mi.Ray3f(
        mi.Point3f(float(p[0]), float(p[1]), float(p[2])),
        mi.Vector3f(float(to_rx_hat[0]), float(to_rx_hat[1]),␣
 ↪float(to_rx_hat[2]))
      )
      ray_rx.maxt = float(dist_rx * 0.9999)
      if dr.any(scene_mi.ray_intersect(ray_rx).is_valid()):
        continue  # blocked

      dist_tx = float(np.linalg.norm(p - origin_np))
      total = dist_tx + dist_rx
      delay = total / SPEED_OF_LIGHT / 1e-9
      cos_i = abs(float(np.dot(n, -dirs_valid[k])))
      cos_r = abs(float(np.dot(n, to_rx_hat)))
      amp = cos_i * cos_r / (dist_tx * dist_rx)

      inc = -to_rx_hat
      az = np.degrees(np.arctan2(inc[1], inc[0]))
      el = np.degrees(np.arcsin(np.clip(inc[2], -1, 1)))
      results.append((delay, amp, az, el))

  return results

# bs_pos_np is set by the Mitsuba CIR cell (tx_positions[0] = BS1 NE corner)
mi_paths_aoa = compute_aoa_mitsuba(scene_mi, bs_pos_np, ue_pos_np_list[0],␣
 ↪max_depth=3)
mi_delays_aoa = np.array([p[0] for p in mi_paths_aoa])
```

```
mi_amps_aoa   = np.array([p[1] for p in mi_paths_aoa])
mi_az_aoa     = np.array([p[2] for p in mi_paths_aoa])
mi_el_aoa     = np.array([p[3] for p in mi_paths_aoa])

#   3. Plot Comparison
fig, axes = plt.subplots(2, 2, figsize=(14, 8))

axes[0, 0].scatter(sionna_tau_ns_aoa, sionna_az_aoa, c='C0', s=60, zorder=3)
axes[0, 0].set_title("Sionna - Azimuth AoA vs Delay (UE0)")
axes[0, 0].set_xlabel("Delay [ns]"); axes[0, 0].set_ylabel("Azimuth AoA [deg]")
axes[0, 0].grid(True)

axes[0, 1].scatter(sionna_tau_ns_aoa, sionna_el_aoa, c='C0', s=60, zorder=3)
axes[0, 1].set_title("Sionna - Elevation AoA vs Delay (UE0)")
axes[0, 1].set_xlabel("Delay [ns]"); axes[0, 1].set_ylabel("Elevation AoA␣
↪[deg]")
axes[0, 1].grid(True)

axes[1, 0].scatter(mi_delays_aoa, mi_az_aoa, c='C1', s=60, zorder=3)
axes[1, 0].set_title("Mitsuba - Azimuth AoA vs Delay (UE0)")
axes[1, 0].set_xlabel("Delay [ns]"); axes[1, 0].set_ylabel("Azimuth AoA [deg]")
axes[1, 0].grid(True)

axes[1, 1].scatter(mi_delays_aoa, mi_el_aoa, c='C1', s=60, zorder=3)
axes[1, 1].set_title("Mitsuba - Elevation AoA vs Delay (UE0)")
axes[1, 1].set_xlabel("Delay [ns]"); axes[1, 1].set_ylabel("Elevation AoA␣
↪[deg]")
axes[1, 1].grid(True)

plt.suptitle("AoA Comparison: Sionna RT vs Mitsuba (UE0, BS1)", fontsize=13,␣
↪fontweight='bold')
plt.tight_layout()
plt.savefig(PICTURES_DIR / "sionna_vs_mitsuba_aoa_comparison.png", dpi=300)
plt.show()

#   4. Numeric Summary
print("=== Sionna AoA (UE0) ===")
for i, (tau_i, az, el) in enumerate(zip(sionna_tau_ns_aoa, sionna_az_aoa,␣
↪sionna_el_aoa)):
  print(f"  Path {i:2d}: delay={tau_i:7.2f} ns | Az={az:8.2f} deg | El={el:8.
↪2f} deg")

print("\n=== Mitsuba AoA (UE0) ===")
for i, (d, az, el) in enumerate(zip(mi_delays_aoa, mi_az_aoa, mi_el_aoa)):
  print(f"  Path {i:2d}: delay={d:7.2f} ns | Az={az:8.2f} deg | El={el:8.2f}␣
↪deg")
```

```python
# LOS AoA reference: incoming direction at UE from BS1
los_vec = np.array(ue_positions[0]) - bs_pos_np
los_vec /= np.linalg.norm(los_vec)
los_az = np.degrees(np.arctan2(los_vec[1], los_vec[0]))
los_el = np.degrees(np.arcsin(np.clip(los_vec[2], -1, 1)))
print(f"\nExpected LOS AoA (BS1→UE0): Az={los_az:.2f} deg, El={los_el:.2f} deg")
```




```
=== Sionna AoA (UE0) ===
  Path  0: delay=   0.00 ns | Az=   50.19 deg | El=   84.70 deg
  Path  1: delay=  39.09 ns | Az=   81.16 deg | El=   85.07 deg
  Path  2: delay=  40.04 ns | Az=   81.16 deg | El=   95.95 deg
  Path  3: delay= 425.44 ns | Az=  -74.73 deg | El=   87.08 deg
  Path  4: delay=   1.02 ns | Az=   50.19 deg | El=   96.39 deg
  Path  5: delay=  42.47 ns | Az=   92.76 deg | El=   85.10 deg
  Path  6: delay=  95.83 ns | Az=   91.50 deg | El=   85.52 deg
  Path  7: delay= 217.18 ns | Az=  170.35 deg | El=   86.25 deg
  Path  8: delay=  50.58 ns | Az=   71.16 deg | El=   85.17 deg
  Path  9: delay=  51.51 ns | Az=   71.16 deg | El=   95.83 deg
  Path 10: delay= 926.26 ns | Az=   58.12 deg | El=   88.09 deg
  Path 11: delay=  43.42 ns | Az=   92.76 deg | El=   95.91 deg
  Path 12: delay= 134.72 ns | Az=   18.57 deg | El=   85.78 deg
  Path 13: delay=  40.41 ns | Az=   79.85 deg | El=   79.95 deg
  Path 14: delay=  42.37 ns | Az=   79.14 deg | El=  102.44 deg
  Path 15: delay= 147.78 ns | Az= -174.46 deg | El=   85.87 deg
  Path 16: delay= 148.57 ns | Az= -174.46 deg | El=   94.98 deg
  Path 17: delay=   1.02 ns | Az=   50.10 deg | El=   96.39 deg
```

```
Path 18: delay=  51.72 ns | Az=   82.13 deg | El=   78.98 deg
Path 19: delay= 531.62 ns | Az=   51.18 deg | El=   87.37 deg
Path 20: delay= 539.50 ns | Az=   50.93 deg | El=   87.39 deg
Path 21: delay=  77.88 ns | Az=   31.21 deg | El=   85.39 deg
Path 22: delay= 389.12 ns | Az=   83.49 deg | El=   87.27 deg
Path 23: delay= 389.67 ns | Az=   83.49 deg | El=   93.51 deg
Path 24: delay= 926.63 ns | Az=   58.12 deg | El=   92.31 deg
Path 25: delay= 882.14 ns | Az=   55.69 deg | El=   88.03 deg
Path 26: delay=   0.23 ns | Az=   49.26 deg | El=   84.70 deg
Path 27: delay= 337.12 ns | Az=   41.76 deg | El=   87.05 deg
Path 28: delay= 664.85 ns | Az=  136.24 deg | El=   92.82 deg
Path 29: delay= 141.46 ns | Az=  120.53 deg | El=   85.83 deg
Path 30: delay= 337.12 ns | Az=   41.76 deg | El=   87.05 deg
Path 31: delay= 515.67 ns | Az=  -91.95 deg | El=   87.33 deg
Path 32: delay=  62.42 ns | Az=   38.80 deg | El=   85.26 deg
Path 33: delay= 388.05 ns | Az= -141.71 deg | El=   86.96 deg
Path 34: delay= 388.64 ns | Az= -141.71 deg | El=   93.67 deg
Path 35: delay= 979.25 ns | Az=  154.29 deg | El=   92.23 deg
Path 36: delay=  53.86 ns | Az=   81.31 deg | El=  103.62 deg
Path 37: delay= 147.56 ns | Az=   32.99 deg | El=   85.87 deg
Path 38: delay= 148.36 ns | Az=   32.99 deg | El=   94.99 deg
Path 39: delay= 984.67 ns | Az=   59.46 deg | El=   88.16 deg
Path 40: delay= 282.42 ns | Az=    1.60 deg | El=   86.56 deg
Path 41: delay= 217.90 ns | Az=  170.35 deg | El=   94.52 deg
Path 42: delay= 397.16 ns | Az=   82.40 deg | El=   78.86 deg
Path 43: delay= 389.12 ns | Az=   83.49 deg | El=   87.27 deg
Path 44: delay= 985.03 ns | Az=   59.46 deg | El=   92.22 deg
Path 45: delay= 692.28 ns | Az=  177.21 deg | El=   76.76 deg
Path 46: delay=1278.06 ns | Az= -129.86 deg | El=   88.46 deg
Path 47: delay= 576.04 ns | Az=   54.47 deg | El=   87.48 deg
Path 48: delay= 576.52 ns | Az=   54.47 deg | El=   93.04 deg
Path 49: delay= 252.17 ns | Az=  -48.17 deg | El=   86.42 deg
Path 50: delay=   5.92 ns | Az=   42.26 deg | El=   84.76 deg
Path 51: delay= 900.33 ns | Az=   57.04 deg | El=   88.05 deg
Path 52: delay= 900.71 ns | Az=   57.04 deg | El=   92.35 deg
Path 53: delay=  96.53 ns | Az=   95.14 deg | El=   85.52 deg
Path 54: delay= 953.76 ns | Az=   58.84 deg | El=   88.12 deg
Path 55: delay= 954.12 ns | Az=   58.84 deg | El=   92.26 deg
Path 56: delay= 530.78 ns | Az=  -74.73 deg | El=   87.37 deg
Path 57: delay=  71.45 ns | Az=   73.75 deg | El=   85.34 deg
Path 58: delay= 426.01 ns | Az=  -74.73 deg | El=   93.53 deg
Path 59: delay= 518.76 ns | Az= -136.63 deg | El=   87.34 deg
Path 60: delay= 519.27 ns | Az= -136.63 deg | El=   93.21 deg
Path 61: delay= 158.35 ns | Az=  -48.17 deg | El=   85.93 deg
Path 62: delay= 252.85 ns | Az=  -48.17 deg | El=   94.31 deg
Path 63: delay= 351.30 ns | Az=  124.87 deg | El=   86.83 deg
Path 64: delay= 351.91 ns | Az=  124.87 deg | El=   93.82 deg
Path 65: delay= 405.53 ns | Az=   51.00 deg | El=   87.19 deg
```

```
Path 66: delay= 602.16 ns | Az=  -65.99 deg | El=   87.54 deg
Path 67: delay= 331.96 ns | Az=  -35.68 deg | El=   86.76 deg
Path 68: delay=1936.51 ns | Az=  143.77 deg | El=   88.87 deg
Path 69: delay= 506.89 ns | Az=  -81.69 deg | El=   87.31 deg
Path 70: delay= 511.26 ns | Az=  -79.59 deg | El=   87.32 deg
Path 71: delay= 576.03 ns | Az=   70.19 deg | El=   87.48 deg
Path 72: delay= 530.17 ns | Az=   71.16 deg | El=   87.37 deg
Path 73: delay=1249.59 ns | Az=   20.89 deg | El=   88.44 deg
Path 74: delay= 173.22 ns | Az=   33.26 deg | El=   86.02 deg
Path 75: delay= 173.98 ns | Az=   33.26 deg | El=   94.80 deg
Path 76: delay= 142.27 ns | Az=  120.53 deg | El=   95.03 deg
Path 77: delay= 556.47 ns | Az= -149.25 deg | El=   87.43 deg
Path 78: delay= 522.45 ns | Az=  -86.31 deg | El=   87.35 deg
Path 79: delay=  40.41 ns | Az=   79.85 deg | El=   79.95 deg
Path 80: delay= 410.75 ns | Az=    0.37 deg | El=   87.03 deg
Path 81: delay= 875.98 ns | Az=  132.96 deg | El=   88.02 deg
Path 82: delay= 913.26 ns | Az=  165.98 deg | El=   88.07 deg
Path 83: delay= 915.13 ns | Az=  174.25 deg | El=   88.07 deg
Path 84: delay=   4.60 ns | Az=   43.14 deg | El=   84.74 deg
Path 85: delay= 882.52 ns | Az=   55.69 deg | El=   92.38 deg
Path 86: delay= 670.48 ns | Az=  -51.19 deg | El=   87.68 deg
Path 87: delay= 670.93 ns | Az=  -51.19 deg | El=   92.80 deg
Path 88: delay= 885.23 ns | Az= -119.40 deg | El=   88.03 deg
Path 89: delay= 885.60 ns | Az= -119.40 deg | El=   92.37 deg
Path 90: delay= 873.58 ns | Az=  126.39 deg | El=   88.02 deg
Path 91: delay= 890.20 ns | Az=  136.26 deg | El=   88.04 deg
Path 92: delay= 351.07 ns | Az=  121.55 deg | El=   86.83 deg
Path 93: delay= 411.82 ns | Az=   83.58 deg | El=   87.27 deg
Path 94: delay= 919.61 ns | Az=   55.47 deg | El=   88.08 deg
Path 95: delay= 516.18 ns | Az=  -91.95 deg | El=   93.22 deg
Path 96: delay=1088.00 ns | Az=   56.50 deg | El=   88.28 deg
Path 97: delay= 539.68 ns | Az=  165.98 deg | El=   87.39 deg
Path 98: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 99: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 100: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 101: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 102: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 103: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 104: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 105: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 106: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 107: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 108: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 109: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 110: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 111: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 112: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
Path 113: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
```

```
  Path 114: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 115: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 116: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 117: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 118: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 119: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 120: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 121: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 122: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 123: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 124: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 125: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 126: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 127: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 128: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 129: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 130: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 131: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 132: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 133: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 134: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 135: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 136: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 137: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg
  Path 138: delay=-1000000000.00 ns | Az=    0.00 deg | El=    0.00 deg

=== Mitsuba AoA (UE0) ===
  Path  0: delay= 523.28 ns | Az=   50.19 deg | El=    5.30 deg
  Path  1: delay= 583.21 ns | Az=   55.82 deg | El=   -0.56 deg
  Path  2: delay= 542.87 ns | Az=   55.41 deg | El=   -0.68 deg
  Path  3: delay= 652.88 ns | Az=   66.56 deg | El=   -0.58 deg
  Path  4: delay= 613.09 ns | Az=   54.71 deg | El=   -0.53 deg
  Path  5: delay= 552.88 ns | Az=   52.30 deg | El=   -0.59 deg
  Path  6: delay= 544.80 ns | Az=   54.19 deg | El=   -0.64 deg
  Path  7: delay= 731.94 ns | Az=   55.49 deg | El=   -0.47 deg
  Path  8: delay= 535.74 ns | Az=   53.19 deg | El=   -0.70 deg
  Path  9: delay= 565.38 ns | Az=   51.06 deg | El=   -0.56 deg
  Path 10: delay= 656.77 ns | Az=   56.20 deg | El=   -0.51 deg
  Path 11: delay= 689.96 ns | Az=   66.43 deg | El=   -0.54 deg
  Path 12: delay= 586.80 ns | Az=   62.84 deg | El=   -0.64 deg
  Path 13: delay= 540.63 ns | Az=   61.89 deg | El=   -0.97 deg
  Path 14: delay=1477.08 ns | Az=   58.79 deg | El=   -0.19 deg
  Path 15: delay= 556.06 ns | Az=   52.19 deg | El=   -0.58 deg
  Path 16: delay= 555.65 ns | Az=   50.60 deg | El=   -0.58 deg
  Path 17: delay= 561.88 ns | Az=   52.32 deg | El=   -0.57 deg
  Path 18: delay= 679.29 ns | Az=   60.83 deg | El=   -0.51 deg
  Path 19: delay= 586.18 ns | Az=   53.92 deg | El=   -0.55 deg
  Path 20: delay= 593.44 ns | Az=   50.54 deg | El=   -0.53 deg
```

```
Path 21: delay= 550.43 ns | Az=   54.32 deg | El=   -0.62 deg
Path 22: delay= 608.47 ns | Az=   51.99 deg | El=   -0.52 deg
Path 23: delay= 564.13 ns | Az=   51.41 deg | El=   -0.56 deg
Path 24: delay= 569.09 ns | Az=   52.52 deg | El=   -0.56 deg
Path 25: delay= 564.43 ns | Az=   54.26 deg | El=   -0.58 deg
Path 26: delay= 540.88 ns | Az=   53.48 deg | El=   -0.65 deg
Path 27: delay= 553.57 ns | Az=   67.15 deg | El=   -0.99 deg
Path 28: delay= 682.97 ns | Az=   52.00 deg | El=   -0.48 deg
Path 29: delay= 548.17 ns | Az=   50.92 deg | El=   -0.60 deg
Path 30: delay= 592.69 ns | Az=   51.71 deg | El=   -0.54 deg
Path 31: delay= 590.66 ns | Az=   56.32 deg | El=   -0.56 deg
Path 32: delay= 663.77 ns | Az=   59.32 deg | El=   -0.51 deg
Path 33: delay= 972.40 ns | Az=   59.51 deg | El=   -0.39 deg
Path 34: delay= 563.31 ns | Az=   81.72 deg | El=    9.60 deg
Path 35: delay= 569.30 ns | Az=   53.57 deg | El=   -0.57 deg
Path 36: delay= 671.76 ns | Az=   53.85 deg | El=   -0.49 deg
Path 37: delay= 652.14 ns | Az=   53.35 deg | El=   -0.50 deg
Path 38: delay= 558.47 ns | Az=   60.46 deg | El=   -0.70 deg
Path 39: delay= 604.22 ns | Az=   60.89 deg | El=   -0.58 deg
Path 40: delay= 829.86 ns | Az=   65.77 deg | El=   -0.45 deg
Path 41: delay= 599.02 ns | Az=   51.54 deg | El=   -0.53 deg
Path 42: delay= 912.66 ns | Az=   52.24 deg | El=    1.77 deg
Path 43: delay= 589.88 ns | Az=   51.86 deg | El=   -0.54 deg
Path 44: delay= 677.62 ns | Az=   55.94 deg | El=   -0.49 deg
Path 45: delay= 910.21 ns | Az=   58.85 deg | El=   -0.41 deg
Path 46: delay= 575.95 ns | Az=   59.68 deg | El=   -0.62 deg
Path 47: delay= 566.79 ns | Az=   50.93 deg | El=   -0.56 deg
Path 48: delay= 814.15 ns | Az=   53.04 deg | El=   -0.43 deg
Path 49: delay= 560.27 ns | Az=   62.38 deg | El=   -0.74 deg
Path 50: delay= 553.08 ns | Az=   60.05 deg | El=   -0.72 deg
Path 51: delay= 599.81 ns | Az=   52.17 deg | El=   -0.53 deg
Path 52: delay= 622.95 ns | Az=   51.51 deg | El=   -0.51 deg
Path 53: delay= 618.90 ns | Az=   68.31 deg | El=   -0.65 deg
Path 54: delay= 583.36 ns | Az=   50.83 deg | El=   -0.54 deg
Path 55: delay= 528.39 ns | Az=   50.36 deg | El=   -0.87 deg
Path 56: delay= 905.38 ns | Az=   50.72 deg | El=    1.67 deg
Path 57: delay= 616.08 ns | Az=   62.39 deg | El=   -0.58 deg
Path 58: delay= 599.05 ns | Az=   54.77 deg | El=   -0.54 deg
Path 59: delay= 537.63 ns | Az=   50.85 deg | El=   -0.65 deg
Path 60: delay= 832.99 ns | Az=   56.33 deg | El=   -0.43 deg
Path 61: delay= 539.95 ns | Az=   59.24 deg | El=   -0.85 deg
Path 62: delay= 675.06 ns | Az=   57.40 deg | El=   -0.50 deg
Path 63: delay= 579.20 ns | Az=   53.94 deg | El=   -0.56 deg
Path 64: delay= 580.41 ns | Az=   58.45 deg | El=   -0.59 deg
Path 65: delay= 575.80 ns | Az=   51.17 deg | El=   -0.55 deg
Path 66: delay= 786.01 ns | Az=   65.18 deg | El=   -0.47 deg
Path 67: delay= 570.82 ns | Az=   50.46 deg | El=   -0.55 deg
Path 68: delay= 617.95 ns | Az=   51.43 deg | El=   -0.52 deg
```

```
Path 69: delay= 725.12 ns | Az=   66.33 deg | El=    0.57 deg
Path 70: delay= 608.88 ns | Az=   50.66 deg | El=   -0.52 deg
Path 71: delay= 745.39 ns | Az=   52.38 deg | El=   -0.46 deg
Path 72: delay= 619.42 ns | Az=   57.80 deg | El=   -0.54 deg
Path 73: delay= 704.95 ns | Az=   56.02 deg | El=   -0.48 deg
Path 74: delay= 533.02 ns | Az=   50.69 deg | El=   -0.70 deg
Path 75: delay= 690.17 ns | Az=   59.09 deg | El=   -0.50 deg
Path 76: delay= 557.42 ns | Az=   52.97 deg | El=   -0.58 deg
Path 77: delay= 593.22 ns | Az=   50.52 deg | El=   -0.53 deg
Path 78: delay= 591.10 ns | Az=   55.84 deg | El=   -0.55 deg
Path 79: delay= 585.70 ns | Az=   70.69 deg | El=   -0.82 deg
Path 80: delay= 541.99 ns | Az=   56.43 deg | El=   -0.71 deg
Path 81: delay= 561.56 ns | Az=   50.66 deg | El=   -0.57 deg
Path 82: delay= 561.12 ns | Az=   52.93 deg | El=   -0.58 deg
Path 83: delay= 658.20 ns | Az=   58.88 deg | El=   -0.52 deg
Path 84: delay= 732.48 ns | Az=   53.63 deg | El=   -0.46 deg
Path 85: delay= 599.83 ns | Az=   51.21 deg | El=   -0.53 deg
Path 86: delay= 877.99 ns | Az=   60.95 deg | El=   -0.42 deg
Path 87: delay= 596.97 ns | Az=   55.10 deg | El=   -0.54 deg
Path 88: delay= 584.75 ns | Az=   50.69 deg | El=   -0.54 deg
Path 89: delay= 547.76 ns | Az=   54.79 deg | El=   -0.63 deg
Path 90: delay= 565.52 ns | Az=   52.44 deg | El=   -0.57 deg
Path 91: delay= 905.77 ns | Az=   50.36 deg | El=   -0.04 deg
Path 92: delay= 574.61 ns | Az=   61.60 deg | El=   -0.66 deg
Path 93: delay=1514.37 ns | Az=   61.36 deg | El=   -0.28 deg
Path 94: delay= 591.85 ns | Az=   52.66 deg | El=   -0.54 deg
Path 95: delay= 578.80 ns | Az=   52.43 deg | El=   -0.55 deg
Path 96: delay= 549.35 ns | Az=   51.72 deg | El=   -0.60 deg
Path 97: delay= 667.22 ns | Az=   56.16 deg | El=   -0.50 deg
Path 98: delay= 547.53 ns | Az=   50.81 deg | El=   -0.60 deg
Path 99: delay= 576.90 ns | Az=   53.35 deg | El=   -0.56 deg
Path 100: delay= 541.64 ns | Az=   50.69 deg | El=   -0.62 deg
Path 101: delay= 599.88 ns | Az=   55.18 deg | El=   -0.54 deg
Path 102: delay= 597.20 ns | Az=   56.93 deg | El=   -0.56 deg
Path 103: delay= 718.06 ns | Az=   62.30 deg | El=   -0.50 deg
Path 104: delay= 534.81 ns | Az=   54.16 deg | El=   -0.74 deg
Path 105: delay= 546.90 ns | Az=   56.69 deg | El=   -0.68 deg
Path 106: delay= 572.23 ns | Az=   53.05 deg | El=   -0.56 deg
Path 107: delay= 534.94 ns | Az=   53.56 deg | El=   -0.72 deg
Path 108: delay= 615.64 ns | Az=   53.93 deg | El=   -0.52 deg
Path 109: delay= 707.25 ns | Az= -160.51 deg | El=   -2.93 deg
Path 110: delay= 609.37 ns | Az=   63.73 deg | El=   -0.61 deg
Path 111: delay= 562.05 ns | Az=   50.75 deg | El=   -0.57 deg
Path 112: delay= 745.58 ns | Az=   56.64 deg | El=   -0.46 deg
Path 113: delay= 539.14 ns | Az=   58.26 deg | El=   -0.82 deg
Path 114: delay= 549.60 ns | Az=   53.98 deg | El=   -0.61 deg
Path 115: delay= 579.36 ns | Az=   58.48 deg | El=   -0.60 deg
Path 116: delay= 571.08 ns | Az=   51.39 deg | El=   -0.56 deg
```

```
Path 117: delay= 953.28 ns | Az=   59.93 deg | El=   -0.40 deg
Path 118: delay=1192.98 ns | Az=   60.33 deg | El=   -0.34 deg
Path 119: delay= 607.13 ns | Az=   57.32 deg | El=   -0.55 deg
Path 120: delay= 604.23 ns | Az=   54.29 deg | El=   -0.53 deg
Path 121: delay= 623.17 ns | Az=   61.43 deg | El=   -0.56 deg
Path 122: delay= 600.99 ns | Az=   58.04 deg | El=   -0.56 deg
Path 123: delay= 566.08 ns | Az=   54.59 deg | El=   -0.58 deg
Path 124: delay= 545.10 ns | Az=   51.38 deg | El=   -0.61 deg
Path 125: delay= 558.51 ns | Az=   53.89 deg | El=   -0.59 deg
Path 126: delay= 568.29 ns | Az=   52.01 deg | El=   -0.56 deg
Path 127: delay= 574.91 ns | Az=   56.70 deg | El=   -0.58 deg
Path 128: delay= 615.88 ns | Az=   51.59 deg | El=   -0.52 deg
Path 129: delay= 630.62 ns | Az=   53.50 deg | El=   -0.51 deg
Path 130: delay= 673.23 ns | Az=   55.65 deg | El=   -0.50 deg
Path 131: delay= 570.49 ns | Az=   51.35 deg | El=   -0.56 deg
Path 132: delay= 577.14 ns | Az=   59.54 deg | El=   -0.61 deg
Path 133: delay= 629.20 ns | Az=   56.48 deg | El=   -0.52 deg
Path 134: delay= 698.45 ns | Az=   58.64 deg | El=   -0.49 deg
Path 135: delay= 701.14 ns | Az=   53.62 deg | El=   -0.48 deg
Path 136: delay= 589.50 ns | Az=   60.63 deg | El=   -0.60 deg
Path 137: delay= 605.99 ns | Az=   70.14 deg | El=   -0.72 deg
Path 138: delay= 655.77 ns | Az=   69.18 deg | El=    0.12 deg
Path 139: delay= 594.25 ns | Az=   54.97 deg | El=   -0.55 deg
Path 140: delay= 589.91 ns | Az=   50.12 deg | El=   -0.54 deg
Path 141: delay= 592.89 ns | Az=   54.74 deg | El=   -0.55 deg
Path 142: delay= 623.13 ns | Az=   56.44 deg | El=   -0.53 deg
Path 143: delay= 546.26 ns | Az=   50.23 deg | El=   -0.60 deg
Path 144: delay= 581.73 ns | Az=   50.57 deg | El=   -0.54 deg
Path 145: delay= 573.27 ns | Az=   53.95 deg | El=   -0.56 deg
Path 146: delay= 576.09 ns | Az=   55.48 deg | El=   -0.57 deg
Path 147: delay= 554.72 ns | Az=   50.67 deg | El=   -0.58 deg
Path 148: delay= 584.24 ns | Az=   54.97 deg | El=   -0.56 deg
Path 149: delay= 573.97 ns | Az=   53.63 deg | El=   -0.56 deg
Path 150: delay= 680.17 ns | Az=   61.34 deg | El=   -0.51 deg
Path 151: delay= 697.66 ns | Az=   54.46 deg | El=   -0.48 deg
Path 152: delay= 598.53 ns | Az=   56.72 deg | El=   -0.55 deg
Path 153: delay= 788.55 ns | Az=   59.20 deg | El=   -0.45 deg
Path 154: delay= 570.53 ns | Az=   51.29 deg | El=   -0.56 deg
Path 155: delay= 574.94 ns | Az=   50.36 deg | El=   -0.55 deg
Path 156: delay= 658.64 ns | Az=   68.97 deg | El=    2.44 deg
Path 157: delay= 619.35 ns | Az=   58.46 deg | El=   -0.54 deg
Path 158: delay= 579.18 ns | Az=   50.45 deg | El=   -0.55 deg
Path 159: delay= 566.19 ns | Az=   54.42 deg | El=   -0.58 deg
Path 160: delay= 552.67 ns | Az=   69.64 deg | El=   -1.13 deg
Path 161: delay= 608.46 ns | Az=   50.70 deg | El=   -0.52 deg
Path 162: delay= 685.31 ns | Az=   62.21 deg | El=   -0.52 deg
Path 163: delay= 629.64 ns | Az=   63.17 deg | El=   -0.57 deg
Path 164: delay= 908.88 ns | Az=   55.59 deg | El=   -0.41 deg
```

```
Path 165: delay= 544.32 ns | Az=   52.82 deg | El=   -0.62 deg
Path 166: delay= 654.00 ns | Az=   53.21 deg | El=   -0.50 deg
Path 167: delay= 620.33 ns | Az=   54.01 deg | El=   -0.52 deg
Path 168: delay= 581.01 ns | Az=   50.95 deg | El=   -0.54 deg
Path 169: delay= 549.44 ns | Az=   50.12 deg | El=   -0.59 deg
Path 170: delay= 861.81 ns | Az=   63.12 deg | El=   -0.43 deg
Path 171: delay= 630.32 ns | Az=   60.80 deg | El=   -0.55 deg
Path 172: delay= 601.78 ns | Az=   54.75 deg | El=   -0.54 deg
Path 173: delay= 744.67 ns | Az= -152.34 deg | El=   -2.51 deg
Path 174: delay= 541.71 ns | Az=   54.43 deg | El=   -0.66 deg
Path 175: delay= 626.17 ns | Az=   50.93 deg | El=   -0.51 deg
Path 176: delay= 939.08 ns | Az=   64.24 deg | El=    1.67 deg
Path 177: delay= 640.46 ns | Az=   69.63 deg | El=    2.09 deg
Path 178: delay= 596.63 ns | Az=   52.29 deg | El=   -0.53 deg
Path 179: delay= 609.66 ns | Az=   50.53 deg | El=   -0.52 deg
Path 180: delay= 605.52 ns | Az=   53.08 deg | El=   -0.53 deg
Path 181: delay= 655.34 ns | Az=   54.37 deg | El=   -0.50 deg
Path 182: delay= 640.78 ns | Az=   55.73 deg | El=   -0.51 deg
Path 183: delay= 577.24 ns | Az=   54.10 deg | El=   -0.56 deg
Path 184: delay= 592.95 ns | Az=   52.62 deg | El=   -0.54 deg
Path 185: delay= 555.33 ns | Az=   51.56 deg | El=   -0.58 deg
Path 186: delay= 579.28 ns | Az=   56.34 deg | El=   -0.57 deg
Path 187: delay= 563.72 ns | Az=   52.00 deg | El=   -0.57 deg
Path 188: delay= 890.88 ns | Az=   53.85 deg | El=   -0.41 deg
Path 189: delay=1146.90 ns | Az=   57.57 deg | El=   -0.35 deg
Path 190: delay= 591.80 ns | Az=   64.93 deg | El=   -0.66 deg
Path 191: delay= 604.96 ns | Az=   51.10 deg | El=   -0.53 deg
Path 192: delay= 549.55 ns | Az=   51.68 deg | El=   -0.59 deg
Path 193: delay= 673.38 ns | Az=   55.67 deg | El=   -0.50 deg
Path 194: delay= 562.66 ns | Az=   55.55 deg | El=   -0.60 deg
Path 195: delay= 550.73 ns | Az=   51.41 deg | El=   -0.59 deg
Path 196: delay= 632.58 ns | Az=   53.10 deg | El=   -0.51 deg
Path 197: delay= 542.54 ns | Az=   50.33 deg | El=   -0.62 deg
Path 198: delay= 588.28 ns | Az=   52.36 deg | El=   -0.54 deg
Path 199: delay= 788.84 ns | Az=   61.81 deg | El=   -0.46 deg
Path 200: delay= 590.98 ns | Az=   51.18 deg | El=   -0.54 deg
Path 201: delay= 664.07 ns | Az=   62.22 deg | El=   -0.53 deg
Path 202: delay= 544.67 ns | Az=   56.29 deg | El=   -0.68 deg
Path 203: delay= 558.74 ns | Az=   57.92 deg | El=   -0.64 deg
Path 204: delay= 689.08 ns | Az=   54.71 deg | El=   -0.49 deg
Path 205: delay= 594.76 ns | Az=   51.87 deg | El=   -0.53 deg
Path 206: delay= 558.58 ns | Az=   55.76 deg | El=   -0.61 deg
Path 207: delay= 579.17 ns | Az=   51.59 deg | El=   -0.55 deg
Path 208: delay= 589.76 ns | Az=   67.26 deg | El=   -0.72 deg
Path 209: delay=1255.54 ns | Az=   59.53 deg | El=   -0.33 deg
Path 210: delay= 596.73 ns | Az=   52.24 deg | El=   -0.53 deg
Path 211: delay= 563.88 ns | Az=   52.49 deg | El=   -0.57 deg
Path 212: delay= 604.98 ns | Az=   62.53 deg | El=   -0.60 deg
```

```
Path 213: delay= 601.55 ns | Az=   54.31 deg | El=   -0.54 deg
Path 214: delay= 677.32 ns | Az=   58.61 deg | El=   -0.50 deg
Path 215: delay= 665.69 ns | Az=   52.23 deg | El=   -0.49 deg
Path 216: delay= 604.29 ns | Az=   52.50 deg | El=   -0.53 deg
Path 217: delay= 661.92 ns | Az=   51.33 deg | El=   -0.49 deg
Path 218: delay=1651.41 ns | Az=   63.11 deg | El=   -0.27 deg
Path 219: delay= 683.44 ns | Az=   50.25 deg | El=   -0.48 deg
Path 220: delay= 619.57 ns | Az=   50.23 deg | El=   -0.52 deg
Path 221: delay= 536.72 ns | Az=   57.38 deg | El=   -0.83 deg
Path 222: delay= 532.17 ns | Az=   51.48 deg | El=   -0.73 deg
Path 223: delay= 735.66 ns | Az=   53.87 deg | El=   -0.46 deg
Path 224: delay= 651.61 ns | Az=   50.25 deg | El=   -0.50 deg
Path 225: delay= 699.14 ns | Az=   68.05 deg | El=    1.38 deg
Path 226: delay= 575.76 ns | Az=   51.54 deg | El=   -0.55 deg
Path 227: delay= 690.17 ns | Az=   59.31 deg | El=   -0.50 deg
Path 228: delay= 636.05 ns | Az=   50.61 deg | El=   -0.51 deg
Path 229: delay= 578.32 ns | Az=   57.86 deg | El=   -0.59 deg
Path 230: delay= 592.21 ns | Az=   52.87 deg | El=   -0.54 deg
Path 231: delay= 662.40 ns | Az=   51.54 deg | El=   -0.49 deg
Path 232: delay= 582.56 ns | Az=   54.96 deg | El=   -0.56 deg
Path 233: delay= 724.14 ns | Az=   66.37 deg | El=   -0.19 deg
Path 234: delay= 818.39 ns | Az=   56.84 deg | El=   -0.44 deg
Path 235: delay= 558.76 ns | Az=   51.94 deg | El=   -0.57 deg
Path 236: delay= 604.82 ns | Az=   54.80 deg | El=   -0.54 deg
Path 237: delay= 571.00 ns | Az=   51.73 deg | El=   -0.56 deg
Path 238: delay= 586.90 ns | Az=   50.27 deg | El=   -0.54 deg
Path 239: delay= 737.34 ns | Az=   52.42 deg | El=   -0.46 deg
Path 240: delay= 577.46 ns | Az=   54.09 deg | El=   -0.56 deg
Path 241: delay= 750.99 ns | Az=   61.94 deg | El=   -0.48 deg
Path 242: delay= 621.99 ns | Az=   50.22 deg | El=   -0.51 deg
Path 243: delay= 633.85 ns | Az=   55.70 deg | El=   -0.52 deg
Path 244: delay= 801.35 ns | Az=   52.90 deg | El=   -0.44 deg
Path 245: delay= 730.37 ns | Az=   53.06 deg | El=   -0.46 deg
Path 246: delay= 624.59 ns | Az=   53.39 deg | El=   -0.52 deg
Path 247: delay= 557.64 ns | Az=   50.68 deg | El=   -0.57 deg
Path 248: delay= 756.73 ns | Az=   50.89 deg | El=   -0.45 deg
Path 249: delay= 760.02 ns | Az=   51.70 deg | El=   -0.45 deg
Path 250: delay= 566.35 ns | Az=   50.14 deg | El=   -0.56 deg
Path 251: delay= 634.67 ns | Az=   53.32 deg | El=   -0.51 deg
Path 252: delay= 570.62 ns | Az=   55.89 deg | El=   -0.58 deg
Path 253: delay= 573.96 ns | Az=   55.11 deg | El=   -0.57 deg
Path 254: delay= 680.25 ns | Az=   63.37 deg | El=   -0.53 deg
Path 255: delay= 557.09 ns | Az=   52.72 deg | El=   -0.58 deg
Path 256: delay= 565.70 ns | Az=   56.78 deg | El=   -0.60 deg
Path 257: delay= 554.51 ns | Az=   51.03 deg | El=   -0.58 deg
Path 258: delay= 733.71 ns | Az=   50.72 deg | El=   -0.46 deg
Path 259: delay= 594.06 ns | Az=   53.11 deg | El=   -0.54 deg
Path 260: delay= 678.80 ns | Az=   56.89 deg | El=   -0.50 deg
```

```
Path 261: delay= 611.16 ns | Az=   58.13 deg | El=   -0.55 deg
Path 262: delay= 582.42 ns | Az=   52.92 deg | El=   -0.55 deg
Path 263: delay= 575.26 ns | Az=   55.18 deg | El=   -0.57 deg
Path 264: delay= 708.80 ns | Az=   53.83 deg | El=   -0.47 deg
Path 265: delay= 539.85 ns | Az=   54.00 deg | El=   -0.67 deg
Path 266: delay= 531.46 ns | Az=   52.05 deg | El=   -0.76 deg
Path 267: delay= 612.00 ns | Az=   52.68 deg | El=   -0.52 deg
Path 268: delay= 569.81 ns | Az=   50.33 deg | El=   -0.56 deg
Path 269: delay= 553.05 ns | Az=   51.72 deg | El=   -0.59 deg
Path 270: delay= 532.39 ns | Az=   51.39 deg | El=   -0.72 deg
Path 271: delay= 788.63 ns | Az=   56.69 deg | El=   -0.45 deg
Path 272: delay= 546.01 ns | Az=   51.46 deg | El=   -0.60 deg
Path 273: delay= 684.18 ns | Az=   52.63 deg | El=   -0.48 deg
Path 274: delay= 617.24 ns | Az=   58.04 deg | El=   -0.54 deg
Path 275: delay= 592.70 ns | Az=   50.54 deg | El=   -0.53 deg
Path 276: delay= 542.07 ns | Az=   54.75 deg | El=   -0.67 deg
Path 277: delay= 715.29 ns | Az=   66.43 deg | El=    1.98 deg
Path 278: delay= 570.29 ns | Az=   50.59 deg | El=   -0.55 deg
Path 279: delay= 553.53 ns | Az=   51.16 deg | El=   -0.58 deg
Path 280: delay= 541.44 ns | Az=   51.72 deg | El=   -0.63 deg
Path 281: delay= 586.10 ns | Az=   52.03 deg | El=   -0.54 deg
Path 282: delay= 844.92 ns | Az=   61.15 deg | El=   -0.43 deg
Path 283: delay= 615.04 ns | Az=   53.85 deg | El=   -0.52 deg
Path 284: delay= 584.40 ns | Az=   56.32 deg | El=   -0.57 deg
Path 285: delay= 581.79 ns | Az=   52.08 deg | El=   -0.55 deg
Path 286: delay= 811.10 ns | Az=   56.79 deg | El=   -0.44 deg
Path 287: delay= 580.71 ns | Az=   52.23 deg | El=   -0.55 deg
Path 288: delay= 595.96 ns | Az=   54.30 deg | El=   -0.54 deg
Path 289: delay= 908.24 ns | Az=   51.09 deg | El=    0.86 deg
Path 290: delay= 785.78 ns | Az=   55.21 deg | El=   -0.45 deg
Path 291: delay= 654.77 ns | Az=   69.10 deg | El=    2.24 deg
Path 292: delay= 748.31 ns | Az=   60.10 deg | El=   -0.47 deg
Path 293: delay= 567.21 ns | Az=   52.67 deg | El=   -0.56 deg
Path 294: delay= 671.55 ns | Az=   62.21 deg | El=   -0.53 deg
Path 295: delay= 591.85 ns | Az=   60.45 deg | El=   -0.60 deg
Path 296: delay= 584.12 ns | Az=   54.56 deg | El=   -0.55 deg
Path 297: delay= 537.62 ns | Az=   54.96 deg | El=   -0.72 deg
Path 298: delay= 597.07 ns | Az=   52.30 deg | El=   -0.53 deg
Path 299: delay= 704.71 ns | Az=   64.76 deg | El=   -0.52 deg
Path 300: delay= 588.63 ns | Az=   52.35 deg | El=   -0.54 deg
Path 301: delay= 541.18 ns | Az=   51.55 deg | El=   -0.63 deg
Path 302: delay= 757.78 ns | Az=   58.33 deg | El=   -0.46 deg
Path 303: delay= 555.89 ns | Az=   53.63 deg | El=   -0.59 deg
Path 304: delay= 545.25 ns | Az=   54.62 deg | El=   -0.64 deg
Path 305: delay= 568.03 ns | Az=   55.07 deg | El=   -0.58 deg
Path 306: delay= 701.89 ns | Az=   56.35 deg | El=   -0.48 deg
Path 307: delay= 809.02 ns | Az=   60.75 deg | El=   -0.45 deg
Path 308: delay= 678.34 ns | Az=   58.25 deg | El=   -0.50 deg
```

```
Path 309: delay= 541.73 ns | Az=   51.73 deg | El=   -0.63 deg
Path 310: delay= 633.96 ns | Az=   70.01 deg | El=    0.82 deg
Path 311: delay= 613.84 ns | Az=   58.46 deg | El=   -0.55 deg
Path 312: delay= 576.56 ns | Az=   55.09 deg | El=   -0.57 deg
Path 313: delay= 625.54 ns | Az=   50.85 deg | El=   -0.51 deg
Path 314: delay= 723.72 ns | Az=   63.10 deg | El=   -0.50 deg
Path 315: delay= 958.41 ns | Az=   64.16 deg | El=   -0.12 deg
Path 316: delay= 602.14 ns | Az=   53.19 deg | El=   -0.53 deg
Path 317: delay= 635.80 ns | Az=   57.59 deg | El=   -0.53 deg
Path 318: delay= 656.06 ns | Az=   69.06 deg | El=    2.17 deg
Path 319: delay= 573.78 ns | Az=   57.49 deg | El=   -0.59 deg
Path 320: delay= 539.77 ns | Az=   50.78 deg | El=   -0.63 deg
Path 321: delay= 564.01 ns | Az= -149.52 deg | El=  -13.75 deg
Path 322: delay= 933.39 ns | Az=   53.90 deg | El=    1.91 deg
Path 323: delay= 634.85 ns | Az=   54.25 deg | El=   -0.51 deg
Path 324: delay= 563.90 ns | Az=   52.49 deg | El=   -0.57 deg
Path 325: delay= 553.60 ns | Az=   60.31 deg | El=   -0.73 deg
Path 326: delay= 577.46 ns | Az=   51.61 deg | El=   -0.55 deg
Path 327: delay= 756.22 ns | Az=   62.99 deg | El=   -0.48 deg
Path 328: delay= 610.12 ns | Az=   53.19 deg | El=   -0.53 deg
Path 329: delay= 589.84 ns | Az=   62.44 deg | El=   -0.63 deg
Path 330: delay= 648.10 ns | Az=   54.67 deg | El=   -0.51 deg
Path 331: delay= 620.36 ns | Az=   53.82 deg | El=   -0.52 deg
Path 332: delay= 533.75 ns | Az=   56.73 deg | El=   -0.88 deg
Path 333: delay= 536.62 ns | Az=   52.05 deg | El=   -0.67 deg
Path 334: delay= 782.38 ns | Az= -151.98 deg | El=   -0.23 deg
Path 335: delay= 558.31 ns | Az=   52.86 deg | El=   -0.58 deg
Path 336: delay= 572.22 ns | Az=   52.61 deg | El=   -0.56 deg
Path 337: delay= 550.92 ns | Az=   52.13 deg | El=   -0.59 deg
Path 338: delay= 705.98 ns | Az=   65.02 deg | El=   -0.52 deg
Path 339: delay= 582.40 ns | Az=   53.89 deg | El=   -0.55 deg
Path 340: delay= 721.60 ns | Az=   52.22 deg | El=   -0.47 deg
Path 341: delay= 548.60 ns | Az=   52.30 deg | El=   -0.60 deg
Path 342: delay= 578.00 ns | Az=   51.45 deg | El=   -0.55 deg
Path 343: delay= 571.45 ns | Az=   51.12 deg | El=   -0.55 deg
Path 344: delay= 577.84 ns | Az=   70.88 deg | El=   -0.87 deg
Path 345: delay= 603.83 ns | Az=   53.79 deg | El=   -0.53 deg
Path 346: delay= 738.67 ns | Az=   56.27 deg | El=   -0.47 deg
Path 347: delay= 556.02 ns | Az=   60.71 deg | El=   -0.72 deg
Path 348: delay= 579.02 ns | Az=   50.59 deg | El=   -0.55 deg
Path 349: delay= 593.42 ns | Az=   54.19 deg | El=   -0.54 deg
Path 350: delay= 599.18 ns | Az=   53.05 deg | El=   -0.53 deg
Path 351: delay= 830.53 ns | Az=   63.43 deg | El=   -0.45 deg
Path 352: delay= 590.23 ns | Az=   59.12 deg | El=   -0.58 deg
Path 353: delay= 630.88 ns | Az=   50.95 deg | El=   -0.51 deg
Path 354: delay= 604.77 ns | Az=   51.11 deg | El=   -0.53 deg
Path 355: delay= 869.55 ns | Az=   50.43 deg | El=   -0.41 deg
Path 356: delay= 599.87 ns | Az=   66.36 deg | El=   -0.67 deg
```

```
Path 357: delay= 553.27 ns | Az=   53.29 deg | El=   -0.60 deg
Path 358: delay= 898.19 ns | Az=   58.64 deg | El=   -0.41 deg
Path 359: delay= 784.15 ns | Az=   50.35 deg | El=   -0.44 deg
Path 360: delay= 531.05 ns | Az=   50.74 deg | El=   -0.75 deg
Path 361: delay= 598.70 ns | Az=   53.25 deg | El=   -0.53 deg
Path 362: delay= 570.72 ns | Az=   57.54 deg | El=   -0.60 deg
Path 363: delay= 582.72 ns | Az=   58.03 deg | El=   -0.58 deg
Path 364: delay= 684.65 ns | Az=   59.59 deg | El=   -0.50 deg
Path 365: delay= 580.58 ns | Az=   53.11 deg | El=   -0.55 deg
Path 366: delay= 557.92 ns | Az=   50.95 deg | El=   -0.57 deg
Path 367: delay= 548.86 ns | Az=   52.27 deg | El=   -0.60 deg
Path 368: delay= 550.66 ns | Az=   51.09 deg | El=   -0.59 deg
Path 369: delay= 614.02 ns | Az=   50.66 deg | El=   -0.52 deg
Path 370: delay= 644.37 ns | Az=   51.37 deg | El=   -0.50 deg
Path 371: delay= 663.77 ns | Az=   52.22 deg | El=   -0.49 deg
Path 372: delay= 648.51 ns | Az=   52.10 deg | El=   -0.50 deg
Path 373: delay= 546.19 ns | Az=   52.75 deg | El=   -0.61 deg
Path 374: delay= 588.84 ns | Az=   51.07 deg | El=   -0.54 deg
Path 375: delay= 545.26 ns | Az=   50.49 deg | El=   -0.60 deg
Path 376: delay= 581.12 ns | Az=   52.37 deg | El=   -0.55 deg
Path 377: delay= 707.82 ns | Az=   54.75 deg | El=   -0.48 deg
Path 378: delay= 563.18 ns | Az=   52.57 deg | El=   -0.57 deg
Path 379: delay= 582.70 ns | Az=   50.22 deg | El=   -0.54 deg
Path 380: delay= 617.98 ns | Az=   55.05 deg | El=   -0.53 deg
Path 381: delay= 603.48 ns | Az=   50.43 deg | El=   -0.53 deg
Path 382: delay= 578.18 ns | Az=   56.14 deg | El=   -0.57 deg
Path 383: delay= 576.77 ns | Az=   50.80 deg | El=   -0.55 deg
Path 384: delay= 535.80 ns | Az=   57.82 deg | El=   -0.87 deg
Path 385: delay=1251.61 ns | Az=   58.02 deg | El=   -0.33 deg
Path 386: delay= 664.76 ns | Az=   67.06 deg | El=   -0.57 deg
Path 387: delay= 567.23 ns | Az=   74.84 deg | El=   -0.47 deg
Path 388: delay= 618.53 ns | Az=   61.29 deg | El=   -0.57 deg
Path 389: delay= 717.31 ns | Az=   67.33 deg | El=    0.05 deg
Path 390: delay= 538.58 ns | Az=   53.19 deg | El=   -0.67 deg
Path 391: delay= 607.51 ns | Az=   59.96 deg | El=   -0.57 deg
Path 392: delay= 718.01 ns | Az=   67.72 deg | El=    1.35 deg
Path 393: delay= 585.70 ns | Az=   51.61 deg | El=   -0.54 deg
Path 394: delay= 602.30 ns | Az=   55.59 deg | El=   -0.54 deg
Path 395: delay= 586.93 ns | Az=   52.94 deg | El=   -0.54 deg
Path 396: delay= 604.64 ns | Az=   54.29 deg | El=   -0.53 deg
Path 397: delay= 690.59 ns | Az=   56.13 deg | El=   -0.49 deg
Path 398: delay= 650.16 ns | Az=   55.10 deg | El=   -0.51 deg
Path 399: delay= 541.90 ns | Az=   50.42 deg | El=   -0.62 deg
Path 400: delay= 596.08 ns | Az=   61.51 deg | El=   -0.60 deg
Path 401: delay= 827.21 ns | Az=   54.30 deg | El=   -0.43 deg
Path 402: delay= 576.43 ns | Az=   52.36 deg | El=   -0.55 deg
Path 403: delay= 659.83 ns | Az=   52.58 deg | El=   -0.50 deg
Path 404: delay= 552.59 ns | Az=   60.16 deg | El=   -0.73 deg
```

```
Path 405: delay= 562.02 ns | Az=   50.28 deg | El=   -0.57 deg
Path 406: delay= 557.99 ns | Az=   57.02 deg | El=   -0.63 deg
Path 407: delay= 594.48 ns | Az=   50.21 deg | El=   -0.53 deg
Path 408: delay= 559.90 ns | Az=   56.94 deg | El=   -0.62 deg
Path 409: delay= 597.98 ns | Az=   51.69 deg | El=   -0.53 deg
Path 410: delay= 608.22 ns | Az=   53.85 deg | El=   -0.53 deg
Path 411: delay= 580.97 ns | Az=   64.54 deg | El=   -0.69 deg
Path 412: delay= 679.31 ns | Az=   61.33 deg | El=   -0.51 deg
Path 413: delay= 812.14 ns | Az=   55.48 deg | El=   -0.44 deg
Path 414: delay= 534.71 ns | Az=   55.80 deg | El=   -0.80 deg
Path 415: delay= 533.09 ns | Az=   53.41 deg | El=   -0.75 deg
Path 416: delay= 650.06 ns | Az=   60.81 deg | El=   -0.53 deg
Path 417: delay= 709.90 ns | Az=   55.11 deg | El=   -0.48 deg
Path 418: delay= 604.56 ns | Az=   55.81 deg | El=   -0.54 deg
Path 419: delay= 573.73 ns | Az=   53.27 deg | El=   -0.56 deg
Path 420: delay= 539.86 ns | Az=   51.42 deg | El=   -0.63 deg
Path 421: delay= 708.54 ns | Az=   51.21 deg | El=   -0.47 deg
Path 422: delay= 919.69 ns | Az=   52.84 deg | El=    1.93 deg
Path 423: delay= 572.18 ns | Az=   54.10 deg | El=   -0.57 deg
Path 424: delay= 597.99 ns | Az=   55.96 deg | El=   -0.55 deg
Path 425: delay= 683.37 ns | Az=   50.94 deg | El=   -0.48 deg
Path 426: delay= 585.40 ns | Az=   51.14 deg | El=   -0.54 deg
Path 427: delay= 629.78 ns | Az=   51.22 deg | El=   -0.51 deg
Path 428: delay= 570.14 ns | Az=   50.46 deg | El=   -0.56 deg
Path 429: delay= 749.01 ns | Az=   65.97 deg | El=   -0.50 deg
Path 430: delay= 712.58 ns | Az=   54.71 deg | El=   -0.47 deg
Path 431: delay= 642.62 ns | Az=   60.07 deg | El=   -0.53 deg
Path 432: delay= 590.55 ns | Az=   68.51 deg | El=   -0.74 deg
Path 433: delay= 607.64 ns | Az=   52.38 deg | El=   -0.53 deg
Path 434: delay= 563.65 ns | Az=   54.42 deg | El=   -0.58 deg
Path 435: delay= 558.13 ns | Az=   51.29 deg | El=   -0.57 deg
Path 436: delay= 796.63 ns | Az=   57.30 deg | El=   -0.44 deg
Path 437: delay= 651.55 ns | Az=   69.29 deg | El=    0.66 deg
Path 438: delay= 552.20 ns | Az=   52.95 deg | El=   -0.60 deg
Path 439: delay= 858.77 ns | Az=   54.68 deg | El=   -0.42 deg
Path 440: delay= 532.59 ns | Az=   55.27 deg | El=   -0.84 deg
Path 441: delay= 609.99 ns | Az=   51.39 deg | El=   -0.52 deg
Path 442: delay= 898.45 ns | Az=   56.22 deg | El=   -0.41 deg
Path 443: delay= 604.19 ns | Az=   52.85 deg | El=   -0.53 deg
Path 444: delay= 664.63 ns | Az=   50.30 deg | El=   -0.49 deg
Path 445: delay= 558.04 ns | Az=   57.06 deg | El=   -0.63 deg
Path 446: delay= 531.61 ns | Az=   58.77 deg | El=   -1.14 deg
Path 447: delay= 586.86 ns | Az=   52.41 deg | El=   -0.54 deg
Path 448: delay= 674.29 ns | Az=   61.44 deg | El=   -0.52 deg
Path 449: delay= 575.27 ns | Az=   53.46 deg | El=   -0.56 deg
Path 450: delay= 528.68 ns | Az=   50.77 deg | El=   -0.85 deg
Path 451: delay= 572.79 ns | Az=   53.74 deg | El=   -0.56 deg
Path 452: delay= 893.54 ns | Az=   64.66 deg | El=   -0.43 deg
```

```
Path 453: delay= 660.85 ns | Az=   62.01 deg | El=   -0.53 deg
Path 454: delay= 549.88 ns | Az=   54.47 deg | El=   -0.62 deg
Path 455: delay= 710.27 ns | Az=   64.17 deg | El=   -0.51 deg
Path 456: delay= 714.68 ns | Az=   66.45 deg | El=    1.48 deg
Path 457: delay= 714.30 ns | Az=   65.61 deg | El=   -0.52 deg
Path 458: delay= 558.28 ns | Az=   52.60 deg | El=   -0.58 deg
Path 459: delay= 630.07 ns | Az=   52.26 deg | El=   -0.51 deg
Path 460: delay= 613.97 ns | Az=   55.33 deg | El=   -0.53 deg
Path 461: delay= 961.89 ns | Az=   64.14 deg | El=    0.50 deg
Path 462: delay= 633.15 ns | Az=   53.65 deg | El=   -0.51 deg
Path 463: delay= 542.79 ns | Az=   51.38 deg | El=   -0.62 deg
Path 464: delay= 697.44 ns | Az=   62.85 deg | El=   -0.51 deg
Path 465: delay= 680.73 ns | Az=   52.88 deg | El=   -0.49 deg
Path 466: delay= 557.49 ns | Az=   58.61 deg | El=   -0.66 deg
Path 467: delay= 862.54 ns | Az=   58.23 deg | El=   -0.42 deg
Path 468: delay= 539.20 ns | Az=   61.95 deg | El=   -1.02 deg
Path 469: delay= 558.52 ns | Az=   52.59 deg | El=   -0.58 deg
Path 470: delay= 552.33 ns | Az=   50.93 deg | El=   -0.58 deg
Path 471: delay= 582.38 ns | Az=   56.16 deg | El=   -0.57 deg
Path 472: delay= 651.51 ns | Az=   63.49 deg | El=   -0.55 deg
Path 473: delay= 553.35 ns | Az=   53.72 deg | El=   -0.60 deg
Path 474: delay= 563.44 ns | Az=   50.79 deg | El=   -0.56 deg
Path 475: delay= 539.70 ns | Az=   54.01 deg | El=   -0.67 deg
Path 476: delay= 735.03 ns | Az=   55.86 deg | El=   -0.47 deg
Path 477: delay= 607.31 ns | Az=   59.74 deg | El=   -0.57 deg
Path 478: delay= 685.88 ns | Az=   55.18 deg | El=   -0.49 deg
Path 479: delay= 604.91 ns | Az=   50.87 deg | El=   -0.53 deg
Path 480: delay= 835.24 ns | Az=   61.68 deg | El=   -0.44 deg
Path 481: delay= 624.64 ns | Az=   60.67 deg | El=   -0.56 deg
Path 482: delay= 556.44 ns | Az=   51.22 deg | El=   -0.58 deg
Path 483: delay= 540.09 ns | Az=   54.74 deg | El=   -0.68 deg
Path 484: delay= 629.74 ns | Az=   58.96 deg | El=   -0.54 deg
Path 485: delay= 727.67 ns | Az=   62.61 deg | El=   -0.49 deg
Path 486: delay= 527.17 ns | Az=   55.42 deg | El=   -1.34 deg
Path 487: delay= 827.39 ns | Az=   61.06 deg | El=   -0.44 deg
Path 488: delay= 567.37 ns | Az=   53.45 deg | El=   -0.57 deg
Path 489: delay= 626.61 ns | Az=   54.84 deg | El=   -0.52 deg
Path 490: delay= 644.45 ns | Az=   55.56 deg | El=   -0.51 deg
Path 491: delay= 653.59 ns | Az=   57.83 deg | El=   -0.51 deg
Path 492: delay= 592.64 ns | Az=   58.82 deg | El=   -0.58 deg
Path 493: delay= 643.00 ns | Az=   57.89 deg | El=   -0.52 deg
Path 494: delay= 660.62 ns | Az=   68.97 deg | El=    0.95 deg
Path 495: delay= 592.07 ns | Az=   50.67 deg | El=   -0.53 deg
Path 496: delay= 576.10 ns | Az=   51.96 deg | El=   -0.55 deg
Path 497: delay= 551.40 ns | Az=   53.04 deg | El=   -0.60 deg
Path 498: delay= 564.64 ns | Az=   51.99 deg | El=   -0.57 deg
Path 499: delay= 758.37 ns | Az=   50.48 deg | El=   -0.45 deg
Path 500: delay= 585.37 ns | Az=   50.67 deg | El=   -0.54 deg
```

```
Path 501: delay= 577.70 ns | Az=   53.81 deg | El=   -0.56 deg
Path 502: delay= 671.08 ns | Az=   56.10 deg | El=   -0.50 deg
Path 503: delay= 798.84 ns | Az=   55.78 deg | El=   -0.44 deg
Path 504: delay= 659.43 ns | Az=   55.50 deg | El=   -0.50 deg
Path 505: delay= 658.93 ns | Az=   52.74 deg | El=   -0.50 deg
Path 506: delay= 620.24 ns | Az=   50.99 deg | El=   -0.52 deg
Path 507: delay= 568.13 ns | Az=   52.47 deg | El=   -0.56 deg
Path 508: delay= 565.02 ns | Az=   50.44 deg | El=   -0.56 deg
Path 509: delay= 587.82 ns | Az=   51.69 deg | El=   -0.54 deg
Path 510: delay= 551.08 ns | Az=   59.45 deg | El=   -0.72 deg
Path 511: delay= 611.07 ns | Az=   55.93 deg | El=   -0.54 deg
Path 512: delay= 825.64 ns | Az=   64.56 deg | El=   -0.45 deg
Path 513: delay= 567.26 ns | Az=   73.51 deg | El=    3.87 deg
Path 514: delay= 564.88 ns | Az=   51.55 deg | El=   -0.56 deg
Path 515: delay= 734.43 ns | Az=   60.67 deg | El=   -0.48 deg
Path 516: delay= 600.65 ns | Az=   52.94 deg | El=   -0.53 deg
Path 517: delay= 603.42 ns | Az=   58.09 deg | El=   -0.56 deg
Path 518: delay= 613.13 ns | Az=   51.29 deg | El=   -0.52 deg
Path 519: delay= 647.63 ns | Az=   54.68 deg | El=   -0.51 deg
Path 520: delay= 545.99 ns | Az=   52.13 deg | El=   -0.61 deg
Path 521: delay= 548.69 ns | Az=   54.37 deg | El=   -0.62 deg
Path 522: delay= 600.86 ns | Az=   60.31 deg | El=   -0.58 deg
Path 523: delay= 647.72 ns | Az=   58.96 deg | El=   -0.52 deg
Path 524: delay= 545.76 ns | Az=   54.17 deg | El=   -0.63 deg
Path 525: delay= 545.52 ns | Az=   50.61 deg | El=   -0.60 deg
Path 526: delay= 580.44 ns | Az=   52.55 deg | El=   -0.55 deg
Path 527: delay= 562.03 ns | Az=   50.15 deg | El=   -0.57 deg
Path 528: delay= 542.24 ns | Az=   50.63 deg | El=   -0.62 deg
Path 529: delay= 612.19 ns | Az=   71.03 deg | El=   -0.71 deg
Path 530: delay= 679.76 ns | Az=   60.34 deg | El=   -0.51 deg
Path 531: delay= 619.14 ns | Az=   50.27 deg | El=   -0.52 deg
Path 532: delay= 640.38 ns | Az=   61.44 deg | El=   -0.55 deg
Path 533: delay= 629.29 ns | Az=   66.13 deg | El=   -0.60 deg
Path 534: delay= 778.78 ns | Az=   60.26 deg | El=   -0.46 deg
Path 535: delay= 657.92 ns | Az=   51.70 deg | El=   -0.50 deg
Path 536: delay= 571.64 ns | Az=   50.64 deg | El=   -0.55 deg
Path 537: delay= 691.73 ns | Az=   51.40 deg | El=   -0.48 deg
Path 538: delay= 538.07 ns | Az=   51.41 deg | El=   -0.65 deg
Path 539: delay= 988.47 ns | Az=   64.02 deg | El=    0.12 deg
Path 540: delay= 652.09 ns | Az=   61.17 deg | El=   -0.53 deg
Path 541: delay= 733.65 ns | Az=   60.83 deg | El=   -0.48 deg
Path 542: delay= 566.11 ns | Az=   50.54 deg | El=   -0.56 deg
Path 543: delay= 547.23 ns | Az=   55.78 deg | El=   -0.65 deg
Path 544: delay= 666.66 ns | Az=   54.86 deg | El=   -0.50 deg
Path 545: delay= 555.74 ns | Az=   54.45 deg | El=   -0.60 deg
Path 546: delay= 542.05 ns | Az=   51.07 deg | El=   -0.62 deg
Path 547: delay= 573.93 ns | Az=   53.65 deg | El=   -0.56 deg
Path 548: delay= 675.80 ns | Az=   57.03 deg | El=   -0.50 deg
```

```
Path 549: delay= 559.97 ns | Az=   51.97 deg | El=   -0.57 deg
Path 550: delay= 629.28 ns | Az=   56.12 deg | El=   -0.52 deg
Path 551: delay= 731.30 ns | Az=   53.18 deg | El=   -0.46 deg
Path 552: delay= 776.66 ns | Az=   56.97 deg | El=   -0.45 deg
Path 553: delay= 608.47 ns | Az=   50.45 deg | El=   -0.52 deg
Path 554: delay= 591.25 ns | Az=   53.88 deg | El=   -0.54 deg
Path 555: delay= 556.98 ns | Az=   52.55 deg | El=   -0.58 deg
Path 556: delay= 634.98 ns | Az=   51.29 deg | El=   -0.51 deg
Path 557: delay= 557.94 ns | Az=   51.92 deg | El=   -0.58 deg
Path 558: delay= 658.65 ns | Az=   57.03 deg | El=   -0.51 deg
Path 559: delay= 688.43 ns | Az=   59.89 deg | El=   -0.50 deg
Path 560: delay= 585.63 ns | Az=   50.86 deg | El=   -0.54 deg
Path 561: delay= 603.15 ns | Az=   52.98 deg | El=   -0.53 deg
Path 562: delay= 584.54 ns | Az=   57.17 deg | El=   -0.57 deg
Path 563: delay= 545.16 ns | Az=   54.49 deg | El=   -0.64 deg
Path 564: delay=1016.53 ns | Az=   58.89 deg | El=   -0.38 deg
Path 565: delay= 554.44 ns | Az=   55.27 deg | El=   -0.61 deg
Path 566: delay= 657.36 ns | Az=   53.15 deg | El=   -0.50 deg
Path 567: delay= 643.01 ns | Az=   61.52 deg | El=   -0.54 deg
Path 568: delay= 600.56 ns | Az=   67.65 deg | El=   -0.69 deg
Path 569: delay= 582.71 ns | Az=   56.23 deg | El=   -0.57 deg
Path 570: delay= 664.30 ns | Az=   68.92 deg | El=    0.05 deg
Path 571: delay= 605.73 ns | Az=   55.77 deg | El=   -0.54 deg
Path 572: delay= 594.56 ns | Az=   60.98 deg | El=   -0.60 deg
Path 573: delay= 686.35 ns | Az=   58.35 deg | El=   -0.50 deg
Path 574: delay= 578.69 ns | Az=   53.79 deg | El=   -0.56 deg
Path 575: delay= 933.27 ns | Az=   56.18 deg | El=   -0.40 deg
Path 576: delay= 555.82 ns | Az=   52.47 deg | El=   -0.58 deg
Path 577: delay= 797.66 ns | Az=   64.29 deg | El=   -0.46 deg
Path 578: delay= 593.62 ns | Az=   50.81 deg | El=   -0.53 deg
Path 579: delay= 570.07 ns | Az=   52.43 deg | El=   -0.56 deg
Path 580: delay= 547.43 ns | Az=   51.01 deg | El=   -0.60 deg
Path 581: delay= 654.73 ns | Az=   50.17 deg | El=   -0.50 deg
Path 582: delay= 706.57 ns | Az=   50.90 deg | El=   -0.47 deg
Path 583: delay= 573.71 ns | Az=   52.56 deg | El=   -0.56 deg
Path 584: delay= 543.23 ns | Az=   51.96 deg | El=   -0.62 deg
Path 585: delay= 575.58 ns | Az=   55.99 deg | El=   -0.58 deg
Path 586: delay= 590.96 ns | Az=   60.44 deg | El=   -0.60 deg
Path 587: delay= 561.09 ns | Az=   61.83 deg | El=   -0.72 deg
Path 588: delay= 554.50 ns | Az=   55.85 deg | El=   -0.62 deg
Path 589: delay= 579.59 ns | Az=   56.68 deg | El=   -0.58 deg
Path 590: delay= 613.62 ns | Az=   53.58 deg | El=   -0.52 deg
Path 591: delay= 880.75 ns | Az=   53.45 deg | El=   -0.41 deg
Path 592: delay= 649.27 ns | Az=   55.63 deg | El=   -0.51 deg
Path 593: delay= 571.21 ns | Az=   60.28 deg | El=   -0.64 deg
Path 594: delay= 600.92 ns | Az=   54.22 deg | El=   -0.54 deg
Path 595: delay= 550.75 ns | Az=   51.97 deg | El=   -0.59 deg
Path 596: delay= 622.70 ns | Az=   54.37 deg | El=   -0.52 deg
```

```
Path 597: delay= 644.73 ns | Az=   56.32 deg | El=   -0.51 deg
Path 598: delay= 576.79 ns | Az=   53.77 deg | El=   -0.56 deg
Path 599: delay= 850.37 ns | Az=   52.99 deg | El=   -0.42 deg
Path 600: delay= 533.77 ns | Az=   53.63 deg | El=   -0.74 deg
Path 601: delay= 803.57 ns | Az=   58.81 deg | El=   -0.45 deg
Path 602: delay= 598.64 ns | Az=   59.41 deg | El=   -0.58 deg
Path 603: delay= 663.50 ns | Az=   50.62 deg | El=   -0.49 deg
Path 604: delay= 558.55 ns | Az=   53.78 deg | El=   -0.59 deg
Path 605: delay= 610.61 ns | Az=   52.98 deg | El=   -0.52 deg
Path 606: delay= 598.80 ns | Az=   51.12 deg | El=   -0.53 deg
Path 607: delay= 644.30 ns | Az=   59.02 deg | El=   -0.53 deg
Path 608: delay= 631.50 ns | Az=   70.18 deg | El=    0.29 deg
Path 609: delay= 576.02 ns | Az=   54.71 deg | El=   -0.56 deg
Path 610: delay= 660.19 ns | Az=   56.45 deg | El=   -0.50 deg
Path 611: delay= 716.95 ns | Az=   67.78 deg | El=    0.06 deg
Path 612: delay= 551.48 ns | Az=   56.60 deg | El=   -0.65 deg
Path 613: delay= 561.29 ns | Az=   52.22 deg | El=   -0.57 deg
Path 614: delay= 554.27 ns | Az=   54.37 deg | El=   -0.60 deg
Path 615: delay= 937.74 ns | Az=   54.19 deg | El=    1.59 deg
Path 616: delay= 546.54 ns | Az=   66.61 deg | El=   -1.10 deg
Path 617: delay= 583.35 ns | Az=   52.55 deg | El=   -0.55 deg
Path 618: delay= 579.98 ns | Az=   50.45 deg | El=   -0.54 deg
Path 619: delay= 604.72 ns | Az=   52.14 deg | El=   -0.53 deg
Path 620: delay= 670.60 ns | Az=   62.89 deg | El=   -0.53 deg
Path 621: delay= 658.37 ns | Az=   61.61 deg | El=   -0.53 deg
Path 622: delay= 654.48 ns | Az=   51.06 deg | El=   -0.50 deg
Path 623: delay= 581.48 ns | Az=   52.90 deg | El=   -0.55 deg
Path 624: delay= 746.88 ns | Az=   52.74 deg | El=   -0.46 deg
Path 625: delay= 596.47 ns | Az=   55.49 deg | El=   -0.55 deg
Path 626: delay= 552.08 ns | Az=   55.15 deg | El=   -0.62 deg
Path 627: delay= 751.60 ns | Az=   60.27 deg | El=   -0.47 deg
Path 628: delay= 572.71 ns | Az=   55.27 deg | El=   -0.57 deg
Path 629: delay= 606.38 ns | Az=   50.22 deg | El=   -0.52 deg
Path 630: delay= 600.10 ns | Az=   54.47 deg | El=   -0.54 deg
Path 631: delay= 796.43 ns | Az=   57.76 deg | El=   -0.45 deg
Path 632: delay= 588.06 ns | Az=   52.90 deg | El=   -0.54 deg
Path 633: delay= 563.36 ns | Az=   53.51 deg | El=   -0.58 deg
Path 634: delay= 553.53 ns | Az=   51.35 deg | El=   -0.58 deg
Path 635: delay= 736.97 ns | Az=  170.83 deg | El=    7.36 deg
Path 636: delay= 631.89 ns | Az=   53.09 deg | El=   -0.51 deg
Path 637: delay= 581.64 ns | Az=   53.21 deg | El=   -0.55 deg
Path 638: delay= 639.81 ns | Az=   53.56 deg | El=   -0.51 deg
Path 639: delay= 554.44 ns | Az=   50.21 deg | El=   -0.58 deg
Path 640: delay= 576.50 ns | Az=   58.66 deg | El=   -0.60 deg
Path 641: delay= 538.59 ns | Az=   61.24 deg | El=   -0.99 deg
Path 642: delay= 591.69 ns | Az=   52.92 deg | El=   -0.54 deg
Path 643: delay= 559.08 ns | Az=   63.63 deg | El=   -0.78 deg
Path 644: delay= 633.00 ns | Az=   65.40 deg | El=   -0.59 deg
```

```
Path 645: delay= 771.46 ns | Az=   52.10 deg | El=   -0.45 deg
Path 646: delay= 697.15 ns | Az=   56.79 deg | El=   -0.49 deg
Path 647: delay= 591.79 ns | Az=   53.81 deg | El=   -0.54 deg
Path 648: delay= 559.38 ns | Az=   55.27 deg | El=   -0.60 deg
Path 649: delay= 568.03 ns | Az=   66.74 deg | El=   -0.81 deg
Path 650: delay= 562.89 ns | Az=   53.25 deg | El=   -0.57 deg
Path 651: delay= 540.37 ns | Az=   53.34 deg | El=   -0.65 deg
Path 652: delay= 529.75 ns | Az=   50.83 deg | El=   -0.80 deg
Path 653: delay= 539.68 ns | Az=   52.17 deg | El=   -0.64 deg
Path 654: delay= 684.56 ns | Az=   61.33 deg | El=   -0.51 deg
Path 655: delay= 659.17 ns | Az=   53.97 deg | El=   -0.50 deg
Path 656: delay= 556.26 ns | Az=   51.83 deg | El=   -0.58 deg
Path 657: delay= 567.72 ns | Az=   55.68 deg | El=   -0.59 deg
Path 658: delay= 846.60 ns | Az=   60.03 deg | El=   -0.43 deg
Path 659: delay= 893.07 ns | Az=   51.20 deg | El=   -0.41 deg
Path 660: delay= 546.75 ns | Az=   51.88 deg | El=   -0.60 deg
Path 661: delay= 558.51 ns | Az=   52.55 deg | El=   -0.58 deg
Path 662: delay=1003.69 ns | Az=   55.63 deg | El=   -0.38 deg
Path 663: delay= 623.40 ns | Az=   52.54 deg | El=   -0.52 deg
Path 664: delay= 557.95 ns | Az=   50.33 deg | El=   -0.57 deg
Path 665: delay= 621.28 ns | Az=   65.08 deg | El=   -0.60 deg
Path 666: delay= 590.55 ns | Az=   51.61 deg | El=   -0.54 deg
Path 667: delay= 548.00 ns | Az=   51.75 deg | El=   -0.60 deg
Path 668: delay= 595.98 ns | Az=   52.57 deg | El=   -0.53 deg
Path 669: delay= 760.60 ns | Az=   50.83 deg | El=   -0.45 deg
Path 670: delay= 557.56 ns | Az=   51.38 deg | El=   -0.57 deg
Path 671: delay= 615.11 ns | Az=   50.32 deg | El=   -0.52 deg
Path 672: delay= 632.85 ns | Az=   54.14 deg | El=   -0.51 deg
Path 673: delay= 595.19 ns | Az=   57.65 deg | El=   -0.56 deg
Path 674: delay= 597.47 ns | Az=   50.68 deg | El=   -0.53 deg
Path 675: delay= 559.68 ns | Az=   52.00 deg | El=   -0.57 deg
Path 676: delay= 578.93 ns | Az=   51.83 deg | El=   -0.55 deg
Path 677: delay= 572.91 ns | Az=   54.31 deg | El=   -0.57 deg
Path 678: delay= 551.18 ns | Az=   52.81 deg | El=   -0.60 deg
Path 679: delay= 564.83 ns | Az=   50.26 deg | El=   -0.56 deg
Path 680: delay= 645.58 ns | Az=   54.24 deg | El=   -0.51 deg
Path 681: delay= 591.85 ns | Az=   53.10 deg | El=   -0.54 deg
Path 682: delay= 797.89 ns | Az=   59.01 deg | El=   -0.45 deg
Path 683: delay= 637.07 ns | Az=   53.88 deg | El=   -0.51 deg
Path 684: delay= 908.72 ns | Az=   62.28 deg | El=   -0.42 deg
Path 685: delay= 614.21 ns | Az=   61.51 deg | El=   -0.58 deg
Path 686: delay= 557.38 ns | Az=   54.98 deg | El=   -0.60 deg
Path 687: delay= 634.43 ns | Az=   53.27 deg | El=   -0.51 deg
Path 688: delay= 738.83 ns | Az=   61.91 deg | El=   -0.48 deg
Path 689: delay= 826.97 ns | Az=   55.83 deg | El=   -0.43 deg
Path 690: delay= 539.52 ns | Az=   56.68 deg | El=   -0.75 deg
Path 691: delay= 614.91 ns | Az=   64.58 deg | El=   -0.61 deg
Path 692: delay= 586.78 ns | Az=   50.60 deg | El=   -0.54 deg
```

```
Path 693: delay= 824.96 ns | Az=   58.29 deg | El=   -0.44 deg
Path 694: delay= 558.56 ns | Az=   52.54 deg | El=   -0.58 deg
Path 695: delay= 536.52 ns | Az=   50.19 deg | El=   -0.66 deg
Path 696: delay= 590.53 ns | Az=   54.20 deg | El=   -0.55 deg
Path 697: delay= 612.80 ns | Az=   52.99 deg | El=   -0.52 deg
Path 698: delay= 598.52 ns | Az=   60.90 deg | El=   -0.59 deg
Path 699: delay= 571.61 ns | Az=   54.35 deg | El=   -0.57 deg
Path 700: delay= 634.05 ns | Az=   54.20 deg | El=   -0.51 deg
Path 701: delay= 563.73 ns | Az=   51.99 deg | El=   -0.57 deg
Path 702: delay= 568.51 ns | Az=   53.52 deg | El=   -0.57 deg
Path 703: delay= 564.89 ns | Az=   50.38 deg | El=   -0.56 deg
Path 704: delay= 593.72 ns | Az=   53.72 deg | El=   -0.54 deg
Path 705: delay= 608.85 ns | Az=   54.65 deg | El=   -0.53 deg
Path 706: delay= 567.67 ns | Az=   51.04 deg | El=   -0.56 deg
Path 707: delay= 560.68 ns | Az=   53.06 deg | El=   -0.58 deg
Path 708: delay= 705.82 ns | Az=   65.94 deg | El=   -0.52 deg
Path 709: delay= 567.86 ns | Az=   51.44 deg | El=   -0.56 deg
Path 710: delay= 630.86 ns | Az=   62.66 deg | El=   -0.57 deg
Path 711: delay= 854.68 ns | Az=   50.72 deg | El=   -0.42 deg
Path 712: delay= 538.23 ns | Az=   55.61 deg | El=   -0.73 deg
Path 713: delay= 565.15 ns | Az=   67.12 deg | El=   -0.85 deg
Path 714: delay= 641.17 ns | Az=   55.62 deg | El=   -0.51 deg
Path 715: delay= 528.17 ns | Az=   50.47 deg | El=   -0.89 deg
Path 716: delay= 568.58 ns | Az=   53.93 deg | El=   -0.57 deg
Path 717: delay= 601.49 ns | Az=   55.91 deg | El=   -0.54 deg
Path 718: delay= 603.39 ns | Az=   51.83 deg | El=   -0.53 deg
Path 719: delay= 555.84 ns | Az=   52.99 deg | El=   -0.59 deg
Path 720: delay= 557.54 ns | Az=   53.16 deg | El=   -0.58 deg
Path 721: delay= 619.87 ns | Az=   58.05 deg | El=   -0.54 deg
Path 722: delay= 682.91 ns | Az=   55.28 deg | El=   -0.49 deg
Path 723: delay= 553.88 ns | Az=   59.09 deg | El=   -0.69 deg
Path 724: delay= 652.09 ns | Az=   52.11 deg | El=   -0.50 deg
Path 725: delay= 648.23 ns | Az=   54.57 deg | El=   -0.51 deg
Path 726: delay= 695.31 ns | Az=   50.91 deg | El=   -0.48 deg
Path 727: delay= 573.79 ns | Az=   57.68 deg | El=   -0.60 deg
Path 728: delay= 567.38 ns | Az=   60.37 deg | El=   -0.66 deg
Path 729: delay= 572.73 ns | Az=   53.34 deg | El=   -0.56 deg
Path 730: delay= 751.98 ns | Az=   59.92 deg | El=   -0.47 deg
Path 731: delay= 571.18 ns | Az=   52.28 deg | El=   -0.56 deg
Path 732: delay= 659.07 ns | Az=   58.52 deg | El=   -0.51 deg
Path 733: delay= 676.09 ns | Az=   55.83 deg | El=   -0.49 deg
Path 734: delay= 719.90 ns | Az=   51.31 deg | El=   -0.47 deg
Path 735: delay= 571.83 ns | Az=   51.70 deg | El=   -0.55 deg
Path 736: delay= 658.26 ns | Az=   51.30 deg | El=   -0.50 deg
Path 737: delay= 621.84 ns | Az=   50.40 deg | El=   -0.51 deg
Path 738: delay= 567.16 ns | Az=   52.06 deg | El=   -0.56 deg
Path 739: delay= 551.67 ns | Az=   54.67 deg | El=   -0.62 deg
Path 740: delay= 713.91 ns | Az=   61.13 deg | El=   -0.49 deg
```

```
Path 741: delay= 856.29 ns | Az=   54.43 deg | El=   -0.42 deg
Path 742: delay= 547.73 ns | Az=   50.62 deg | El=   -0.60 deg
Path 743: delay= 537.16 ns | Az=   51.87 deg | El=   -0.66 deg
Path 744: delay= 590.70 ns | Az=   50.81 deg | El=   -0.54 deg
Path 745: delay= 552.27 ns | Az=   50.83 deg | El=   -0.58 deg
Path 746: delay= 608.25 ns | Az=   50.54 deg | El=   -0.52 deg
Path 747: delay= 540.09 ns | Az=   51.63 deg | El=   -0.63 deg
Path 748: delay= 735.29 ns | Az=   63.84 deg | El=   -0.49 deg
Path 749: delay= 541.75 ns | Az=   59.75 deg | El=   -0.84 deg
Path 750: delay= 759.47 ns | Az=   51.93 deg | El=   -0.45 deg
Path 751: delay= 553.50 ns | Az=   55.80 deg | El=   -0.63 deg
Path 752: delay= 594.88 ns | Az=   52.35 deg | El=   -0.53 deg
Path 753: delay= 550.21 ns | Az=   50.83 deg | El=   -0.59 deg
Path 754: delay= 589.81 ns | Az=   67.68 deg | El=   -0.72 deg
Path 755: delay= 649.43 ns | Az=   52.65 deg | El=   -0.50 deg
Path 756: delay= 785.44 ns | Az=   62.79 deg | El=   -0.46 deg
Path 757: delay= 669.95 ns | Az=   52.06 deg | El=   -0.49 deg
Path 758: delay= 772.21 ns | Az=   62.02 deg | El=   -0.47 deg
Path 759: delay= 588.36 ns | Az=   58.20 deg | El=   -0.58 deg
Path 760: delay= 617.53 ns | Az=   61.08 deg | El=   -0.57 deg
Path 761: delay= 566.24 ns | Az=   51.13 deg | El=   -0.56 deg
Path 762: delay= 535.54 ns | Az=   51.16 deg | El=   -0.67 deg
Path 763: delay= 600.05 ns | Az=   50.73 deg | El=   -0.53 deg
Path 764: delay= 576.88 ns | Az=   53.79 deg | El=   -0.56 deg
Path 765: delay= 601.22 ns | Az=   51.59 deg | El=   -0.53 deg
Path 766: delay= 601.31 ns | Az=   53.83 deg | El=   -0.53 deg
Path 767: delay= 531.27 ns | Az=   59.15 deg | El=   -1.21 deg
Path 768: delay= 867.69 ns | Az=   56.57 deg | El=   -0.42 deg
Path 769: delay= 551.66 ns | Az=   56.06 deg | El=   -0.64 deg
Path 770: delay= 572.68 ns | Az=   57.13 deg | El=   -0.59 deg
Path 771: delay= 865.54 ns | Az=   60.94 deg | El=   -0.43 deg
Path 772: delay= 707.48 ns | Az=   58.80 deg | El=   -0.49 deg
Path 773: delay= 613.48 ns | Az=   51.22 deg | El=   -0.52 deg
Path 774: delay= 572.17 ns | Az=   62.56 deg | El=   -0.68 deg
Path 775: delay=1311.18 ns | Az=   60.44 deg | El=   -0.32 deg
Path 776: delay= 627.99 ns | Az=   54.63 deg | El=   -0.52 deg
Path 777: delay= 628.52 ns | Az=   70.19 deg | El=    2.34 deg
Path 778: delay= 574.08 ns | Az=   58.06 deg | El=   -0.60 deg
Path 779: delay= 557.12 ns | Az=   53.80 deg | El=   -0.59 deg
Path 780: delay= 642.13 ns | Az=   59.09 deg | El=   -0.53 deg
Path 781: delay= 563.99 ns | Az=   76.34 deg | El=    3.15 deg
Path 782: delay= 562.36 ns | Az=   53.13 deg | El=   -0.57 deg
Path 783: delay= 620.63 ns | Az=   57.25 deg | El=   -0.54 deg
Path 784: delay= 558.63 ns | Az=   55.78 deg | El=   -0.61 deg
Path 785: delay= 641.64 ns | Az=   58.03 deg | El=   -0.52 deg
Path 786: delay= 662.48 ns | Az=   60.44 deg | El=   -0.52 deg
Path 787: delay= 547.02 ns | Az=   55.56 deg | El=   -0.65 deg
Path 788: delay= 636.86 ns | Az=   56.90 deg | El=   -0.52 deg
```

```
Path 789: delay= 617.18 ns | Az=   50.87 deg | El=   -0.52 deg
Path 790: delay= 633.74 ns | Az=   57.56 deg | El=   -0.53 deg
Path 791: delay= 541.32 ns | Az=   51.23 deg | El=   -0.62 deg
Path 792: delay= 533.08 ns | Az=   50.52 deg | El=   -0.70 deg
Path 793: delay= 592.53 ns | Az=   58.49 deg | El=   -0.57 deg
Path 794: delay= 573.82 ns | Az=   50.95 deg | El=   -0.55 deg
Path 795: delay= 621.08 ns | Az=   51.76 deg | El=   -0.52 deg
Path 796: delay= 576.67 ns | Az=   53.74 deg | El=   -0.56 deg
Path 797: delay= 539.47 ns | Az=   50.57 deg | El=   -0.63 deg
Path 798: delay= 663.58 ns | Az=   50.24 deg | El=   -0.49 deg
Path 799: delay= 776.54 ns | Az=   55.51 deg | El=   -0.45 deg
Path 800: delay= 850.32 ns | Az=   64.38 deg | El=   -0.44 deg
Path 801: delay= 584.32 ns | Az=   50.59 deg | El=   -0.54 deg
Path 802: delay= 564.61 ns | Az=   53.48 deg | El=   -0.57 deg
Path 803: delay= 629.06 ns | Az=   53.37 deg | El=   -0.51 deg
Path 804: delay= 666.97 ns | Az=   57.54 deg | El=   -0.51 deg
Path 805: delay= 565.25 ns | Az=   51.13 deg | El=   -0.56 deg
Path 806: delay= 582.60 ns | Az=   58.87 deg | El=   -0.59 deg
Path 807: delay= 549.72 ns | Az=   53.54 deg | El=   -0.61 deg
Path 808: delay= 586.18 ns | Az=   50.55 deg | El=   -0.54 deg
Path 809: delay= 589.66 ns | Az=   50.26 deg | El=   -0.54 deg
Path 810: delay= 547.85 ns | Az=   51.75 deg | El=   -0.60 deg
Path 811: delay= 618.43 ns | Az=   53.82 deg | El=   -0.52 deg
Path 812: delay= 543.73 ns | Az=   50.98 deg | El=   -0.61 deg
Path 813: delay= 576.17 ns | Az=   52.12 deg | El=   -0.55 deg
Path 814: delay= 580.83 ns | Az=   50.68 deg | El=   -0.54 deg
Path 815: delay= 561.03 ns | Az=   51.23 deg | El=   -0.57 deg
Path 816: delay= 597.37 ns | Az=   61.39 deg | El=   -0.60 deg
Path 817: delay= 858.30 ns | Az=   63.24 deg | El=   -0.43 deg
Path 818: delay= 663.00 ns | Az=   51.27 deg | El=   -0.49 deg
Path 819: delay= 691.93 ns | Az=   57.07 deg | El=   -0.49 deg
Path 820: delay= 561.63 ns | Az=   53.72 deg | El=   -0.58 deg
Path 821: delay= 559.54 ns | Az=   52.24 deg | El=   -0.57 deg
Path 822: delay= 577.61 ns | Az=   50.38 deg | El=   -0.55 deg
Path 823: delay= 537.75 ns | Az=   52.33 deg | El=   -0.66 deg
Path 824: delay= 629.99 ns | Az=   68.87 deg | El=   -0.64 deg
Path 825: delay= 593.99 ns | Az=   52.09 deg | El=   -0.53 deg
Path 826: delay= 622.99 ns | Az=   68.31 deg | El=   -0.64 deg
Path 827: delay= 580.01 ns | Az=   51.98 deg | El=   -0.55 deg
Path 828: delay= 698.41 ns | Az=   68.08 deg | El=    0.74 deg
Path 829: delay= 569.73 ns | Az=   51.52 deg | El=   -0.56 deg
Path 830: delay= 587.81 ns | Az=   60.28 deg | El=   -0.60 deg
Path 831: delay= 682.66 ns | Az=   66.36 deg | El=   -0.55 deg
Path 832: delay= 581.71 ns | Az=   51.26 deg | El=   -0.54 deg
Path 833: delay= 825.05 ns | Az=   52.79 deg | El=   -0.43 deg
Path 834: delay= 606.16 ns | Az=   52.84 deg | El=   -0.53 deg
Path 835: delay= 604.16 ns | Az=   50.15 deg | El=   -0.53 deg
Path 836: delay= 558.89 ns | Az=   61.48 deg | El=   -0.72 deg
```

```
  Path 837: delay= 552.42 ns | Az=   51.38 deg | El=   -0.59 deg
  Path 838: delay= 643.13 ns | Az=   65.67 deg | El=   -0.58 deg
  Path 839: delay= 549.01 ns | Az=   50.84 deg | El=   -0.59 deg
  Path 840: delay= 574.07 ns | Az=   59.01 deg | El=   -0.61 deg
  Path 841: delay= 603.07 ns | Az=   61.57 deg | El=   -0.59 deg
  Path 842: delay= 650.35 ns | Az=   56.78 deg | El=   -0.51 deg
  Path 843: delay= 639.54 ns | Az=   53.57 deg | El=   -0.51 deg
  Path 844: delay= 552.35 ns | Az=   57.31 deg | El=   -0.66 deg
  Path 845: delay= 559.55 ns | Az=   53.94 deg | El=   -0.59 deg
  Path 846: delay= 534.27 ns | Az=   54.95 deg | El=   -0.78 deg
  Path 847: delay= 669.34 ns | Az=   58.30 deg | El=   -0.51 deg
  Path 848: delay= 607.30 ns | Az=   56.94 deg | El=   -0.55 deg
  Path 849: delay= 668.76 ns | Az=   68.83 deg | El=   -0.45 deg
  Path 850: delay= 613.97 ns | Az=   53.34 deg | El=   -0.52 deg
  Path 851: delay= 625.21 ns | Az=   52.96 deg | El=   -0.52 deg

Expected LOS AoA (BS1→UE0): Az=-129.81 deg, El=-5.30 deg
```

[18]:
```python
#   Channel Gain Calculation and Comparison

#   1. Sionna Channel Gain
# Channel gain = sum of squared path amplitudes (total received power relative␣
↪to transmitted)
# a shape in Sionna 1.2: [rx, rx_ant, tx, tx_ant, paths, time] (6D, no batch␣
↪dim)
sionna_gain_linear = np.sum(np.abs(a[0, :, 0, :, :, 0])**2)  # sum over rx_ant,␣
↪tx_ant, paths
sionna_gain_dB = 10 * np.log10(sionna_gain_linear + 1e-30)

# Per-path gains for UE0, first antenna pair
sionna_path_gains = np.abs(sionna_amp)**2
sionna_path_gains_dB = 10 * np.log10(sionna_path_gains + 1e-30)

print("=== Sionna Channel Gain (UE0) ===")
print(f"  Total channel gain : {sionna_gain_linear:.4e} (linear) |␣
↪{sionna_gain_dB:.2f} dB")
for i, (g, g_dB, tau_i) in enumerate(zip(sionna_path_gains,␣
↪sionna_path_gains_dB, sionna_tau_ns)):
  print(f"  Path {i:2d}: delay={tau_i:7.2f} ns | gain={g:.4e} | {g_dB:.2f} dB")

#   2. Mitsuba Channel Gain
# Use amplitudes from previously computed mi_paths_aoa
mi_path_gains = mi_amps_aoa**2
mi_gain_linear = np.sum(mi_path_gains)
mi_gain_dB = 10 * np.log10(mi_gain_linear + 1e-30)
mi_path_gains_dB = 10 * np.log10(mi_path_gains + 1e-30)
```

```
print("\n=== Mitsuba Channel Gain (UE0) ===")
print(f"  Total channel gain : {mi_gain_linear:.4e} (linear) | {mi_gain_dB:.2f}␣
↪dB")
for i, (g, g_dB, d) in enumerate(zip(mi_path_gains, mi_path_gains_dB,␣
↪mi_delays_aoa)):
  print(f"  Path {i:2d}: delay={d:7.2f} ns | gain={g:.4e} | {g_dB:.2f} dB")

#   3. Plot Comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Sionna
axes[0].stem(sionna_tau_ns, sionna_path_gains_dB, linefmt='C0-',␣
↪markerfmt='C0o', basefmt='k-')
axes[0].axhline(sionna_gain_dB, color='C0', linestyle='--', linewidth=1.5,
        label=f"Total gain: {sionna_gain_dB:.2f} dB")
axes[0].set_title("Sionna RT - Per-Path Channel Gain (UE0)")
axes[0].set_xlabel("Delay [ns]")
axes[0].set_ylabel("Path Gain [dB]")
axes[0].legend()
axes[0].grid(True)
axes[0].set_xlim(left=0)

# Mitsuba
axes[1].stem(mi_delays_aoa, mi_path_gains_dB, linefmt='C1-', markerfmt='C1o',␣
↪basefmt='k-')
axes[1].axhline(mi_gain_dB, color='C1', linestyle='--', linewidth=1.5,
        label=f"Total gain: {mi_gain_dB:.2f} dB")
axes[1].set_title("Mitsuba - Per-Path Channel Gain (UE0)")
axes[1].set_xlabel("Delay [ns]")
axes[1].set_ylabel("Path Gain [dB]")
axes[1].legend()
axes[1].grid(True)
axes[1].set_xlim(left=0)

plt.suptitle("Channel Gain Comparison: Sionna RT vs Mitsuba (UE0)",␣
↪fontsize=13, fontweight='bold')
plt.tight_layout()
# Figure needs to be saved before plt.show() to avoid blank output in some␣
↪environments (e.g. Sionna Playground)
plt.savefig(PICTURES_DIR / "sionna_vs_mitsuba_channel_gain_comparison.png",␣
↪dpi=300)
plt.show()

#   4. Summary Table
print("\n=== Summary ===")
print(f"{'Metric':<35} {'Sionna':>15} {'Mitsuba':>15}")
```

```python
print("-" * 65)
print(f"{'Number of paths':<35} {len(sionna_tau_ns):>15} {len(mi_delays_aoa):
↪>15}")
print(f"{'Total channel gain [linear]':<35} {sionna_gain_linear:>15.4e}␣
↪{mi_gain_linear:>15.4e}")
print(f"{'Total channel gain [dB]':<35} {sionna_gain_dB:>15.2f} {mi_gain_dB:>15.
↪2f}")
print(f"{'Strongest path gain [dB]':<35} {sionna_path_gains_dB.max():>15.2f}␣
↪{mi_path_gains_dB.max():>15.2f}")
print(f"{'Delay of strongest path [ns]':<35} {sionna_tau_ns[np.
↪argmax(sionna_path_gains)]:>15.2f} {mi_delays_aoa[np.argmax(mi_path_gains)]:
↪>15.2f}")
```

```
=== Sionna Channel Gain (UE0) ===
  Total channel gain : 2.6634e-07 (linear) | -65.75 dB
  Path  0: delay=   0.00 ns | gain=1.0720e-08 | -79.70 dB
  Path  1: delay=  39.09 ns | gain=3.4451e-09 | -84.63 dB
  Path  2: delay=  40.04 ns | gain=6.4580e-10 | -91.90 dB
  Path  3: delay= 425.44 ns | gain=3.8234e-18 | -174.18 dB
  Path  4: delay=   1.02 ns | gain=1.7422e-09 | -87.59 dB
  Path  5: delay=  42.47 ns | gain=5.3327e-13 | -122.73 dB
  Path  6: delay=  95.83 ns | gain=4.0575e-12 | -113.92 dB
  Path  7: delay= 217.18 ns | gain=5.6677e-14 | -132.47 dB
  Path  8: delay=  50.58 ns | gain=2.9410e-13 | -125.32 dB
  Path  9: delay=  51.51 ns | gain=1.0800e-14 | -139.67 dB
  Path 10: delay= 926.26 ns | gain=3.0248e-19 | -185.19 dB
  Path 11: delay=  43.42 ns | gain=3.4952e-14 | -134.57 dB
  Path 12: delay= 134.72 ns | gain=2.1823e-16 | -156.61 dB
  Path 13: delay=  40.41 ns | gain=3.0865e-13 | -125.11 dB
  Path 14: delay=  42.37 ns | gain=5.8262e-16 | -152.35 dB
  Path 15: delay= 147.78 ns | gain=1.0647e-13 | -129.73 dB
  Path 16: delay= 148.57 ns | gain=1.4976e-14 | -138.25 dB
  Path 17: delay=   1.02 ns | gain=5.9831e-11 | -102.23 dB
  Path 18: delay=  51.72 ns | gain=1.4965e-15 | -148.25 dB
  Path 19: delay= 531.62 ns | gain=1.1235e-15 | -149.49 dB
  Path 20: delay= 539.50 ns | gain=1.1321e-15 | -149.46 dB
  Path 21: delay=  77.88 ns | gain=3.9434e-16 | -154.04 dB
  Path 22: delay= 389.12 ns | gain=6.7879e-18 | -171.68 dB
  Path 23: delay= 389.67 ns | gain=2.8820e-17 | -165.40 dB
  Path 24: delay= 926.63 ns | gain=2.7214e-20 | -195.65 dB
  Path 25: delay= 882.14 ns | gain=6.5400e-18 | -171.84 dB
  Path 26: delay=   0.23 ns | gain=4.4144e-13 | -123.55 dB
  Path 27: delay= 337.12 ns | gain=5.5287e-19 | -182.57 dB
  Path 28: delay= 664.85 ns | gain=1.2791e-19 | -188.93 dB
  Path 29: delay= 141.46 ns | gain=7.2882e-14 | -131.37 dB
  Path 30: delay= 337.12 ns | gain=5.5551e-19 | -182.55 dB
  Path 31: delay= 515.67 ns | gain=5.4497e-15 | -142.64 dB
```

```
Path 32: delay=  62.42 ns | gain=1.0300e-12 | -119.87 dB
Path 33: delay= 388.05 ns | gain=1.0934e-17 | -169.61 dB
Path 34: delay= 388.64 ns | gain=1.4407e-18 | -178.41 dB
Path 35: delay= 979.25 ns | gain=1.6552e-16 | -157.81 dB
Path 36: delay=  53.86 ns | gain=3.4057e-18 | -174.68 dB
Path 37: delay= 147.56 ns | gain=5.0063e-15 | -143.00 dB
Path 38: delay= 148.36 ns | gain=3.9799e-15 | -144.00 dB
Path 39: delay= 984.67 ns | gain=4.4147e-19 | -183.55 dB
Path 40: delay= 282.42 ns | gain=2.0726e-15 | -146.83 dB
Path 41: delay= 217.90 ns | gain=6.5503e-15 | -141.84 dB
Path 42: delay= 397.16 ns | gain=2.6467e-18 | -175.77 dB
Path 43: delay= 389.12 ns | gain=6.8643e-18 | -171.63 dB
Path 44: delay= 985.03 ns | gain=4.0443e-20 | -193.93 dB
Path 45: delay= 692.28 ns | gain=1.0041e-15 | -149.98 dB
Path 46: delay=1278.06 ns | gain=4.9922e-12 | -113.02 dB
Path 47: delay= 576.04 ns | gain=1.1558e-15 | -149.37 dB
Path 48: delay= 576.52 ns | gain=5.0524e-16 | -152.97 dB
Path 49: delay= 252.17 ns | gain=1.3034e-14 | -138.85 dB
Path 50: delay=   5.92 ns | gain=2.9036e-14 | -135.37 dB
Path 51: delay= 900.33 ns | gain=3.5345e-19 | -184.52 dB
Path 52: delay= 900.71 ns | gain=3.0829e-20 | -195.11 dB
Path 53: delay=  96.53 ns | gain=6.0679e-16 | -152.17 dB
Path 54: delay= 953.76 ns | gain=1.1609e-19 | -189.35 dB
Path 55: delay= 954.12 ns | gain=1.0454e-20 | -199.81 dB
Path 56: delay= 530.78 ns | gain=9.6564e-17 | -160.15 dB
Path 57: delay=  71.45 ns | gain=2.5369e-16 | -155.96 dB
Path 58: delay= 426.01 ns | gain=9.4431e-19 | -180.25 dB
Path 59: delay= 518.76 ns | gain=1.3679e-14 | -138.64 dB
Path 60: delay= 519.27 ns | gain=1.8550e-15 | -147.32 dB
Path 61: delay= 158.35 ns | gain=7.0707e-17 | -161.51 dB
Path 62: delay= 252.85 ns | gain=3.8272e-15 | -144.17 dB
Path 63: delay= 351.30 ns | gain=6.7640e-15 | -141.70 dB
Path 64: delay= 351.91 ns | gain=2.3696e-15 | -146.25 dB
Path 65: delay= 405.53 ns | gain=7.7026e-19 | -181.13 dB
Path 66: delay= 602.16 ns | gain=4.1683e-17 | -163.80 dB
Path 67: delay= 331.96 ns | gain=2.2974e-15 | -146.39 dB
Path 68: delay=1936.51 ns | gain=1.9988e-16 | -156.99 dB
Path 69: delay= 506.89 ns | gain=4.2879e-15 | -143.68 dB
Path 70: delay= 511.26 ns | gain=1.6218e-15 | -147.90 dB
Path 71: delay= 576.03 ns | gain=4.1449e-16 | -153.82 dB
Path 72: delay= 530.17 ns | gain=4.0615e-16 | -153.91 dB
Path 73: delay=1249.59 ns | gain=1.0916e-15 | -149.62 dB
Path 74: delay= 173.22 ns | gain=3.0516e-15 | -145.15 dB
Path 75: delay= 173.98 ns | gain=2.7143e-15 | -145.66 dB
Path 76: delay= 142.27 ns | gain=3.5978e-14 | -134.44 dB
Path 77: delay= 556.47 ns | gain=6.7509e-18 | -171.71 dB
Path 78: delay= 522.45 ns | gain=2.1229e-16 | -156.73 dB
Path 79: delay=  40.41 ns | gain=3.0983e-13 | -125.09 dB
```

```
Path 80: delay= 410.75 ns | gain=4.1676e-15 | -143.80 dB
Path 81: delay= 875.98 ns | gain=4.4474e-14 | -133.52 dB
Path 82: delay= 913.26 ns | gain=2.5555e-15 | -145.93 dB
Path 83: delay= 915.13 ns | gain=1.1901e-11 | -109.24 dB
Path 84: delay=   4.60 ns | gain=1.0604e-14 | -139.75 dB
Path 85: delay= 882.52 ns | gain=5.6366e-19 | -182.49 dB
Path 86: delay= 670.48 ns | gain=4.6108e-16 | -153.36 dB
Path 87: delay= 670.93 ns | gain=7.5049e-16 | -151.25 dB
Path 88: delay= 885.23 ns | gain=5.0759e-15 | -142.94 dB
Path 89: delay= 885.60 ns | gain=5.5568e-15 | -142.55 dB
Path 90: delay= 873.58 ns | gain=6.5388e-14 | -131.85 dB
Path 91: delay= 890.20 ns | gain=7.6179e-19 | -181.18 dB
Path 92: delay= 351.07 ns | gain=7.0647e-15 | -141.51 dB
Path 93: delay= 411.82 ns | gain=8.1348e-19 | -180.90 dB
Path 94: delay= 919.61 ns | gain=2.2004e-20 | -196.58 dB
Path 95: delay= 516.18 ns | gain=2.2706e-15 | -146.44 dB
Path 96: delay=1088.00 ns | gain=1.3674e-20 | -198.64 dB
Path 97: delay= 539.68 ns | gain=8.9140e-16 | -150.50 dB
Path 98: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 99: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 100: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 101: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 102: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 103: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 104: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 105: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 106: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 107: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 108: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 109: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 110: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 111: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 112: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 113: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 114: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 115: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 116: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 117: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 118: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 119: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 120: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 121: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 122: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 123: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 124: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 125: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 126: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
Path 127: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
```

```
  Path 128: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 129: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 130: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 131: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 132: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 133: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 134: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 135: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 136: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 137: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB
  Path 138: delay=-1000000000.00 ns | gain=0.0000e+00 | -300.00 dB

=== Mitsuba Channel Gain (UE0) ===
  Total channel gain : 4.0675e-05 (linear) | -43.91 dB
  Path  0: delay= 523.28 ns | gain=4.0633e-05 | -43.91 dB
  Path  1: delay= 583.21 ns | gain=4.2700e-12 | -113.70 dB
  Path  2: delay= 542.87 ns | gain=1.3880e-12 | -118.58 dB
  Path  3: delay= 652.88 ns | gain=2.4366e-13 | -126.13 dB
  Path  4: delay= 613.09 ns | gain=4.0233e-12 | -113.95 dB
  Path  5: delay= 552.88 ns | gain=8.2432e-12 | -110.84 dB
  Path  6: delay= 544.80 ns | gain=2.5759e-12 | -115.89 dB
  Path  7: delay= 731.94 ns | gain=3.1490e-13 | -125.02 dB
  Path  8: delay= 535.74 ns | gain=1.2851e-12 | -118.91 dB
  Path  9: delay= 565.38 ns | gain=1.4240e-11 | -108.46 dB
  Path 10: delay= 656.77 ns | gain=1.2895e-12 | -118.90 dB
  Path 11: delay= 689.96 ns | gain=1.7591e-13 | -127.55 dB
  Path 12: delay= 586.80 ns | gain=5.7279e-13 | -122.42 dB
  Path 13: delay= 540.63 ns | gain=3.2004e-13 | -124.95 dB
  Path 14: delay=1477.08 ns | gain=8.2398e-11 | -100.84 dB
  Path 15: delay= 556.06 ns | gain=9.6619e-12 | -110.15 dB
  Path 16: delay= 555.65 ns | gain=1.2220e-11 | -109.13 dB
  Path 17: delay= 561.88 ns | gain=1.0920e-11 | -109.62 dB
  Path 18: delay= 679.29 ns | gain=4.3804e-13 | -123.58 dB
  Path 19: delay= 586.18 ns | gain=7.3726e-12 | -111.32 dB
  Path 20: delay= 593.44 ns | gain=1.0804e-11 | -109.66 dB
  Path 21: delay= 550.43 ns | gain=3.6588e-12 | -114.37 dB
  Path 22: delay= 608.47 ns | gain=6.8543e-12 | -111.64 dB
  Path 23: delay= 564.13 ns | gain=1.3470e-11 | -108.71 dB
  Path 24: delay= 569.09 ns | gain=1.1282e-11 | -109.48 dB
  Path 25: delay= 564.43 ns | gain=6.3103e-12 | -112.00 dB
  Path 26: delay= 540.88 ns | gain=2.2738e-12 | -116.43 dB
  Path 27: delay= 553.57 ns | gain=2.5888e-13 | -125.87 dB
  Path 28: delay= 682.97 ns | gain=1.0602e-12 | -119.75 dB
  Path 29: delay= 548.17 ns | gain=8.2357e-12 | -110.84 dB
  Path 30: delay= 592.69 ns | gain=1.0120e-11 | -109.95 dB
  Path 31: delay= 590.66 ns | gain=3.5730e-12 | -114.47 dB
  Path 32: delay= 663.77 ns | gain=6.9987e-13 | -121.55 dB
  Path 33: delay= 972.40 ns | gain=9.2507e-15 | -140.34 dB
```

```
Path 34: delay= 563.31 ns | gain=5.7874e-10 | -92.38 dB
Path 35: delay= 569.30 ns | gain=8.4635e-12 | -110.72 dB
Path 36: delay= 671.76 ns | gain=1.2248e-12 | -119.12 dB
Path 37: delay= 652.14 ns | gain=2.0242e-12 | -116.94 dB
Path 38: delay= 558.47 ns | gain=6.4521e-13 | -121.90 dB
Path 39: delay= 604.22 ns | gain=9.4982e-13 | -120.22 dB
Path 40: delay= 829.86 ns | gain=3.5951e-14 | -134.44 dB
Path 41: delay= 599.02 ns | gain=8.9691e-12 | -110.47 dB
Path 42: delay= 912.66 ns | gain=3.9962e-09 | -83.98 dB
Path 43: delay= 589.88 ns | gain=1.0495e-11 | -109.79 dB
Path 44: delay= 677.62 ns | gain=8.7515e-13 | -120.58 dB
Path 45: delay= 910.21 ns | gain=1.9158e-14 | -137.18 dB
Path 46: delay= 575.95 ns | gain=1.1658e-12 | -119.33 dB
Path 47: delay= 566.79 ns | gain=1.4534e-11 | -108.38 dB
Path 48: delay= 814.15 ns | gain=8.4561e-14 | -130.73 dB
Path 49: delay= 560.27 ns | gain=4.4321e-13 | -123.53 dB
Path 50: delay= 553.08 ns | gain=6.0014e-13 | -122.22 dB
Path 51: delay= 599.81 ns | gain=8.2233e-12 | -110.85 dB
Path 52: delay= 622.95 ns | gain=4.9196e-12 | -113.08 dB
Path 53: delay= 618.90 ns | gain=2.2824e-13 | -126.42 dB
Path 54: delay= 583.36 ns | gain=1.2975e-11 | -108.87 dB
Path 55: delay= 528.39 ns | gain=4.7992e-13 | -123.19 dB
Path 56: delay= 905.38 ns | gain=4.7066e-09 | -83.27 dB
Path 57: delay= 616.08 ns | gain=6.4519e-13 | -121.90 dB
Path 58: delay= 599.05 ns | gain=5.0442e-12 | -112.97 dB
Path 59: delay= 537.63 ns | gain=2.8050e-12 | -115.52 dB
Path 60: delay= 832.99 ns | gain=5.7799e-14 | -132.38 dB
Path 61: delay= 539.95 ns | gain=4.0931e-13 | -123.88 dB
Path 62: delay= 675.06 ns | gain=7.6676e-13 | -121.15 dB
Path 63: delay= 579.20 ns | gain=7.7059e-12 | -111.13 dB
Path 64: delay= 580.41 ns | gain=1.7894e-12 | -117.47 dB
Path 65: delay= 575.80 ns | gain=1.3903e-11 | -108.57 dB
Path 66: delay= 786.01 ns | gain=6.3872e-14 | -131.95 dB
Path 67: delay= 570.82 ns | gain=1.4985e-11 | -108.24 dB
Path 68: delay= 617.95 ns | gain=5.6400e-12 | -112.49 dB
Path 69: delay= 725.12 ns | gain=3.0701e-10 | -95.13 dB
Path 70: delay= 608.88 ns | gain=7.4031e-12 | -111.31 dB
Path 71: delay= 745.39 ns | gain=2.8155e-13 | -125.50 dB
Path 72: delay= 619.42 ns | gain=1.8187e-12 | -117.40 dB
Path 73: delay= 704.95 ns | gain=5.0474e-13 | -122.97 dB
Path 74: delay= 533.02 ns | gain=1.2793e-12 | -118.93 dB
Path 75: delay= 690.17 ns | gain=4.6986e-13 | -123.28 dB
Path 76: delay= 557.42 ns | gain=8.1506e-12 | -110.89 dB
Path 77: delay= 593.22 ns | gain=1.0864e-11 | -109.64 dB
Path 78: delay= 591.10 ns | gain=4.1185e-12 | -113.85 dB
Path 79: delay= 585.70 ns | gain=1.9252e-13 | -127.16 dB
Path 80: delay= 541.99 ns | gain=9.1329e-13 | -120.39 dB
Path 81: delay= 561.56 ns | gain=1.4066e-11 | -108.52 dB
```

```
Path 82: delay= 561.12 ns | gain=9.1229e-12 | -110.40 dB
Path 83: delay= 658.20 ns | gain=8.2163e-13 | -120.85 dB
Path 84: delay= 732.48 ns | gain=3.4545e-13 | -124.62 dB
Path 85: delay= 599.83 ns | gain=9.0354e-12 | -110.44 dB
Path 86: delay= 877.99 ns | gain=2.6172e-14 | -135.82 dB
Path 87: delay= 596.97 ns | gain=4.7723e-12 | -113.21 dB
Path 88: delay= 584.75 ns | gain=1.2768e-11 | -108.94 dB
Path 89: delay= 547.76 ns | gain=2.5651e-12 | -115.91 dB
Path 90: delay= 565.52 ns | gain=1.1180e-11 | -109.52 dB
Path 91: delay= 905.77 ns | gain=4.5295e-09 | -83.44 dB
Path 92: delay= 574.61 ns | gain=6.7770e-13 | -121.69 dB
Path 93: delay=1514.37 ns | gain=1.2919e-16 | -158.89 dB
Path 94: delay= 591.85 ns | gain=8.9631e-12 | -110.48 dB
Path 95: delay= 578.80 ns | gain=1.1203e-11 | -109.51 dB
Path 96: delay= 549.35 ns | gain=7.8298e-12 | -111.06 dB
Path 97: delay= 667.22 ns | gain=1.0521e-12 | -119.78 dB
Path 98: delay= 547.53 ns | gain=7.9672e-12 | -110.99 dB
Path 99: delay= 576.90 ns | gain=9.1322e-12 | -110.39 dB
Path 100: delay= 541.64 ns | gain=4.7554e-12 | -113.23 dB
Path 101: delay= 599.88 ns | gain=4.4979e-12 | -113.47 dB
Path 102: delay= 597.20 ns | gain=2.8729e-12 | -115.42 dB
Path 103: delay= 718.06 ns | gain=2.0688e-13 | -126.84 dB
Path 104: delay= 534.81 ns | gain=8.5322e-13 | -120.69 dB
Path 105: delay= 546.90 ns | gain=1.1950e-12 | -119.23 dB
Path 106: delay= 572.23 ns | gain=9.9500e-12 | -110.02 dB
Path 107: delay= 534.94 ns | gain=1.0263e-12 | -119.89 dB
Path 108: delay= 615.64 ns | gain=4.4156e-12 | -113.55 dB
Path 109: delay= 707.25 ns | gain=6.9897e-13 | -121.56 dB
Path 110: delay= 609.37 ns | gain=4.9728e-13 | -123.03 dB
Path 111: delay= 562.05 ns | gain=1.4088e-11 | -108.51 dB
Path 112: delay= 745.58 ns | gain=2.2801e-13 | -126.42 dB
Path 113: delay= 539.14 ns | gain=4.6439e-13 | -123.33 dB
Path 114: delay= 549.60 ns | gain=3.9596e-12 | -114.02 dB
Path 115: delay= 579.36 ns | gain=1.7543e-12 | -117.56 dB
Path 116: delay= 571.08 ns | gain=1.3910e-11 | -108.57 dB
Path 117: delay= 953.28 ns | gain=1.1268e-14 | -139.48 dB
Path 118: delay=1192.98 ns | gain=1.1784e-15 | -149.29 dB
Path 119: delay= 607.13 ns | gain=2.3656e-12 | -116.26 dB
Path 120: delay= 604.23 ns | gain=5.1826e-12 | -112.85 dB
Path 121: delay= 623.17 ns | gain=7.6312e-13 | -121.17 dB
Path 122: delay= 600.99 ns | gain=2.0438e-12 | -116.90 dB
Path 123: delay= 566.08 ns | gain=5.7731e-12 | -112.39 dB
Path 124: delay= 545.10 ns | gain=6.1170e-12 | -112.13 dB
Path 125: delay= 558.51 ns | gain=6.1996e-12 | -112.08 dB
Path 126: delay= 568.29 ns | gain=1.2551e-11 | -109.01 dB
Path 127: delay= 574.91 ns | gain=3.0522e-12 | -115.15 dB
Path 128: delay= 615.88 ns | gain=5.8874e-12 | -112.30 dB
Path 129: delay= 630.62 ns | gain=3.3423e-12 | -114.76 dB
```

```
Path 130: delay= 673.23 ns | gain=9.8985e-13 | -120.04 dB
Path 131: delay= 570.49 ns | gain=1.3996e-11 | -108.54 dB
Path 132: delay= 577.14 ns | gain=1.2336e-12 | -119.09 dB
Path 133: delay= 629.20 ns | gain=2.0848e-12 | -116.81 dB
Path 134: delay= 698.45 ns | gain=4.3201e-13 | -123.65 dB
Path 135: delay= 701.14 ns | gain=6.5298e-13 | -121.85 dB
Path 136: delay= 589.50 ns | gain=1.0009e-12 | -120.00 dB
Path 137: delay= 605.99 ns | gain=1.9084e-13 | -127.19 dB
Path 138: delay= 655.77 ns | gain=5.8315e-10 | -92.34 dB
Path 139: delay= 594.25 ns | gain=5.1047e-12 | -112.92 dB
Path 140: delay= 589.91 ns | gain=1.1698e-11 | -109.32 dB
Path 141: delay= 592.89 ns | gain=5.5205e-12 | -112.58 dB
Path 142: delay= 623.13 ns | gain=2.3339e-12 | -116.32 dB
Path 143: delay= 546.26 ns | gain=7.4576e-12 | -111.27 dB
Path 144: delay= 581.73 ns | gain=1.3472e-11 | -108.71 dB
Path 145: delay= 573.27 ns | gain=7.6949e-12 | -111.14 dB
Path 146: delay= 576.09 ns | gain=4.7136e-12 | -113.27 dB
Path 147: delay= 554.72 ns | gain=1.1769e-11 | -109.29 dB
Path 148: delay= 584.24 ns | gain=5.5540e-12 | -112.55 dB
Path 149: delay= 573.97 ns | gain=8.5000e-12 | -110.71 dB
Path 150: delay= 680.17 ns | gain=4.0081e-13 | -123.97 dB
Path 151: delay= 697.66 ns | gain=6.6420e-13 | -121.78 dB
Path 152: delay= 598.53 ns | gain=3.0132e-12 | -115.21 dB
Path 153: delay= 788.55 ns | gain=9.6247e-14 | -130.17 dB
Path 154: delay= 570.53 ns | gain=1.4099e-11 | -108.51 dB
Path 155: delay= 574.94 ns | gain=1.4680e-11 | -108.33 dB
Path 156: delay= 658.64 ns | gain=6.2699e-10 | -92.03 dB
Path 157: delay= 619.35 ns | gain=1.5567e-12 | -118.08 dB
Path 158: delay= 579.18 ns | gain=1.4005e-11 | -108.54 dB
Path 159: delay= 566.19 ns | gain=6.1493e-12 | -112.11 dB
Path 160: delay= 552.67 ns | gain=2.6695e-13 | -125.74 dB
Path 161: delay= 608.46 ns | gain=7.4754e-12 | -111.26 dB
Path 162: delay= 685.31 ns | gain=3.2958e-13 | -124.82 dB
Path 163: delay= 629.64 ns | gain=5.1033e-13 | -122.92 dB
Path 164: delay= 908.88 ns | gain=2.1300e-14 | -136.72 dB
Path 165: delay= 544.32 ns | gain=3.9911e-12 | -113.99 dB
Path 166: delay= 654.00 ns | gain=1.9607e-12 | -117.08 dB
Path 167: delay= 620.33 ns | gain=3.9359e-12 | -114.05 dB
Path 168: delay= 581.01 ns | gain=1.3341e-11 | -108.75 dB
Path 169: delay= 549.44 ns | gain=9.2780e-12 | -110.33 dB
Path 170: delay= 861.81 ns | gain=2.8691e-14 | -135.42 dB
Path 171: delay= 630.32 ns | gain=8.2118e-13 | -120.86 dB
Path 172: delay= 601.78 ns | gain=4.8671e-12 | -113.13 dB
Path 173: delay= 744.67 ns | gain=3.2726e-13 | -124.85 dB
Path 174: delay= 541.71 ns | gain=1.7762e-12 | -117.50 dB
Path 175: delay= 626.17 ns | gain=4.6424e-12 | -113.33 dB
Path 176: delay= 939.08 ns | gain=1.8909e-11 | -107.23 dB
Path 177: delay= 640.46 ns | gain=6.4394e-10 | -91.91 dB
```

```
Path 178: delay= 596.63 ns | gain=8.6650e-12 | -110.62 dB
Path 179: delay= 609.66 ns | gain=7.2763e-12 | -111.38 dB
Path 180: delay= 605.52 ns | gain=6.3502e-12 | -111.97 dB
Path 181: delay= 655.34 ns | gain=1.6894e-12 | -117.72 dB
Path 182: delay= 640.78 ns | gain=1.9167e-12 | -117.17 dB
Path 183: delay= 577.24 ns | gain=7.3884e-12 | -111.31 dB
Path 184: delay= 592.95 ns | gain=8.8498e-12 | -110.53 dB
Path 185: delay= 555.33 ns | gain=1.0781e-11 | -109.67 dB
Path 186: delay= 579.28 ns | gain=3.5745e-12 | -114.47 dB
Path 187: delay= 563.72 ns | gain=1.2086e-11 | -109.18 dB
Path 188: delay= 890.88 ns | gain=2.7743e-14 | -135.57 dB
Path 189: delay=1146.90 ns | gain=1.7885e-15 | -147.48 dB
Path 190: delay= 591.80 ns | gain=3.8716e-13 | -124.12 dB
Path 191: delay= 604.96 ns | gain=8.0284e-12 | -110.95 dB
Path 192: delay= 549.55 ns | gain=7.9881e-12 | -110.98 dB
Path 193: delay= 673.38 ns | gain=9.8404e-13 | -120.07 dB
Path 194: delay= 562.66 ns | gain=3.7607e-12 | -114.25 dB
Path 195: delay= 550.73 ns | gain=8.9895e-12 | -110.46 dB
Path 196: delay= 632.58 ns | gain=3.3417e-12 | -114.76 dB
Path 197: delay= 542.54 ns | gain=5.3244e-12 | -112.74 dB
Path 198: delay= 588.28 ns | gain=1.0028e-11 | -109.99 dB
Path 199: delay= 788.84 ns | gain=7.9890e-14 | -130.98 dB
Path 200: delay= 590.98 ns | gain=1.1019e-11 | -109.58 dB
Path 201: delay= 664.07 ns | gain=4.3233e-13 | -123.64 dB
Path 202: delay= 544.67 ns | gain=1.1759e-12 | -119.30 dB
Path 203: delay= 558.74 ns | gain=1.3866e-12 | -118.58 dB
Path 204: delay= 689.08 ns | gain=7.7924e-13 | -121.08 dB
Path 205: delay= 594.76 ns | gain=9.5219e-12 | -110.21 dB
Path 206: delay= 558.58 ns | gain=3.0462e-12 | -115.16 dB
Path 207: delay= 579.17 ns | gain=1.2814e-11 | -108.92 dB
Path 208: delay= 589.76 ns | gain=2.6849e-13 | -125.71 dB
Path 209: delay=1255.54 ns | gain=7.3170e-16 | -151.36 dB
Path 210: delay= 596.73 ns | gain=8.7033e-12 | -110.60 dB
Path 211: delay= 563.88 ns | gain=1.0822e-11 | -109.66 dB
Path 212: delay= 604.98 ns | gain=6.4422e-13 | -121.91 dB
Path 213: delay= 601.55 ns | gain=5.4059e-12 | -112.67 dB
Path 214: delay= 677.32 ns | gain=6.2309e-13 | -122.05 dB
Path 215: delay= 665.69 ns | gain=1.5816e-12 | -118.01 dB
Path 216: delay= 604.29 ns | gain=7.1199e-12 | -111.48 dB
Path 217: delay= 661.92 ns | gain=1.8040e-12 | -117.44 dB
Path 218: delay=1651.41 ns | gain=5.9399e-17 | -162.26 dB
Path 219: delay= 683.44 ns | gain=1.0866e-12 | -119.64 dB
Path 220: delay= 619.57 ns | gain=5.6124e-12 | -112.51 dB
Path 221: delay= 536.72 ns | gain=4.6997e-13 | -123.28 dB
Path 222: delay= 532.17 ns | gain=9.9769e-13 | -120.01 dB
Path 223: delay= 735.66 ns | gain=3.2141e-13 | -124.93 dB
Path 224: delay= 651.61 ns | gain=2.3902e-12 | -116.22 dB
Path 225: delay= 699.14 ns | gain=4.7119e-10 | -93.27 dB
```

```
Path 226: delay= 575.76 ns | gain=1.3331e-11 | -108.75 dB
Path 227: delay= 690.17 ns | gain=4.5702e-13 | -123.40 dB
Path 228: delay= 636.05 ns | gain=3.5913e-12 | -114.45 dB
Path 229: delay= 578.32 ns | gain=2.1283e-12 | -116.72 dB
Path 230: delay= 592.21 ns | gain=8.5743e-12 | -110.67 dB
Path 231: delay= 662.40 ns | gain=1.7698e-12 | -117.52 dB
Path 232: delay= 582.56 ns | gain=5.6188e-12 | -112.50 dB
Path 233: delay= 724.14 ns | gain=3.0107e-10 | -95.21 dB
Path 234: delay= 818.39 ns | gain=7.0102e-14 | -131.54 dB
Path 235: delay= 558.76 ns | gain=1.1126e-11 | -109.54 dB
Path 236: delay= 604.82 ns | gain=4.5738e-12 | -113.40 dB
Path 237: delay= 571.00 ns | gain=1.3250e-11 | -108.78 dB
Path 238: delay= 586.90 ns | gain=1.2410e-11 | -109.06 dB
Path 239: delay= 737.34 ns | gain=3.2855e-13 | -124.83 dB
Path 240: delay= 577.46 ns | gain=7.4009e-12 | -111.31 dB
Path 241: delay= 750.99 ns | gain=1.3450e-13 | -128.71 dB
Path 242: delay= 621.99 ns | gain=5.2598e-12 | -112.79 dB
Path 243: delay= 633.85 ns | gain=2.2123e-12 | -116.55 dB
Path 244: delay= 801.35 ns | gain=1.0404e-13 | -129.83 dB
Path 245: delay= 730.37 ns | gain=3.6862e-13 | -124.33 dB
Path 246: delay= 624.59 ns | gain=3.9164e-12 | -114.07 dB
Path 247: delay= 557.64 ns | gain=1.2914e-11 | -108.89 dB
Path 248: delay= 756.73 ns | gain=2.3309e-13 | -126.32 dB
Path 249: delay= 760.02 ns | gain=2.1722e-13 | -126.63 dB
Path 250: delay= 566.35 ns | gain=1.4972e-11 | -108.25 dB
Path 251: delay= 634.67 ns | gain=3.0993e-12 | -115.09 dB
Path 252: delay= 570.62 ns | gain=3.8756e-12 | -114.12 dB
Path 253: delay= 573.96 ns | gain=5.2799e-12 | -112.77 dB
Path 254: delay= 680.25 ns | gain=2.9768e-13 | -125.26 dB
Path 255: delay= 557.09 ns | gain=8.6912e-12 | -110.61 dB
Path 256: delay= 565.70 ns | gain=2.5501e-12 | -115.93 dB
Path 257: delay= 554.51 ns | gain=1.1312e-11 | -109.46 dB
Path 258: delay= 733.71 ns | gain=3.6382e-13 | -124.39 dB
Path 259: delay= 594.06 ns | gain=7.9277e-12 | -111.01 dB
Path 260: delay= 678.80 ns | gain=7.6328e-13 | -121.17 dB
Path 261: delay= 611.16 ns | gain=1.8398e-12 | -117.35 dB
Path 262: delay= 582.42 ns | gain=9.7561e-12 | -110.11 dB
Path 263: delay= 575.26 ns | gain=5.1961e-12 | -112.84 dB
Path 264: delay= 708.80 ns | gain=5.4937e-13 | -122.60 dB
Path 265: delay= 539.85 ns | gain=1.6976e-12 | -117.70 dB
Path 266: delay= 531.46 ns | gain=7.9373e-13 | -121.00 dB
Path 267: delay= 612.00 ns | gain=5.8101e-12 | -112.36 dB
Path 268: delay= 569.81 ns | gain=1.5056e-11 | -108.22 dB
Path 269: delay= 553.05 ns | gain=9.5309e-12 | -110.21 dB
Path 270: delay= 532.39 ns | gain=1.0554e-12 | -119.77 dB
Path 271: delay= 788.63 ns | gain=1.1125e-13 | -129.54 dB
Path 272: delay= 546.01 ns | gain=6.5047e-12 | -111.87 dB
Path 273: delay= 684.18 ns | gain=1.0022e-12 | -119.99 dB
```

```
Path 274: delay= 617.24 ns | gain=1.7649e-12 | -117.53 dB
Path 275: delay= 592.70 ns | gain=1.0981e-11 | -109.59 dB
Path 276: delay= 542.07 ns | gain=1.6408e-12 | -117.85 dB
Path 277: delay= 715.29 ns | gain=3.5563e-10 | -94.49 dB
Path 278: delay= 570.29 ns | gain=1.4932e-11 | -108.26 dB
Path 279: delay= 553.53 ns | gain=1.0692e-11 | -109.71 dB
Path 280: delay= 541.44 ns | gain=4.0125e-12 | -113.97 dB
Path 281: delay= 586.10 ns | gain=1.0954e-11 | -109.60 dB
Path 282: delay= 844.92 ns | gain=3.9335e-14 | -134.05 dB
Path 283: delay= 615.04 ns | gain=4.5403e-12 | -113.43 dB
Path 284: delay= 584.40 ns | gain=3.6405e-12 | -114.39 dB
Path 285: delay= 581.79 ns | gain=1.1560e-11 | -109.37 dB
Path 286: delay= 811.10 ns | gain=7.8306e-14 | -131.06 dB
Path 287: delay= 580.71 ns | gain=1.1400e-11 | -109.43 dB
Path 288: delay= 595.96 ns | gain=5.9211e-12 | -112.28 dB
Path 289: delay= 908.24 ns | gain=1.4305e-10 | -98.45 dB
Path 290: delay= 785.78 ns | gain=1.2477e-13 | -129.04 dB
Path 291: delay= 654.77 ns | gain=6.3157e-10 | -92.00 dB
Path 292: delay= 748.31 ns | gain=1.6542e-13 | -127.81 dB
Path 293: delay= 567.21 ns | gain=1.0743e-11 | -109.69 dB
Path 294: delay= 671.55 ns | gain=3.9432e-13 | -124.04 dB
Path 295: delay= 591.85 ns | gain=1.0597e-12 | -119.75 dB
Path 296: delay= 584.12 ns | gain=6.2785e-12 | -112.02 dB
Path 297: delay= 537.62 ns | gain=9.5582e-13 | -120.20 dB
Path 298: delay= 597.07 ns | gain=8.5678e-12 | -110.67 dB
Path 299: delay= 704.71 ns | gain=1.8369e-13 | -127.36 dB
Path 300: delay= 588.63 ns | gain=9.9911e-12 | -110.00 dB
Path 301: delay= 541.18 ns | gain=4.0301e-12 | -113.95 dB
Path 302: delay= 757.78 ns | gain=1.6456e-13 | -127.84 dB
Path 303: delay= 555.89 ns | gain=6.1709e-12 | -112.10 dB
Path 304: delay= 545.25 ns | gain=2.2711e-12 | -116.44 dB
Path 305: delay= 568.03 ns | gain=5.0195e-12 | -112.99 dB
Path 306: delay= 701.89 ns | gain=5.1897e-13 | -122.85 dB
Path 307: delay= 809.02 ns | gain=6.5033e-14 | -131.87 dB
Path 308: delay= 678.34 ns | gain=6.4426e-13 | -121.91 dB
Path 309: delay= 541.73 ns | gain=4.1328e-12 | -113.84 dB
Path 310: delay= 633.96 ns | gain=6.1593e-10 | -92.10 dB
Path 311: delay= 613.84 ns | gain=1.6501e-12 | -117.82 dB
Path 312: delay= 576.56 ns | gain=5.3850e-12 | -112.69 dB
Path 313: delay= 625.54 ns | gain=4.7324e-12 | -113.25 dB
Path 314: delay= 723.72 ns | gain=1.7488e-13 | -127.57 dB
Path 315: delay= 958.41 ns | gain=1.5098e-11 | -108.21 dB
Path 316: delay= 602.14 ns | gain=6.6932e-12 | -111.74 dB
Path 317: delay= 635.80 ns | gain=1.4947e-12 | -118.25 dB
Path 318: delay= 656.06 ns | gain=6.2783e-10 | -92.02 dB
Path 319: delay= 573.78 ns | gain=2.2958e-12 | -116.39 dB
Path 320: delay= 539.77 ns | gain=3.7827e-12 | -114.22 dB
Path 321: delay= 564.01 ns | gain=5.1774e-10 | -92.86 dB
```

```
Path 322: delay= 933.39 ns | gain=3.0270e-09 | -85.19 dB
Path 323: delay= 634.85 ns | gain=2.7357e-12 | -115.63 dB
Path 324: delay= 563.90 ns | gain=1.0816e-11 | -109.66 dB
Path 325: delay= 553.60 ns | gain=5.7444e-13 | -122.41 dB
Path 326: delay= 577.46 ns | gain=1.3010e-11 | -108.86 dB
Path 327: delay= 756.22 ns | gain=1.1351e-13 | -129.45 dB
Path 328: delay= 610.12 ns | gain=5.6335e-12 | -112.49 dB
Path 329: delay= 589.84 ns | gain=6.3846e-13 | -121.95 dB
Path 330: delay= 648.10 ns | gain=1.9163e-12 | -117.18 dB
Path 331: delay= 620.36 ns | gain=4.0541e-12 | -113.92 dB
Path 332: delay= 533.75 ns | gain=4.2426e-13 | -123.72 dB
Path 333: delay= 536.62 ns | gain=1.9823e-12 | -117.03 dB
Path 334: delay= 782.38 ns | gain=2.0243e-10 | -96.94 dB
Path 335: delay= 558.31 ns | gain=8.6594e-12 | -110.63 dB
Path 336: delay= 572.22 ns | gain=1.1115e-11 | -109.54 dB
Path 337: delay= 550.92 ns | gain=7.8086e-12 | -111.07 dB
Path 338: delay= 705.98 ns | gain=1.7525e-13 | -127.56 dB
Path 339: delay= 582.40 ns | gain=7.6676e-12 | -111.15 dB
Path 340: delay= 721.60 ns | gain=4.5258e-13 | -123.44 dB
Path 341: delay= 548.60 ns | gain=6.4923e-12 | -111.88 dB
Path 342: delay= 578.00 ns | gain=1.3205e-11 | -108.79 dB
Path 343: delay= 571.45 ns | gain=1.4335e-11 | -108.44 dB
Path 344: delay= 577.84 ns | gain=1.9682e-13 | -127.06 dB
Path 345: delay= 603.83 ns | gain=5.7885e-12 | -112.37 dB
Path 346: delay= 738.67 ns | gain=2.6411e-13 | -125.78 dB
Path 347: delay= 556.02 ns | gain=5.6468e-13 | -122.48 dB
Path 348: delay= 579.02 ns | gain=1.3972e-11 | -108.55 dB
Path 349: delay= 593.42 ns | gain=6.2991e-12 | -112.01 dB
Path 350: delay= 599.18 ns | gain=7.2858e-12 | -111.38 dB
Path 351: delay= 830.53 ns | gain=4.1429e-14 | -133.83 dB
Path 352: delay= 590.23 ns | gain=1.5354e-12 | -118.14 dB
Path 353: delay= 630.88 ns | gain=4.0905e-12 | -113.88 dB
Path 354: delay= 604.77 ns | gain=8.0633e-12 | -110.93 dB
Path 355: delay= 869.55 ns | gain=3.8231e-14 | -134.18 dB
Path 356: delay= 599.87 ns | gain=3.0991e-13 | -125.09 dB
Path 357: delay= 553.27 ns | gain=6.2184e-12 | -112.06 dB
Path 358: delay= 898.19 ns | gain=2.2329e-14 | -136.51 dB
Path 359: delay= 784.15 ns | gain=1.4301e-13 | -128.45 dB
Path 360: delay= 531.05 ns | gain=8.4451e-13 | -120.73 dB
Path 361: delay= 598.70 ns | gain=7.0926e-12 | -111.49 dB
Path 362: delay= 570.72 ns | gain=2.1530e-12 | -116.67 dB
Path 363: delay= 582.72 ns | gain=2.0820e-12 | -116.82 dB
Path 364: delay= 684.65 ns | gain=4.8162e-13 | -123.17 dB
Path 365: delay= 580.58 ns | gain=9.4969e-12 | -110.22 dB
Path 366: delay= 557.92 ns | gain=1.2730e-11 | -108.95 dB
Path 367: delay= 548.86 ns | gain=6.6551e-12 | -111.77 dB
Path 368: delay= 550.66 ns | gain=9.3956e-12 | -110.27 dB
Path 369: delay= 614.02 ns | gain=6.4703e-12 | -111.89 dB
```

```
Path 370: delay= 644.37 ns | gain=2.8199e-12 | -115.50 dB
Path 371: delay= 663.77 ns | gain=1.6582e-12 | -117.80 dB
Path 372: delay= 648.51 ns | gain=2.4425e-12 | -116.12 dB
Path 373: delay= 546.19 ns | gain=4.7745e-12 | -113.21 dB
Path 374: delay= 588.84 ns | gain=1.1591e-11 | -109.36 dB
Path 375: delay= 545.26 ns | gain=6.8334e-12 | -111.65 dB
Path 376: delay= 581.12 ns | gain=1.1069e-11 | -109.56 dB
Path 377: delay= 707.82 ns | gain=5.2787e-13 | -122.77 dB
Path 378: delay= 563.18 ns | gain=1.0499e-11 | -109.79 dB
Path 379: delay= 582.70 ns | gain=1.3355e-11 | -108.74 dB
Path 380: delay= 617.98 ns | gain=3.4194e-12 | -114.66 dB
Path 381: delay= 603.48 ns | gain=8.5362e-12 | -110.69 dB
Path 382: delay= 578.18 ns | gain=3.8045e-12 | -114.20 dB
Path 383: delay= 576.77 ns | gain=1.4192e-11 | -108.48 dB
Path 384: delay= 535.80 ns | gain=4.1137e-13 | -123.86 dB
Path 385: delay=1251.61 ns | gain=7.6502e-16 | -151.16 dB
Path 386: delay= 664.76 ns | gain=2.0597e-13 | -126.86 dB
Path 387: delay= 567.23 ns | gain=3.4715e-10 | -94.59 dB
Path 388: delay= 618.53 ns | gain=8.1160e-13 | -120.91 dB
Path 389: delay= 717.31 ns | gain=2.8218e-10 | -95.49 dB
Path 390: delay= 538.58 ns | gain=1.9070e-12 | -117.20 dB
Path 391: delay= 607.51 ns | gain=1.1858e-12 | -119.26 dB
Path 392: delay= 718.01 ns | gain=3.9924e-10 | -93.99 dB
Path 393: delay= 585.70 ns | gain=1.1653e-11 | -109.34 dB
Path 394: delay= 602.30 ns | gain=3.9244e-12 | -114.06 dB
Path 395: delay= 586.93 ns | gain=9.1912e-12 | -110.37 dB
Path 396: delay= 604.64 ns | gain=5.1364e-12 | -112.89 dB
Path 397: delay= 690.59 ns | gain=6.6155e-13 | -121.79 dB
Path 398: delay= 650.16 ns | gain=1.7302e-12 | -117.62 dB
Path 399: delay= 541.90 ns | gain=4.9593e-12 | -113.05 dB
Path 400: delay= 596.08 ns | gain=8.1571e-13 | -120.88 dB
Path 401: delay= 827.21 ns | gain=6.7149e-14 | -131.73 dB
Path 402: delay= 576.43 ns | gain=1.1561e-11 | -109.37 dB
Path 403: delay= 659.83 ns | gain=1.7866e-12 | -117.48 dB
Path 404: delay= 552.59 ns | gain=5.7414e-13 | -122.41 dB
Path 405: delay= 562.02 ns | gain=1.4355e-11 | -108.43 dB
Path 406: delay= 557.99 ns | gain=1.8524e-12 | -117.32 dB
Path 407: delay= 594.48 ns | gain=1.0605e-11 | -109.75 dB
Path 408: delay= 559.90 ns | gain=2.0454e-12 | -116.89 dB
Path 409: delay= 597.98 ns | gain=9.0511e-12 | -110.43 dB
Path 410: delay= 608.22 ns | gain=5.2396e-12 | -112.81 dB
Path 411: delay= 580.97 ns | gain=3.9057e-13 | -124.08 dB
Path 412: delay= 679.31 ns | gain=4.0648e-13 | -123.91 dB
Path 413: delay= 812.14 ns | gain=8.1332e-14 | -130.90 dB
Path 414: delay= 534.71 ns | gain=5.5067e-13 | -122.59 dB
Path 415: delay= 533.09 ns | gain=8.0117e-13 | -120.96 dB
Path 416: delay= 650.06 ns | gain=6.5648e-13 | -121.83 dB
Path 417: delay= 709.90 ns | gain=4.9339e-13 | -123.07 dB
```

```
Path 418: delay= 604.56 ns | gain=3.6035e-12 | -114.43 dB
Path 419: delay= 573.73 ns | gain=9.3835e-12 | -110.28 dB
Path 420: delay= 539.86 ns | gain=3.5227e-12 | -114.53 dB
Path 421: delay= 708.54 ns | gain=6.1047e-13 | -122.14 dB
Path 422: delay= 919.69 ns | gain=3.6398e-09 | -84.39 dB
Path 423: delay= 572.18 ns | gain=7.3180e-12 | -111.36 dB
Path 424: delay= 597.99 ns | gain=3.7452e-12 | -114.27 dB
Path 425: delay= 683.37 ns | gain=1.0816e-12 | -119.66 dB
Path 426: delay= 585.40 ns | gain=1.2279e-11 | -109.11 dB
Path 427: delay= 629.78 ns | gain=4.1692e-12 | -113.80 dB
Path 428: delay= 570.14 ns | gain=1.5007e-11 | -108.24 dB
Path 429: delay= 749.01 ns | gain=9.3967e-14 | -130.27 dB
Path 430: delay= 712.58 ns | gain=4.8146e-13 | -123.17 dB
Path 431: delay= 642.62 ns | gain=8.3007e-13 | -120.81 dB
Path 432: delay= 590.55 ns | gain=2.3140e-13 | -126.36 dB
Path 433: delay= 607.64 ns | gain=6.6937e-12 | -111.74 dB
Path 434: delay= 563.65 ns | gain=5.8562e-12 | -112.32 dB
Path 435: delay= 558.13 ns | gain=1.2285e-11 | -109.11 dB
Path 436: delay= 796.63 ns | gain=9.5206e-14 | -130.21 dB
Path 437: delay= 651.55 ns | gain=6.0333e-10 | -92.19 dB
Path 438: delay= 552.20 ns | gain=6.5950e-12 | -111.81 dB
Path 439: delay= 858.77 ns | gain=4.2128e-14 | -133.75 dB
Path 440: delay= 532.59 ns | gain=4.9120e-13 | -123.09 dB
Path 441: delay= 609.99 ns | gain=6.9446e-12 | -111.58 dB
Path 442: delay= 898.45 ns | gain=2.3928e-14 | -136.21 dB
Path 443: delay= 604.19 ns | gain=6.7791e-12 | -111.69 dB
Path 444: delay= 664.63 ns | gain=1.7154e-12 | -117.66 dB
Path 445: delay= 558.04 ns | gain=1.8273e-12 | -117.38 dB
Path 446: delay= 531.61 ns | gain=3.5835e-13 | -124.46 dB
Path 447: delay= 586.86 ns | gain=1.0178e-11 | -109.92 dB
Path 448: delay= 674.29 ns | gain=4.2839e-13 | -123.68 dB
Path 449: delay= 575.27 ns | gain=8.9082e-12 | -110.50 dB
Path 450: delay= 528.68 ns | gain=5.0372e-13 | -122.98 dB
Path 451: delay= 572.79 ns | gain=8.2010e-12 | -110.86 dB
Path 452: delay= 893.54 ns | gain=1.8324e-14 | -137.37 dB
Path 453: delay= 660.85 ns | gain=4.6589e-13 | -123.32 dB
Path 454: delay= 549.88 ns | gain=3.3416e-12 | -114.76 dB
Path 455: delay= 710.27 ns | gain=1.8427e-13 | -127.35 dB
Path 456: delay= 714.68 ns | gain=3.5252e-10 | -94.53 dB
Path 457: delay= 714.30 ns | gain=1.4806e-13 | -128.30 dB
Path 458: delay= 558.28 ns | gain=9.3282e-12 | -110.30 dB
Path 459: delay= 630.07 ns | gain=3.8689e-12 | -114.12 dB
Path 460: delay= 613.97 ns | gain=3.4750e-12 | -114.59 dB
Path 461: delay= 961.89 ns | gain=1.4695e-11 | -108.33 dB
Path 462: delay= 633.15 ns | gain=3.0865e-12 | -115.11 dB
Path 463: delay= 542.79 ns | gain=4.9378e-12 | -113.06 dB
Path 464: delay= 697.44 ns | gain=2.5661e-13 | -125.91 dB
Path 465: delay= 680.73 ns | gain=1.0700e-12 | -119.71 dB
```

```
Path 466: delay= 557.49 ns | gain=1.0562e-12 | -119.76 dB
Path 467: delay= 862.54 ns | gain=3.5729e-14 | -134.47 dB
Path 468: delay= 539.20 ns | gain=3.1881e-13 | -124.96 dB
Path 469: delay= 558.52 ns | gain=9.4059e-12 | -110.27 dB
Path 470: delay= 552.33 ns | gain=1.0418e-11 | -109.82 dB
Path 471: delay= 582.38 ns | gain=3.8292e-12 | -114.17 dB
Path 472: delay= 651.51 ns | gain=4.0133e-13 | -123.96 dB
Path 473: delay= 553.35 ns | gain=5.3615e-12 | -112.71 dB
Path 474: delay= 563.44 ns | gain=1.4310e-11 | -108.44 dB
Path 475: delay= 539.70 ns | gain=1.6677e-12 | -117.78 dB
Path 476: delay= 735.03 ns | gain=2.9006e-13 | -125.38 dB
Path 477: delay= 607.31 ns | gain=1.2529e-12 | -119.02 dB
Path 478: delay= 685.88 ns | gain=7.9928e-13 | -120.97 dB
Path 479: delay= 604.91 ns | gain=8.1311e-12 | -110.90 dB
Path 480: delay= 835.24 ns | gain=4.3294e-14 | -133.64 dB
Path 481: delay= 624.64 ns | gain=8.8941e-13 | -120.51 dB
Path 482: delay= 556.44 ns | gain=1.1808e-11 | -109.28 dB
Path 483: delay= 540.09 ns | gain=1.3470e-12 | -118.71 dB
Path 484: delay= 629.74 ns | gain=1.2263e-12 | -119.11 dB
Path 485: delay= 727.67 ns | gain=1.7467e-13 | -127.58 dB
Path 486: delay= 527.17 ns | gain=4.3942e-13 | -123.57 dB
Path 487: delay= 827.39 ns | gain=4.9760e-14 | -133.03 dB
Path 488: delay= 567.37 ns | gain=8.6237e-12 | -110.64 dB
Path 489: delay= 626.61 ns | gain=2.9824e-12 | -115.25 dB
Path 490: delay= 644.45 ns | gain=1.8255e-12 | -117.39 dB
Path 491: delay= 653.59 ns | gain=1.0602e-12 | -119.75 dB
Path 492: delay= 592.64 ns | gain=1.6798e-12 | -117.75 dB
Path 493: delay= 643.00 ns | gain=1.2533e-12 | -119.02 dB
Path 494: delay= 660.62 ns | gain=5.9354e-10 | -92.27 dB
Path 495: delay= 592.07 ns | gain=1.1084e-11 | -109.55 dB
Path 496: delay= 576.10 ns | gain=1.2482e-11 | -109.04 dB
Path 497: delay= 551.40 ns | gain=6.1335e-12 | -112.12 dB
Path 498: delay= 564.64 ns | gain=1.2255e-11 | -109.12 dB
Path 499: delay= 758.37 ns | gain=2.2659e-13 | -126.45 dB
Path 500: delay= 585.37 ns | gain=1.2643e-11 | -108.98 dB
Path 501: delay= 577.70 ns | gain=8.0290e-12 | -110.95 dB
Path 502: delay= 671.08 ns | gain=9.7987e-13 | -120.09 dB
Path 503: delay= 798.84 ns | gain=9.8795e-14 | -130.05 dB
Path 504: delay= 659.43 ns | gain=1.3449e-12 | -118.71 dB
Path 505: delay= 658.93 ns | gain=1.8057e-12 | -117.43 dB
Path 506: delay= 620.24 ns | gain=5.4263e-12 | -112.65 dB
Path 507: delay= 568.13 ns | gain=1.1340e-11 | -109.45 dB
Path 508: delay= 565.02 ns | gain=1.4792e-11 | -108.30 dB
Path 509: delay= 587.82 ns | gain=1.1124e-11 | -109.54 dB
Path 510: delay= 551.08 ns | gain=6.4620e-13 | -121.90 dB
Path 511: delay= 611.07 ns | gain=3.1826e-12 | -114.97 dB
Path 512: delay= 825.64 ns | gain=4.0938e-14 | -133.88 dB
Path 513: delay= 567.26 ns | gain=3.3357e-10 | -94.77 dB
```

```
Path 514: delay= 564.88 ns | gain=1.3289e-11 | -108.76 dB
Path 515: delay= 734.43 ns | gain=1.9398e-13 | -127.12 dB
Path 516: delay= 600.65 ns | gain=7.2088e-12 | -111.42 dB
Path 517: delay= 603.42 ns | gain=1.9859e-12 | -117.02 dB
Path 518: delay= 613.13 ns | gain=6.4485e-12 | -111.91 dB
Path 519: delay= 647.63 ns | gain=1.9331e-12 | -117.14 dB
Path 520: delay= 545.99 ns | gain=5.6164e-12 | -112.51 dB
Path 521: delay= 548.69 ns | gain=3.2143e-12 | -114.93 dB
Path 522: delay= 600.86 ns | gain=1.1056e-12 | -119.56 dB
Path 523: delay= 647.72 ns | gain=9.5476e-13 | -120.20 dB
Path 524: delay= 545.76 ns | gain=2.8056e-12 | -115.52 dB
Path 525: delay= 545.52 ns | gain=6.9355e-12 | -111.59 dB
Path 526: delay= 580.44 ns | gain=1.0769e-11 | -109.68 dB
Path 527: delay= 562.03 ns | gain=1.4362e-11 | -108.43 dB
Path 528: delay= 542.24 ns | gain=5.0922e-12 | -112.93 dB
Path 529: delay= 612.19 ns | gain=1.7165e-13 | -127.65 dB
Path 530: delay= 679.76 ns | gain=4.6739e-13 | -123.30 dB
Path 531: delay= 619.14 ns | gain=5.6777e-12 | -112.46 dB
Path 532: delay= 640.38 ns | gain=6.5143e-13 | -121.86 dB
Path 533: delay= 629.29 ns | gain=3.0221e-13 | -125.20 dB
Path 534: delay= 778.78 ns | gain=1.0337e-13 | -129.86 dB
Path 535: delay= 657.92 ns | gain=1.9662e-12 | -117.06 dB
Path 536: delay= 571.64 ns | gain=1.4840e-11 | -108.29 dB
Path 537: delay= 691.73 ns | gain=8.8324e-13 | -120.54 dB
Path 538: delay= 538.07 ns | gain=2.7831e-12 | -115.55 dB
Path 539: delay= 988.47 ns | gain=1.1520e-11 | -109.39 dB
Path 540: delay= 652.09 ns | gain=5.9953e-13 | -122.22 dB
Path 541: delay= 733.65 ns | gain=1.9300e-13 | -127.14 dB
Path 542: delay= 566.11 ns | gain=1.4850e-11 | -108.28 dB
Path 543: delay= 547.23 ns | gain=1.6945e-12 | -117.71 dB
Path 544: delay= 666.66 ns | gain=1.2440e-12 | -119.05 dB
Path 545: delay= 555.74 ns | gain=4.5367e-12 | -113.43 dB
Path 546: delay= 542.05 ns | gain=4.7854e-12 | -113.20 dB
Path 547: delay= 573.93 ns | gain=8.4362e-12 | -110.74 dB
Path 548: delay= 675.80 ns | gain=7.9371e-13 | -121.00 dB
Path 549: delay= 559.97 ns | gain=1.1368e-11 | -109.44 dB
Path 550: delay= 629.28 ns | gain=2.2348e-12 | -116.51 dB
Path 551: delay= 731.30 ns | gain=3.6027e-13 | -124.43 dB
Path 552: delay= 776.66 ns | gain=1.3264e-13 | -128.77 dB
Path 553: delay= 608.47 ns | gain=7.5154e-12 | -111.24 dB
Path 554: delay= 591.25 ns | gain=7.0015e-12 | -111.55 dB
Path 555: delay= 556.98 ns | gain=9.0638e-12 | -110.43 dB
Path 556: delay= 634.98 ns | gain=3.6205e-12 | -114.41 dB
Path 557: delay= 557.94 ns | gain=1.0910e-11 | -109.62 dB
Path 558: delay= 658.65 ns | gain=1.0986e-12 | -119.59 dB
Path 559: delay= 688.43 ns | gain=4.3479e-13 | -123.62 dB
Path 560: delay= 585.63 ns | gain=1.2469e-11 | -109.04 dB
Path 561: delay= 603.15 ns | gain=6.7871e-12 | -111.68 dB
```

```
Path 562: delay= 584.54 ns | gain=2.7597e-12 | -115.59 dB
Path 563: delay= 545.16 ns | gain=2.3691e-12 | -116.25 dB
Path 564: delay=1016.53 ns | gain=5.9115e-15 | -142.28 dB
Path 565: delay= 554.44 ns | gain=3.1193e-12 | -115.06 dB
Path 566: delay= 657.36 ns | gain=1.8170e-12 | -117.41 dB
Path 567: delay= 643.01 ns | gain=6.2387e-13 | -122.05 dB
Path 568: delay= 600.56 ns | gain=2.5732e-13 | -125.90 dB
Path 569: delay= 582.71 ns | gain=3.7473e-12 | -114.26 dB
Path 570: delay= 664.30 ns | gain=5.6362e-10 | -92.49 dB
Path 571: delay= 605.73 ns | gain=3.5800e-12 | -114.46 dB
Path 572: delay= 594.56 ns | gain=9.2869e-13 | -120.32 dB
Path 573: delay= 686.35 ns | gain=5.5216e-13 | -122.58 dB
Path 574: delay= 578.69 ns | gain=8.0566e-12 | -110.94 dB
Path 575: delay= 933.27 ns | gain=1.5622e-14 | -138.06 dB
Path 576: delay= 555.82 ns | gain=8.9056e-12 | -110.50 dB
Path 577: delay= 797.66 ns | gain=5.9112e-14 | -132.28 dB
Path 578: delay= 593.62 ns | gain=1.0655e-11 | -109.72 dB
Path 579: delay= 570.07 ns | gain=1.1545e-11 | -109.38 dB
Path 580: delay= 547.43 ns | gain=7.7497e-12 | -111.11 dB
Path 581: delay= 654.73 ns | gain=2.2056e-12 | -116.56 dB
Path 582: delay= 706.57 ns | gain=6.4003e-13 | -121.94 dB
Path 583: delay= 573.71 ns | gain=1.1225e-11 | -109.50 dB
Path 584: delay= 543.23 ns | gain=4.5820e-12 | -113.39 dB
Path 585: delay= 575.58 ns | gain=3.9340e-12 | -114.05 dB
Path 586: delay= 590.96 ns | gain=1.0581e-12 | -119.75 dB
Path 587: delay= 561.09 ns | gain=5.0281e-13 | -122.99 dB
Path 588: delay= 554.50 ns | gain=2.4934e-12 | -116.03 dB
Path 589: delay= 579.59 ns | gain=3.1888e-12 | -114.96 dB
Path 590: delay= 613.62 ns | gain=4.8954e-12 | -113.10 dB
Path 591: delay= 880.75 ns | gain=3.1943e-14 | -134.96 dB
Path 592: delay= 649.27 ns | gain=1.6340e-12 | -117.87 dB
Path 593: delay= 571.21 ns | gain=9.1000e-13 | -120.41 dB
Path 594: delay= 600.92 ns | gain=5.5777e-12 | -112.54 dB
Path 595: delay= 550.75 ns | gain=8.0409e-12 | -110.95 dB
Path 596: delay= 622.70 ns | gain=3.5171e-12 | -114.54 dB
Path 597: delay= 644.73 ns | gain=1.6071e-12 | -117.94 dB
Path 598: delay= 576.79 ns | gain=8.1375e-12 | -110.90 dB
Path 599: delay= 850.37 ns | gain=4.9095e-14 | -133.09 dB
Path 600: delay= 533.77 ns | gain=8.4316e-13 | -120.74 dB
Path 601: delay= 803.57 ns | gain=7.8952e-14 | -131.03 dB
Path 602: delay= 598.64 ns | gain=1.4113e-12 | -118.50 dB
Path 603: delay= 663.50 ns | gain=1.7606e-12 | -117.54 dB
Path 604: delay= 558.55 ns | gain=6.4429e-12 | -111.91 dB
Path 605: delay= 610.61 ns | gain=5.7575e-12 | -112.40 dB
Path 606: delay= 598.80 ns | gain=9.3143e-12 | -110.31 dB
Path 607: delay= 644.30 ns | gain=9.9189e-13 | -120.04 dB
Path 608: delay= 631.50 ns | gain=6.0170e-10 | -92.21 dB
Path 609: delay= 576.02 ns | gain=6.0872e-12 | -112.16 dB
```

```
Path 610: delay= 660.19 ns | gain=1.1631e-12 | -119.34 dB
Path 611: delay= 716.95 ns | gain=3.8601e-10 | -94.13 dB
Path 612: delay= 551.48 ns | gain=1.6156e-12 | -117.92 dB
Path 613: delay= 561.29 ns | gain=1.1044e-11 | -109.57 dB
Path 614: delay= 554.27 ns | gain=4.3943e-12 | -113.57 dB
Path 615: delay= 937.74 ns | gain=2.8463e-09 | -85.46 dB
Path 616: delay= 546.54 ns | gain=2.8638e-13 | -125.43 dB
Path 617: delay= 583.35 ns | gain=1.0421e-11 | -109.82 dB
Path 618: delay= 579.98 ns | gain=1.3860e-11 | -108.58 dB
Path 619: delay= 604.72 ns | gain=7.3781e-12 | -111.32 dB
Path 620: delay= 670.60 ns | gain=3.5932e-13 | -124.45 dB
Path 621: delay= 658.37 ns | gain=5.1355e-13 | -122.89 dB
Path 622: delay= 654.48 ns | gain=2.1943e-12 | -116.59 dB
Path 623: delay= 581.48 ns | gain=9.9002e-12 | -110.04 dB
Path 624: delay= 746.88 ns | gain=2.7097e-13 | -125.67 dB
Path 625: delay= 596.47 ns | gain=4.3298e-12 | -113.64 dB
Path 626: delay= 552.08 ns | gain=2.9082e-12 | -115.36 dB
Path 627: delay= 751.60 ns | gain=1.5502e-13 | -128.10 dB
Path 628: delay= 572.71 ns | gain=4.9420e-12 | -113.06 dB
Path 629: delay= 606.38 ns | gain=7.9452e-12 | -111.00 dB
Path 630: delay= 600.10 ns | gain=5.3399e-12 | -112.72 dB
Path 631: delay= 796.43 ns | gain=9.3187e-14 | -130.31 dB
Path 632: delay= 588.06 ns | gain=9.1138e-12 | -110.40 dB
Path 633: delay= 563.36 ns | gain=7.9777e-12 | -110.98 dB
Path 634: delay= 553.53 ns | gain=1.0400e-11 | -109.83 dB
Path 635: delay= 736.97 ns | gain=2.3672e-09 | -86.26 dB
Path 636: delay= 631.89 ns | gain=3.4054e-12 | -114.68 dB
Path 637: delay= 581.64 ns | gain=9.2043e-12 | -110.36 dB
Path 638: delay= 639.81 ns | gain=2.6638e-12 | -115.74 dB
Path 639: delay= 554.44 ns | gain=1.1831e-11 | -109.27 dB
Path 640: delay= 576.50 ns | gain=1.6097e-12 | -117.93 dB
Path 641: delay= 538.59 ns | gain=3.2618e-13 | -124.87 dB
Path 642: delay= 591.69 ns | gain=8.5596e-12 | -110.68 dB
Path 643: delay= 559.08 ns | gain=3.5231e-13 | -124.53 dB
Path 644: delay= 633.00 ns | gain=3.3388e-13 | -124.76 dB
Path 645: delay= 771.46 ns | gain=1.7547e-13 | -127.56 dB
Path 646: delay= 697.15 ns | gain=5.4410e-13 | -122.64 dB
Path 647: delay= 591.79 ns | gain=7.0744e-12 | -111.50 dB
Path 648: delay= 559.38 ns | gain=3.7923e-12 | -114.21 dB
Path 649: delay= 568.03 ns | gain=2.6461e-13 | -125.77 dB
Path 650: delay= 562.89 ns | gain=8.5939e-12 | -110.66 dB
Path 651: delay= 540.37 ns | gain=2.2575e-12 | -116.46 dB
Path 652: delay= 529.75 ns | gain=6.3125e-13 | -122.00 dB
Path 653: delay= 539.68 ns | gain=2.9351e-12 | -115.32 dB
Path 654: delay= 684.56 ns | gain=3.7745e-13 | -124.23 dB
Path 655: delay= 659.17 ns | gain=1.6157e-12 | -117.92 dB
Path 656: delay= 556.26 ns | gain=1.0572e-11 | -109.76 dB
Path 657: delay= 567.72 ns | gain=4.0041e-12 | -113.97 dB
```

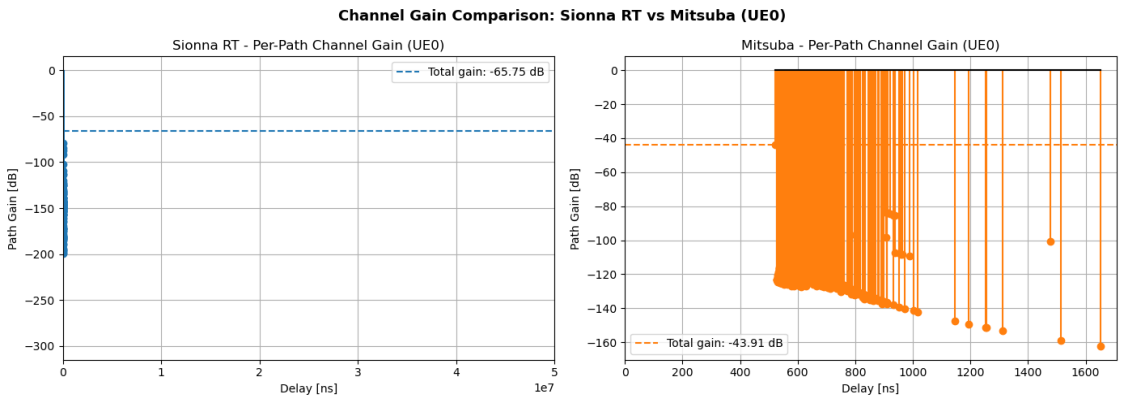
```
Path 658: delay= 846.60 ns | gain=4.0749e-14 | -133.90 dB
Path 659: delay= 893.07 ns | gain=2.7730e-14 | -135.57 dB
Path 660: delay= 546.75 ns | gain=6.3312e-12 | -111.99 dB
Path 661: delay= 558.51 ns | gain=9.5105e-12 | -110.22 dB
Path 662: delay=1003.69 ns | gain=7.1613e-15 | -141.45 dB
Path 663: delay= 623.40 ns | gain=4.4620e-12 | -113.50 dB
Path 664: delay= 557.95 ns | gain=1.3216e-11 | -108.79 dB
Path 665: delay= 621.28 ns | gain=3.7306e-13 | -124.28 dB
Path 666: delay= 590.55 ns | gain=1.0676e-11 | -109.72 dB
Path 667: delay= 548.00 ns | gain=7.1191e-12 | -111.48 dB
Path 668: delay= 595.98 ns | gain=8.4102e-12 | -110.75 dB
Path 669: delay= 760.60 ns | gain=2.1709e-13 | -126.63 dB
Path 670: delay= 557.56 ns | gain=1.1942e-11 | -109.23 dB
Path 671: delay= 615.11 ns | gain=6.3205e-12 | -111.99 dB
Path 672: delay= 632.85 ns | gain=2.9068e-12 | -115.37 dB
Path 673: delay= 595.19 ns | gain=2.3535e-12 | -116.28 dB
Path 674: delay= 597.47 ns | gain=9.8154e-12 | -110.08 dB
Path 675: delay= 559.68 ns | gain=1.1225e-11 | -109.50 dB
Path 676: delay= 578.93 ns | gain=1.2418e-11 | -109.06 dB
Path 677: delay= 572.91 ns | gain=6.8663e-12 | -111.63 dB
Path 678: delay= 551.18 ns | gain=6.5302e-12 | -111.85 dB
Path 679: delay= 564.83 ns | gain=1.4821e-11 | -108.29 dB
Path 680: delay= 645.58 ns | gain=2.1434e-12 | -116.69 dB
Path 681: delay= 591.85 ns | gain=8.2536e-12 | -110.83 dB
Path 682: delay= 797.89 ns | gain=8.4814e-14 | -130.72 dB
Path 683: delay= 637.07 ns | gain=2.7332e-12 | -115.63 dB
Path 684: delay= 908.72 ns | gain=1.7159e-14 | -137.66 dB
Path 685: delay= 614.21 ns | gain=7.8981e-13 | -121.02 dB
Path 686: delay= 557.38 ns | gain=3.9544e-12 | -114.03 dB
Path 687: delay= 634.43 ns | gain=3.1353e-12 | -115.04 dB
Path 688: delay= 738.83 ns | gain=1.6046e-13 | -127.95 dB
Path 689: delay= 826.97 ns | gain=6.4263e-14 | -131.92 dB
Path 690: delay= 539.52 ns | gain=6.9070e-13 | -121.61 dB
Path 691: delay= 614.91 ns | gain=4.1722e-13 | -123.80 dB
Path 692: delay= 586.78 ns | gain=1.2354e-11 | -109.08 dB
Path 693: delay= 824.96 ns | gain=5.9628e-14 | -132.25 dB
Path 694: delay= 558.56 ns | gain=9.5558e-12 | -110.20 dB
Path 695: delay= 536.52 ns | gain=2.4375e-12 | -116.13 dB
Path 696: delay= 590.53 ns | gain=6.5262e-12 | -111.85 dB
Path 697: delay= 612.80 ns | gain=5.4659e-12 | -112.62 dB
Path 698: delay= 598.52 ns | gain=9.5149e-13 | -120.22 dB
Path 699: delay= 571.61 ns | gain=6.7099e-12 | -111.73 dB
Path 700: delay= 634.05 ns | gain=2.8064e-12 | -115.52 dB
Path 701: delay= 563.73 ns | gain=1.2108e-11 | -109.17 dB
Path 702: delay= 568.51 ns | gain=8.5298e-12 | -110.69 dB
Path 703: delay= 564.89 ns | gain=1.4803e-11 | -108.30 dB
Path 704: delay= 593.72 ns | gain=7.0135e-12 | -111.54 dB
Path 705: delay= 608.85 ns | gain=4.4097e-12 | -113.56 dB
```

```
Path 706: delay= 567.67 ns | gain=1.4445e-11 | -108.40 dB
Path 707: delay= 560.68 ns | gain=8.6775e-12 | -110.62 dB
Path 708: delay= 705.82 ns | gain=1.5715e-13 | -128.04 dB
Path 709: delay= 567.86 ns | gain=1.3788e-11 | -108.61 dB
Path 710: delay= 630.86 ns | gain=5.5867e-13 | -122.53 dB
Path 711: delay= 854.68 ns | gain=4.7156e-14 | -133.26 dB
Path 712: delay= 538.23 ns | gain=8.3619e-13 | -120.78 dB
Path 713: delay= 565.15 ns | gain=2.5379e-13 | -125.96 dB
Path 714: delay= 641.17 ns | gain=1.9349e-12 | -117.13 dB
Path 715: delay= 528.17 ns | gain=4.5877e-13 | -123.38 dB
Path 716: delay= 568.58 ns | gain=7.5006e-12 | -111.25 dB
Path 717: delay= 601.49 ns | gain=3.6538e-12 | -114.37 dB
Path 718: delay= 603.39 ns | gain=7.8791e-12 | -111.04 dB
Path 719: delay= 555.84 ns | gain=7.6425e-12 | -111.17 dB
Path 720: delay= 557.54 ns | gain=7.7056e-12 | -111.13 dB
Path 721: delay= 619.87 ns | gain=1.7055e-12 | -117.68 dB
Path 722: delay= 682.91 ns | gain=8.4194e-13 | -120.75 dB
Path 723: delay= 553.88 ns | gain=7.9662e-13 | -120.99 dB
Path 724: delay= 652.09 ns | gain=2.2289e-12 | -116.52 dB
Path 725: delay= 648.23 ns | gain=1.9361e-12 | -117.13 dB
Path 726: delay= 695.31 ns | gain=8.2175e-13 | -120.85 dB
Path 727: delay= 573.79 ns | gain=2.1514e-12 | -116.67 dB
Path 728: delay= 567.38 ns | gain=8.2452e-13 | -120.84 dB
Path 729: delay= 572.73 ns | gain=9.1991e-12 | -110.36 dB
Path 730: delay= 751.98 ns | gain=1.5883e-13 | -127.99 dB
Path 731: delay= 571.18 ns | gain=1.1940e-11 | -109.23 dB
Path 732: delay= 659.07 ns | gain=8.5983e-13 | -120.66 dB
Path 733: delay= 676.09 ns | gain=9.1429e-13 | -120.39 dB
Path 734: delay= 719.90 ns | gain=4.7893e-13 | -123.20 dB
Path 735: delay= 571.83 ns | gain=1.3281e-11 | -108.77 dB
Path 736: delay= 658.26 ns | gain=1.9792e-12 | -117.04 dB
Path 737: delay= 621.84 ns | gain=5.2748e-12 | -112.78 dB
Path 738: delay= 567.16 ns | gain=1.2346e-11 | -109.08 dB
Path 739: delay= 551.67 ns | gain=3.4299e-12 | -114.65 dB
Path 740: delay= 713.91 ns | gain=2.5173e-13 | -125.99 dB
Path 741: delay= 856.29 ns | gain=4.3867e-14 | -133.58 dB
Path 742: delay= 547.73 ns | gain=8.1977e-12 | -110.86 dB
Path 743: delay= 537.16 ns | gain=2.2333e-12 | -116.51 dB
Path 744: delay= 590.70 ns | gain=1.1342e-11 | -109.45 dB
Path 745: delay= 552.27 ns | gain=1.0486e-11 | -109.79 dB
Path 746: delay= 608.25 ns | gain=7.5455e-12 | -111.22 dB
Path 747: delay= 540.09 ns | gain=3.4901e-12 | -114.57 dB
Path 748: delay= 735.29 ns | gain=1.3824e-13 | -128.59 dB
Path 749: delay= 541.75 ns | gain=4.0754e-13 | -123.90 dB
Path 750: delay= 759.47 ns | gain=2.1855e-13 | -126.60 dB
Path 751: delay= 553.50 ns | gain=2.4280e-12 | -116.15 dB
Path 752: delay= 594.88 ns | gain=8.8944e-12 | -110.51 dB
Path 753: delay= 550.21 ns | gain=9.4222e-12 | -110.26 dB
```

```
Path 754: delay= 589.81 ns | gain=2.5460e-13 | -125.94 dB
Path 755: delay= 649.43 ns | gain=2.2968e-12 | -116.39 dB
Path 756: delay= 785.44 ns | gain=7.7730e-14 | -131.09 dB
Path 757: delay= 669.95 ns | gain=1.4390e-12 | -118.42 dB
Path 758: delay= 772.21 ns | gain=9.9038e-14 | -130.04 dB
Path 759: delay= 588.36 ns | gain=2.0102e-12 | -116.97 dB
Path 760: delay= 617.53 ns | gain=8.5631e-13 | -120.67 dB
Path 761: delay= 566.24 ns | gain=1.4220e-11 | -108.47 dB
Path 762: delay= 535.54 ns | gain=1.9505e-12 | -117.10 dB
Path 763: delay= 600.05 ns | gain=9.2163e-12 | -110.35 dB
Path 764: delay= 576.88 ns | gain=8.1023e-12 | -110.91 dB
Path 765: delay= 601.22 ns | gain=8.4834e-12 | -110.71 dB
Path 766: delay= 601.31 ns | gain=6.0157e-12 | -112.21 dB
Path 767: delay= 531.27 ns | gain=3.7218e-13 | -124.29 dB
Path 768: delay= 867.69 ns | gain=3.5369e-14 | -134.51 dB
Path 769: delay= 551.66 ns | gain=1.9940e-12 | -117.00 dB
Path 770: delay= 572.68 ns | gain=2.5580e-12 | -115.92 dB
Path 771: delay= 865.54 ns | gain=3.0564e-14 | -135.15 dB
Path 772: delay= 707.48 ns | gain=3.6344e-13 | -124.40 dB
Path 773: delay= 613.48 ns | gain=6.4160e-12 | -111.93 dB
Path 774: delay= 572.17 ns | gain=5.2478e-13 | -122.80 dB
Path 775: delay=1311.18 ns | gain=4.8297e-16 | -153.16 dB
Path 776: delay= 627.99 ns | gain=3.0021e-12 | -115.23 dB
Path 777: delay= 628.52 ns | gain=6.4166e-10 | -91.93 dB
Path 778: delay= 574.08 ns | gain=1.8972e-12 | -117.22 dB
Path 779: delay= 557.12 ns | gain=6.0914e-12 | -112.15 dB
Path 780: delay= 642.13 ns | gain=1.0101e-12 | -119.96 dB
Path 781: delay= 563.99 ns | gain=3.9072e-10 | -94.08 dB
Path 782: delay= 562.36 ns | gain=8.8139e-12 | -110.55 dB
Path 783: delay= 620.63 ns | gain=2.0312e-12 | -116.92 dB
Path 784: delay= 558.63 ns | gain=3.0374e-12 | -115.17 dB
Path 785: delay= 641.64 ns | gain=1.2494e-12 | -119.03 dB
Path 786: delay= 662.48 ns | gain=5.9314e-13 | -122.27 dB
Path 787: delay= 547.02 ns | gain=1.8162e-12 | -117.41 dB
Path 788: delay= 636.86 ns | gain=1.6791e-12 | -117.75 dB
Path 789: delay= 617.18 ns | gain=5.9135e-12 | -112.28 dB
Path 790: delay= 633.74 ns | gain=1.5550e-12 | -118.08 dB
Path 791: delay= 541.32 ns | gain=4.3201e-12 | -113.65 dB
Path 792: delay= 533.08 ns | gain=1.3060e-12 | -118.84 dB
Path 793: delay= 592.53 ns | gain=1.8467e-12 | -117.34 dB
Path 794: delay= 573.82 ns | gain=1.4391e-11 | -108.42 dB
Path 795: delay= 621.08 ns | gain=5.0802e-12 | -112.94 dB
Path 796: delay= 576.67 ns | gain=8.2094e-12 | -110.86 dB
Path 797: delay= 539.47 ns | gain=3.6900e-12 | -114.33 dB
Path 798: delay= 663.58 ns | gain=1.7616e-12 | -117.54 dB
Path 799: delay= 776.54 ns | gain=1.4342e-13 | -128.43 dB
Path 800: delay= 850.32 ns | gain=3.0717e-14 | -135.13 dB
Path 801: delay= 584.32 ns | gain=1.2912e-11 | -108.89 dB
```

```
Path 802: delay= 564.61 ns | gain=8.2390e-12 | -110.84 dB
Path 803: delay= 629.06 ns | gain=3.5270e-12 | -114.53 dB
Path 804: delay= 666.97 ns | gain=8.7280e-13 | -120.59 dB
Path 805: delay= 565.25 ns | gain=1.4117e-11 | -108.50 dB
Path 806: delay= 582.60 ns | gain=1.5948e-12 | -117.97 dB
Path 807: delay= 549.72 ns | gain=4.6969e-12 | -113.28 dB
Path 808: delay= 586.18 ns | gain=1.2512e-11 | -109.03 dB
Path 809: delay= 589.66 ns | gain=1.1757e-11 | -109.30 dB
Path 810: delay= 547.85 ns | gain=7.0571e-12 | -111.51 dB
Path 811: delay= 618.43 ns | gain=4.2298e-12 | -113.74 dB
Path 812: delay= 543.73 ns | gain=5.7308e-12 | -112.42 dB
Path 813: delay= 576.17 ns | gain=1.2132e-11 | -109.16 dB
Path 814: delay= 580.83 ns | gain=1.3588e-11 | -108.67 dB
Path 815: delay= 561.03 ns | gain=1.3211e-11 | -108.79 dB
Path 816: delay= 597.37 ns | gain=8.4087e-13 | -120.75 dB
Path 817: delay= 858.30 ns | gain=2.9734e-14 | -135.27 dB
Path 818: delay= 663.00 ns | gain=1.7588e-12 | -117.55 dB
Path 819: delay= 691.93 ns | gain=5.8267e-13 | -122.35 dB
Path 820: delay= 561.63 ns | gain=7.1714e-12 | -111.44 dB
Path 821: delay= 559.54 ns | gain=1.0599e-11 | -109.75 dB
Path 822: delay= 577.61 ns | gain=1.4295e-11 | -108.45 dB
Path 823: delay= 537.75 ns | gain=2.1827e-12 | -116.61 dB
Path 824: delay= 629.99 ns | gain=2.0327e-13 | -126.92 dB
Path 825: delay= 593.99 ns | gain=9.4021e-12 | -110.27 dB
Path 826: delay= 622.99 ns | gain=2.2507e-13 | -126.48 dB
Path 827: delay= 580.01 ns | gain=1.1992e-11 | -109.21 dB
Path 828: delay= 698.41 ns | gain=4.6383e-10 | -93.34 dB
Path 829: delay= 569.73 ns | gain=1.3681e-11 | -108.64 dB
Path 830: delay= 587.81 ns | gain=1.0916e-12 | -119.62 dB
Path 831: delay= 682.66 ns | gain=1.9134e-13 | -127.18 dB
Path 832: delay= 581.71 ns | gain=1.2870e-11 | -108.90 dB
Path 833: delay= 825.05 ns | gain=7.1755e-14 | -131.44 dB
Path 834: delay= 606.16 ns | gain=6.5044e-12 | -111.87 dB
Path 835: delay= 604.16 ns | gain=8.4066e-12 | -110.75 dB
Path 836: delay= 558.89 ns | gain=5.1427e-13 | -122.89 dB
Path 837: delay= 552.42 ns | gain=9.8443e-12 | -110.07 dB
Path 838: delay= 643.13 ns | gain=2.9975e-13 | -125.23 dB
Path 839: delay= 549.01 ns | gain=8.7733e-12 | -110.57 dB
Path 840: delay= 574.07 ns | gain=1.3945e-12 | -118.56 dB
Path 841: delay= 603.07 ns | gain=8.0546e-13 | -120.94 dB
Path 842: delay= 650.35 ns | gain=1.3371e-12 | -118.74 dB
Path 843: delay= 639.54 ns | gain=2.6754e-12 | -115.73 dB
Path 844: delay= 552.35 ns | gain=1.3097e-12 | -118.83 dB
Path 845: delay= 559.55 ns | gain=6.2752e-12 | -112.02 dB
Path 846: delay= 534.27 ns | gain=6.4195e-13 | -121.92 dB
Path 847: delay= 669.34 ns | gain=7.4814e-13 | -121.26 dB
Path 848: delay= 607.30 ns | gain=2.6080e-12 | -115.84 dB
Path 849: delay= 668.76 ns | gain=5.3987e-10 | -92.68 dB
```

```
Path 850: delay= 613.97 ns | gain=5.0501e-12 | -112.97 dB
Path 851: delay= 625.21 ns | gain=4.0696e-12 | -113.90 dB
```




```
=== Summary ===
Metric                                          Sionna         Mitsuba
-------------------------------------------------------------------
Number of paths                                    139             852
Total channel gain [linear]                 2.6634e-07      4.0675e-05
Total channel gain [dB]                        -65.75         -43.91
Strongest path gain [dB]                       -79.70         -43.91
Delay of strongest path [ns]                     0.00         523.28
```

[19]:
```python
import pandas as pd

#   BS position: derived from bs_pos_np set in the Mitsuba CIR cell
_bp  = globals().get("bs_pos_np", np.array([100.0, 120.0, 16.0]))
BS_X = float(_bp[0])
BS_Y = float(_bp[1])

#   Sionna: extract AoD (phi_t / theta_t)
# paths.phi_t / theta_t are drjit TensorXf in Sionna 1.2 - use np.array() to␣
↪convert.
phi_t = np.array(paths.phi_t)      # azimuth AoD
theta_t = np.array(paths.theta_t)  # elevation AoD

# Shape: [rx, rx_ant, tx, tx_ant, paths] (5D) or [rx, tx, paths] (3D)
if phi_t.ndim == 5:
  # Current shape: [rx, rx_ant, tx, tx_ant, paths]
  phi_t_link = phi_t[0, 0, 0, 0, :]
  theta_t_link = theta_t[0, 0, 0, 0, :]
elif phi_t.ndim == 3:
  # Compact shape: [rx, tx, paths]
```

```
  phi_t_link = phi_t[0, 0, :]
  theta_t_link = theta_t[0, 0, :]
else:
  phi_t_link = np.asarray(phi_t).reshape(-1)
  theta_t_link = np.asarray(theta_t).reshape(-1)

sionna_az_aod = np.degrees(np.asarray(phi_t_link).reshape(-1))
sionna_el_aod = np.degrees(np.asarray(theta_t_link).reshape(-1))

#   Sionna: OFDM frequency-domain channel (H)
# fft_size and subcarrier_spacing are defined in the constants cell (cell 9).
rg = ResourceGrid(num_ofdm_symbols=14,
           fft_size=fft_size,
           subcarrier_spacing=subcarrier_spacing)
frequencies = subcarrier_frequencies(fft_size, subcarrier_spacing)

# paths.cir() in Sionna 1.2 returns [rx, rx_ant, tx, tx_ant, paths, time] (no␣
 ↪batch dim).
# cir_to_ofdm_channel expects a batch axis → add one with [np.newaxis].
a_cir, tau_cir = paths.cir(out_type='numpy')
H = cir_to_ofdm_channel(frequencies, a_cir[np.newaxis], tau_cir[np.newaxis],␣
 ↪normalize=True)
H = np.asarray(H)
# H: [batch=1, num_rx, num_rx_ant, num_tx, num_tx_ant, num_time_steps, fft_size]
h_link = H[0, 0, 0, 0, 0]   # -> [num_time_steps, fft_size]
H_ue0 = h_link.reshape(-1)   # flatten to (num_subcarriers,)

# Align all Sionna vectors to the same path count
n_paths = min(len(sionna_tau_ns), len(sionna_az_aoa),
        len(sionna_az_aod), len(sionna_amp))

sionna_tau_ns_exp = sionna_tau_ns[:n_paths]
sionna_amp_exp = sionna_amp[:n_paths]
sionna_az_aoa_exp = sionna_az_aoa[:n_paths]
sionna_el_aoa_exp = sionna_el_aoa[:n_paths]
sionna_az_aod_exp = sionna_az_aod[:n_paths]
sionna_el_aod_exp = sionna_el_aod[:n_paths]
sionna_gain_path = np.abs(sionna_amp_exp)**2
sionna_gain_dB_exp = 10 * np.log10(sionna_gain_path + 1e-30)

#   Build Sionna path-level DataFrame
df_sionna_paths = pd.DataFrame({
  "source"            : "Sionna",
  "bs_x"              : BS_X,
  "bs_y"              : BS_Y,
  "bs_z"              : BS_H,
  "ue_x"              : ue_positions[0][0],
```

```
  "ue_y"              : ue_positions[0][1],
  "ue_z"              : ue_positions[0][2],
  "carrier_freq_Hz"   : fc,
  "path_index"        : np.arange(n_paths),
  "delay_ns"          : sionna_tau_ns_exp,
  "amplitude_abs"     : sionna_amp_exp,
  "channel_gain_linear": sionna_gain_path,
  "channel_gain_dB"   : sionna_gain_dB_exp,
  "az_aoa_deg"        : sionna_az_aoa_exp,
  "el_aoa_deg"        : sionna_el_aoa_exp,
  "az_aod_deg"        : sionna_az_aod_exp,
  "el_aod_deg"        : sionna_el_aod_exp,
})

sionna_paths_csv = os.path.join(scene_dir, "sionna_paths.csv")
df_sionna_paths.to_csv(sionna_paths_csv, index=False)
print(f"Saved Sionna path dataset  -> {sionna_paths_csv}")

#   Build Sionna frequency-domain DataFrame
frequencies_np = np.asarray(frequencies)
n_sc = min(len(H_ue0), len(frequencies_np))
H_ue0 = H_ue0[:n_sc]
freq_sel = frequencies_np[:n_sc]

df_sionna_freq = pd.DataFrame({
  "source"          : "Sionna",
  "bs_x"            : BS_X,
  "bs_y"            : BS_Y,
  "bs_z"            : BS_H,
  "ue_x"            : ue_positions[0][0],
  "ue_y"            : ue_positions[0][1],
  "ue_z"            : ue_positions[0][2],
  "carrier_freq_Hz" : fc,
  "subcarrier_index": np.arange(n_sc),
  "subcarrier_freq_Hz": fc + freq_sel,
  "H_real"          : H_ue0.real,
  "H_imag"          : H_ue0.imag,
  "H_magnitude"     : np.abs(H_ue0),
  "H_phase_deg"     : np.degrees(np.angle(H_ue0)),
  "H_gain_dB"       : 20 * np.log10(np.abs(H_ue0) + 1e-30),
})

sionna_freq_csv = os.path.join(scene_dir, "sionna_frequency_domain.csv")
df_sionna_freq.to_csv(sionna_freq_csv, index=False)
print(f"Saved Sionna frequency-domain dataset -> {sionna_freq_csv}")

#   Preview
```

```python
print("\n--- Sionna paths (first 5 rows) ---")
print(df_sionna_paths.head())
print("\n--- Sionna frequency domain (first 5 rows) ---")
print(df_sionna_freq.head())
```

```
Saved Sionna path dataset  ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/sionna_paths.csv
Saved Sionna frequency-domain dataset ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/sionna_frequency_domain.csv

--- Sionna paths (first 5 rows) ---
   source   bs_x   bs_y  bs_z  ue_x  ue_y  ue_z  carrier_freq_Hz  path_index  \
0  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           0
1  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           1
2  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           2
3  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           3
4  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           4

     delay_ns  amplitude_abs  channel_gain_linear  channel_gain_dB  \
0    0.000000   1.035361e-04         1.071972e-08       -79.698174
1   39.093891   5.869537e-05         3.445147e-09       -84.627930
2   40.042721   2.541263e-05         6.458016e-10       -91.899017
3  425.444244   1.955347e-09         3.823381e-18      -174.175537
4    1.019600   4.173970e-05         1.742203e-09       -87.589020

   az_aoa_deg  el_aoa_deg  az_aod_deg  el_aod_deg
0   50.194431   84.696617 -129.805573   95.303383
1   81.163445   85.066216 -144.734680   94.933754
2   81.163765   95.947517 -144.734650   95.947845
3  -74.728996   87.077713 -151.012238   92.922241
4   50.194435   96.392059 -129.805557   96.392342

--- Sionna frequency domain (first 5 rows) ---
   source   bs_x   bs_y  bs_z  ue_x  ue_y  ue_z  carrier_freq_Hz  \
0  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09
1  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09
2  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09
3  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09
4  Sionna  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09

   subcarrier_index  subcarrier_freq_Hz    H_real    H_imag  H_magnitude  \
0                 0        3.452480e+09  1.227923  0.886974     1.514767
1                 1        3.452510e+09  1.236310  0.884327     1.520031
2                 2        3.452540e+09  1.244902  0.879637     1.524317
3                 3        3.452570e+09  1.253261  0.872890     1.527286
4                 4        3.452600e+09  1.260954  0.864161     1.528653
```

```
   H_phase_deg  H_gain_dB
0    35.841965   3.606917
1    35.575974   3.637052
2    35.244694   3.661506
3    34.857033   3.678405
4    34.423725   3.686178
```

[20]:
```python
print('H shape:', np.shape(H))
print('frequencies shape:', np.shape(frequencies))
print('H_ue0 shape:', np.shape(H_ue0))
```

```
H shape: (1, 2, 1, 8, 32, 1, 3168)
frequencies shape: (3168,)
H_ue0 shape: (3168,)
```

[21]:
```python
import pandas as pd

#   Mitsuba: AoD proxy (direction FROM BS TO first bounce / target)
# For Mitsuba, AoD is the direction a ray leaves the TX toward the reflection
# point (or directly toward RX for LOS).
def compute_aod_mitsuba(origin_np, target_np, paths_aoa_list):
  """Compute AoD at TX for each path in mi_paths_aoa."""
  az_aod_list, el_aod_list = [], []
  # LOS: direction from TX -> RX
  d = target_np - origin_np
  d /= np.linalg.norm(d)
  az_aod_list.append(np.degrees(np.arctan2(d[1], d[0])))
  el_aod_list.append(np.degrees(np.arcsin(np.clip(d[2], -1, 1))))

  # For reflected paths we do not store the bounce point in mi_paths_aoa,
  # so we approximate AoD as opposite of AoA direction for each bounce path.
  for path in paths_aoa_list[1:]:
    az_aoa_r, el_aoa_r = path[2], path[3]
    # incoming ray direction (unit vector from scatter to RX)
    inc = np.array([
      np.cos(np.radians(el_aoa_r)) * np.cos(np.radians(az_aoa_r)),
      np.cos(np.radians(el_aoa_r)) * np.sin(np.radians(az_aoa_r)),
      np.sin(np.radians(el_aoa_r)),
    ])
    # crude AoD approximation: reverse incoming at scatterer
    out = -inc
    az_aod_list.append(np.degrees(np.arctan2(out[1], out[0])))
    el_aod_list.append(np.degrees(np.arcsin(np.clip(out[2], -1, 1))))
  return np.array(az_aod_list), np.array(el_aod_list)

tx_pos_np = globals().get("bs_pos_np", np.array(tx_positions[0], dtype=np.
 ↪float64))
```

```
mi_az_aod, mi_el_aod = compute_aod_mitsuba(tx_pos_np, ue_pos_np_list[0],␣
 ↪mi_paths_aoa)

#   Build Mitsuba path-level DataFrame
df_mitsuba_paths = pd.DataFrame({
  "source"             : "Mitsuba",
  "bs_x"               : BS_X,
  "bs_y"               : BS_Y,
  "bs_z"               : BS_H,
  "ue_x"               : ue_positions[0][0],
  "ue_y"               : ue_positions[0][1],
  "ue_z"               : ue_positions[0][2],
  "carrier_freq_Hz"    : fc,
  "path_index"         : np.arange(len(mi_delays_aoa)),
  "delay_ns"           : mi_delays_aoa,
  "amplitude_abs"      : mi_amps_aoa,
  "channel_gain_linear": mi_path_gains,
  "channel_gain_dB"    : mi_path_gains_dB,
  "az_aoa_deg"         : mi_az_aoa,
  "el_aoa_deg"         : mi_el_aoa,
  "az_aod_deg"         : mi_az_aod,
  "el_aod_deg"         : mi_el_aod,
})

mitsuba_paths_csv = os.path.join(scene_dir, "mitsuba_paths.csv")
df_mitsuba_paths.to_csv(mitsuba_paths_csv, index=False)
print(f"Saved Mitsuba path dataset -> {mitsuba_paths_csv}")

#   Preview
print("\n--- Mitsuba paths (first 5 rows) ---")
print(df_mitsuba_paths.head())
```

```
Saved Mitsuba path dataset ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/mitsuba_paths.csv

--- Mitsuba paths (first 5 rows) ---
    source   bs_x   bs_y  bs_z  ue_x  ue_y  ue_z  carrier_freq_Hz  path_index  \
0  Mitsuba  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           0
1  Mitsuba  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           1
2  Mitsuba  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           2
3  Mitsuba  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           3
4  Mitsuba  100.0  120.0  16.0   0.0   0.0   1.5     3.500000e+09           4

     delay_ns  amplitude_abs  channel_gain_linear  channel_gain_dB  \
0  523.283824   6.374439e-03         4.063348e-05       -43.911160
1  583.214722   2.066390e-06         4.269966e-12      -113.695755
2  542.868455   1.178138e-06         1.388009e-12      -118.576077
```

```
3  652.878093   4.936189e-07         2.436596e-13      -126.132164
4  613.093798   2.005817e-06         4.023300e-12      -113.954176

   az_aoa_deg  el_aoa_deg  az_aod_deg  el_aod_deg
0   50.194429    5.303382 -129.805571   -5.303382
1   55.822289   -0.563592 -124.177711    0.563592
2   55.410635   -0.675196 -124.589365    0.675196
3   66.555097   -0.577206 -113.444903    0.577206
4   54.705569   -0.528838 -125.294431    0.528838
```

[22]:

```python
import h5py

h5_path = os.path.join(scene_dir, "sionna_dataset.h5")

with h5py.File(h5_path, "w") as f:

  #   Metadata
  meta = f.create_group("metadata")
  meta.attrs["source"]          = "Sionna RT"
  meta.attrs["carrier_freq_Hz"] = fc
  meta.attrs["scene"]           = "OtaniemiScene"

  #   BS / UE locations
  loc = f.create_group("locations")
  loc.create_dataset("bs_position",  data=np.array([BS_X, BS_Y, BS_H]))
  loc.create_dataset("ue_positions", data=np.array(ue_positions))
  loc["bs_position"].attrs["columns"]  = ["x", "y", "z"]
  loc["ue_positions"].attrs["columns"] = ["x", "y", "z"]

  #   CIR (complex path coefficients & delays)
  cir_grp = f.create_group("CIR")
  # a: [batch, rx, rx_ant, tx, tx_ant, paths, time]
  cir_grp.create_dataset("a_real",  data=a.real,  compression="gzip")
  cir_grp.create_dataset("a_imag",  data=a.imag,  compression="gzip")
  cir_grp.create_dataset("tau_s",   data=tau,     compression="gzip")
  cir_grp.create_dataset("tau_ns",  data=tau / 1e-9, compression="gzip")
  cir_grp.attrs["shape_description"] = \
    "[batch, rx, rx_ant, tx, tx_ant, paths, time]"

  #   Per-path features (UE0, first antenna pair)
  path_grp = f.create_group("paths")
  path_grp.create_dataset("delay_ns",           data=sionna_tau_ns_exp)
  path_grp.create_dataset("amplitude_abs",      data=sionna_amp_exp)
  path_grp.create_dataset("channel_gain_linear",data=sionna_gain_path)
  path_grp.create_dataset("channel_gain_dB",    data=sionna_gain_dB_exp)
  path_grp.create_dataset("az_aoa_deg",         data=sionna_az_aoa_exp)
  path_grp.create_dataset("el_aoa_deg",         data=sionna_el_aoa_exp)
```

```
  path_grp.create_dataset("az_aod_deg",         data=sionna_az_aod_exp)
  path_grp.create_dataset("el_aod_deg",         data=sionna_el_aod_exp)
  path_grp.attrs["link"] = "UE0, rx_ant=0, tx_ant=0"

  #   Frequency-domain channel (OFDM)
  freq_grp = f.create_group("frequency_domain")
  freq_grp.create_dataset("subcarrier_freq_Hz", data=fc + freq_sel)
  freq_grp.create_dataset("H_real",             data=H_ue0.real, ␣
↪compression="gzip")
  freq_grp.create_dataset("H_imag",             data=H_ue0.imag, ␣
↪compression="gzip")
  freq_grp.create_dataset("H_magnitude",        data=np.abs(H_ue0))
  freq_grp.create_dataset("H_phase_deg",        data=np.degrees(np.
↪angle(H_ue0)))
  freq_grp.create_dataset("H_gain_dB",          data=20 * np.log10(np.
↪abs(H_ue0) + 1e-30))
  freq_grp.attrs["subcarrier_spacing_Hz"] = subcarrier_spacing
  freq_grp.attrs["fft_size"]              = fft_size
  freq_grp.attrs["link"]                  = "UE0, rx_ant=0, tx_ant=0"

print(f"Saved Sionna HDF5 dataset -> {h5_path}")

#   Quick verification
with h5py.File(h5_path, "r") as f:
  def print_tree(name, obj):
    indent = "  " * name.count("/")
    if isinstance(obj, h5py.Dataset):
      print(f"{indent}{name}: shape={obj.shape}, dtype={obj.dtype}")
    else:
      print(f"{indent}{name}/")
  f.visititems(print_tree)

  # Build a LaTeX table describing the Sionna HDF5 dataset structure
  dataset_rows = []

  def collect_dataset_info(name, obj):
    if isinstance(obj, h5py.Dataset):
      dataset_rows.append({
        "Path": name,
        "Shape": str(tuple(obj.shape)),
        "DType": str(obj.dtype),
      })

  f.visititems(collect_dataset_info)

  df_h5_schema = pd.DataFrame(dataset_rows).sort_values("Path").
↪reset_index(drop=True)
```

```python
sections_dir = os.path.join(scene_dir, "sections")
os.makedirs(sections_dir, exist_ok=True)
latex_path = os.path.join(sections_dir, "sionna_dataset.tex")

latex_table = df_h5_schema.to_latex(
  index=False,
  caption="Sionna dataset structure (HDF5)",
  label="tab:sionna_dataset",
  escape=True
)

with open(latex_path, "w") as tex_file:
  tex_file.write(latex_table)

print(f"Saved LaTeX table -> {latex_path}")

print(f"File size: {os.path.getsize(h5_path) / 1024:.1f} KB")
```

```
Saved Sionna HDF5 dataset ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/sionna_dataset.h5
CIR/
  CIR/a_imag: shape=(2, 1, 8, 32, 139, 1), dtype=float32
  CIR/a_real: shape=(2, 1, 8, 32, 139, 1), dtype=float32
  CIR/tau_ns: shape=(2, 8, 139), dtype=float32
  CIR/tau_s: shape=(2, 8, 139), dtype=float32
frequency_domain/
  frequency_domain/H_gain_dB: shape=(3168,), dtype=float32
  frequency_domain/H_imag: shape=(3168,), dtype=float32
  frequency_domain/H_magnitude: shape=(3168,), dtype=float32
  frequency_domain/H_phase_deg: shape=(3168,), dtype=float32
  frequency_domain/H_real: shape=(3168,), dtype=float32
  frequency_domain/subcarrier_freq_Hz: shape=(3168,), dtype=float32
locations/
  locations/bs_position: shape=(3,), dtype=float64
  locations/ue_positions: shape=(2, 3), dtype=float64
metadata/
paths/
  paths/amplitude_abs: shape=(139,), dtype=float32
  paths/az_aoa_deg: shape=(139,), dtype=float32
  paths/az_aod_deg: shape=(139,), dtype=float32
  paths/channel_gain_dB: shape=(139,), dtype=float32
  paths/channel_gain_linear: shape=(139,), dtype=float32
  paths/delay_ns: shape=(139,), dtype=float32
  paths/el_aoa_deg: shape=(139,), dtype=float32
  paths/el_aod_deg: shape=(139,), dtype=float32
Saved LaTeX table -> /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine
```

```
Learning for Wireless Comunications E7340/Project/sections/sionna_dataset.tex
File size: 297.5 KB
```

[23]:
```python
import h5py

mi_h5_path = os.path.join(scene_dir, "mitsuba_dataset.h5")

#   Mitsuba CIR: complex path coefficients
# Phase from propagation delay at carrier: a_k * exp(-j*2*pi*fc*tau_k)
tau_s_mi  = mi_delays_aoa * 1e-9          # ns -> s
cir_complex_mi = mi_amps_aoa * np.exp(-1j * 2 * np.pi * fc * tau_s_mi)

#   Mitsuba frequency-domain channel
# H(f) = sum_k a_k * exp(-j * 2*pi * f_abs * tau_k)
# Uses the same OFDM subcarrier grid as the Sionna export (freq_sel, fft_size)
freq_sel_np = np.asarray(freq_sel, dtype=np.float64)   # ensure plain NumPy
f_abs_mi    = fc + freq_sel_np                          # absolute subcarrier␣
 ↪frequencies [Hz]
H_mi = np.zeros(len(f_abs_mi), dtype=complex)
for a_k, t_k in zip(mi_amps_aoa, tau_s_mi):
    H_mi += a_k * np.exp(-1j * 2 * np.pi * f_abs_mi * t_k)

with h5py.File(mi_h5_path, "w") as f:

    #   Metadata
    meta = f.create_group("metadata")
    meta.attrs["source"]          = "Mitsuba Ray-Tracer"
    meta.attrs["carrier_freq_Hz"] = fc
    meta.attrs["scene"]           = "OtaniemiScene"

    #   BS / UE locations
    loc = f.create_group("locations")
    loc.create_dataset("bs_position",  data=np.array([BS_X, BS_Y, BS_H]))
    loc.create_dataset("ue_positions", data=np.array(ue_positions))
    loc["bs_position"].attrs["columns"]  = ["x", "y", "z"]
    loc["ue_positions"].attrs["columns"] = ["x", "y", "z"]

    #   CIR (complex path coefficients & delays)
    cir_grp = f.create_group("CIR")
    cir_grp.create_dataset("a_real",  data=cir_complex_mi.real,␣
 ↪compression="gzip")
    cir_grp.create_dataset("a_imag",  data=cir_complex_mi.imag,␣
 ↪compression="gzip")
    cir_grp.create_dataset("tau_s",   data=tau_s_mi,           ␣
 ↪compression="gzip")
    cir_grp.create_dataset("tau_ns",  data=mi_delays_aoa,      ␣
 ↪compression="gzip")
```

```
    cir_grp.attrs["shape_description"] = "[num_paths]"
    cir_grp.attrs["amplitude_model"]   = \
        "cos_i*cos_r/(d_tx*d_rx); LOS=1/r; phase=exp(-j*2pi*fc*tau)"
    cir_grp.attrs["link"]              = "UE0"

    #   Per-path features (UE0)
    path_grp = f.create_group("paths")
    path_grp.create_dataset("delay_ns",            data=mi_delays_aoa)
    path_grp.create_dataset("amplitude_abs",       data=mi_amps_aoa)
    path_grp.create_dataset("channel_gain_linear", data=mi_path_gains)
    path_grp.create_dataset("channel_gain_dB",     data=mi_path_gains_dB)
    path_grp.create_dataset("az_aoa_deg",          data=mi_az_aoa)
    path_grp.create_dataset("el_aoa_deg",          data=mi_el_aoa)
    path_grp.create_dataset("az_aod_deg",          data=mi_az_aod)
    path_grp.create_dataset("el_aod_deg",          data=mi_el_aod)
    path_grp.attrs["link"]            = "UE0"
    path_grp.attrs["max_depth"]       = 2
    path_grp.attrs["n_probe_rays"]    = 2048
    path_grp.attrs["amplitude_model"] = "cos_i * cos_r / (d_tx * d_rx); LOS=1/r"

    #   Frequency-domain channel (OFDM)
    freq_grp = f.create_group("frequency_domain")
    freq_grp.create_dataset("subcarrier_freq_Hz", data=f_abs_mi)
    freq_grp.create_dataset("H_real",             data=H_mi.real, ␣
↪compression="gzip")
    freq_grp.create_dataset("H_imag",             data=H_mi.imag, ␣
↪compression="gzip")
    freq_grp.create_dataset("H_magnitude",        data=np.abs(H_mi))
    freq_grp.create_dataset("H_phase_deg",        data=np.degrees(np.
↪angle(H_mi)))
    freq_grp.create_dataset("H_gain_dB",          data=20 * np.log10(np.
↪abs(H_mi) + 1e-30))
    freq_grp.attrs["subcarrier_spacing_Hz"] = subcarrier_spacing
    freq_grp.attrs["fft_size"]              = fft_size
    freq_grp.attrs["link"]                  = "UE0"
    freq_grp.attrs["method"] = \
        "H(f) = sum_k a_k * exp(-j*2*pi*f_abs*tau_k); f_abs = fc +␣
↪subcarrier_offset"

print(f"Saved Mitsuba HDF5 dataset -> {mi_h5_path}")

#   Quick verification
with h5py.File(mi_h5_path, "r") as f:
    def print_tree(name, obj):
        indent = "  " * name.count("/")
        if isinstance(obj, h5py.Dataset):
            print(f"{indent}{name}: shape={obj.shape}, dtype={obj.dtype}")
```

```
        else:
            print(f"{indent}{name}/")
    f.visititems(print_tree)

    # Build a LaTeX table describing the Mitsuba HDF5 dataset structure
    dataset_rows = []

    def collect_dataset_info(name, obj):
        if isinstance(obj, h5py.Dataset):
            dataset_rows.append({
                "Path": name,
                "Shape": str(tuple(obj.shape)),
                "DType": str(obj.dtype),
            })

    f.visititems(collect_dataset_info)

    df_h5_schema = pd.DataFrame(dataset_rows).sort_values("Path").
 ↪reset_index(drop=True)

    sections_dir = os.path.join(scene_dir, "sections")
    os.makedirs(sections_dir, exist_ok=True)
    latex_path = os.path.join(sections_dir, "mitsuba_dataset.tex")

    latex_table = df_h5_schema.to_latex(
        index=False,
        caption="Mitsuba dataset structure (HDF5)",
        label="tab:mitsuba_dataset",
        escape=True
    )

    with open(latex_path, "w") as tex_file:
        tex_file.write(latex_table)

    print(f"Saved LaTeX table -> {latex_path}")

print(f"File size: {os.path.getsize(mi_h5_path) / 1024:.1f} KB")
```

```
Saved Mitsuba HDF5 dataset ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/mitsuba_dataset.h5
CIR/
  CIR/a_imag: shape=(852,), dtype=float64
  CIR/a_real: shape=(852,), dtype=float64
  CIR/tau_ns: shape=(852,), dtype=float64
  CIR/tau_s: shape=(852,), dtype=float64
frequency_domain/
  frequency_domain/H_gain_dB: shape=(3168,), dtype=float64
```

```
  frequency_domain/H_imag: shape=(3168,), dtype=float64
  frequency_domain/H_magnitude: shape=(3168,), dtype=float64
  frequency_domain/H_phase_deg: shape=(3168,), dtype=float64
  frequency_domain/H_real: shape=(3168,), dtype=float64
  frequency_domain/subcarrier_freq_Hz: shape=(3168,), dtype=float64
locations/
  locations/bs_position: shape=(3,), dtype=float64
  locations/ue_positions: shape=(2, 3), dtype=float64
metadata/
paths/
  paths/amplitude_abs: shape=(852,), dtype=float64
  paths/az_aoa_deg: shape=(852,), dtype=float64
  paths/az_aod_deg: shape=(852,), dtype=float64
  paths/channel_gain_dB: shape=(852,), dtype=float64
  paths/channel_gain_linear: shape=(852,), dtype=float64
  paths/delay_ns: shape=(852,), dtype=float64
  paths/el_aoa_deg: shape=(852,), dtype=float64
  paths/el_aod_deg: shape=(852,), dtype=float64
Saved LaTeX table -> /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine
Learning for Wireless Comunications E7340/Project/sections/mitsuba_dataset.tex
File size: 257.0 KB
```

[24]:
```python
import pandas as pd
import numpy as np
import os
from pathlib import Path
import h5py

import matplotlib.pyplot as plt

#   Load datasets from HDF5 files

sionna_h5 = os.path.join(scene_dir, "sionna_dataset.h5")
mitsuba_h5 = os.path.join(scene_dir, "mitsuba_dataset.h5")

with h5py.File(sionna_h5, "r") as f_s, h5py.File(mitsuba_h5, "r") as f_m:
  # Extract Sionna data
  sionna_delay = f_s["paths/delay_ns"][:]
  sionna_amplitude = f_s["paths/amplitude_abs"][:]
  sionna_gain_dB = f_s["paths/channel_gain_dB"][:]
  sionna_az_aoa = f_s["paths/az_aoa_deg"][:]
  sionna_el_aoa = f_s["paths/el_aoa_deg"][:]
  sionna_az_aod = f_s["paths/az_aod_deg"][:]
  sionna_el_aod = f_s["paths/el_aod_deg"][:]
  sionna_total_gain = np.sum(10**(sionna_gain_dB / 10))
  sionna_total_gain_dB = 10 * np.log10(sionna_total_gain + 1e-30)
```

```
  # Extract Mitsuba data
  mitsuba_delay = f_m["paths/delay_ns"][:]
  mitsuba_amplitude = f_m["paths/amplitude_abs"][:]
  mitsuba_gain_dB = f_m["paths/channel_gain_dB"][:]
  mitsuba_az_aoa = f_m["paths/az_aoa_deg"][:]
  mitsuba_el_aoa = f_m["paths/el_aoa_deg"][:]
  mitsuba_az_aod = f_m["paths/az_aod_deg"][:]
  mitsuba_el_aod = f_m["paths/el_aod_deg"][:]
  mitsuba_total_gain = np.sum(10**(mitsuba_gain_dB / 10))
  mitsuba_total_gain_dB = 10 * np.log10(mitsuba_total_gain + 1e-30)

#   Statistical Analysis
def compute_statistics(name, delays, amplitudes, gains_dB):
  """Compute key statistics for a dataset."""
  return {
    "Tool": name,
    "Num Paths": len(delays),
    "Min Delay (ns)": delays.min(),
    "Max Delay (ns)": delays.max(),
    "Mean Delay (ns)": delays.mean(),
    "Std Delay (ns)": delays.std(),
    "Min Amplitude": amplitudes.min(),
    "Max Amplitude": amplitudes.max(),
    "Mean Amplitude": amplitudes.mean(),
    "Min Gain (dB)": gains_dB.min(),
    "Max Gain (dB)": gains_dB.max(),
    "Mean Gain (dB)": gains_dB.mean(),
    "Total Gain (dB)": 10 * np.log10(np.sum(10**(gains_dB / 10)) + 1e-30),
  }

stats_sionna = compute_statistics("Sionna", sionna_delay, sionna_amplitude,␣
 ↪sionna_gain_dB)
stats_mitsuba = compute_statistics("Mitsuba", mitsuba_delay, mitsuba_amplitude,␣
 ↪mitsuba_gain_dB)

df_stats = pd.DataFrame([stats_sionna, stats_mitsuba])

#   RMS Delay Spread
def rms_delay_spread(delays, gains):
  """Compute RMS delay spread from path delays and gains."""
  gains_linear = 10**(gains / 10)
  mean_delay = np.sum(delays * gains_linear) / np.sum(gains_linear)
  rms = np.sqrt(np.sum((delays - mean_delay)**2 * gains_linear) / np.
 ↪sum(gains_linear))
  return rms, mean_delay

rms_sionna, mean_delay_sionna = rms_delay_spread(sionna_delay, sionna_gain_dB)
```

```
rms_mitsuba, mean_delay_mitsuba = rms_delay_spread(mitsuba_delay,␣
↪mitsuba_gain_dB)

df_stats["RMS Delay Spread (ns)"] = [rms_sionna, rms_mitsuba]
df_stats["Mean Delay (ns)"] = [mean_delay_sionna, mean_delay_mitsuba]

#   Angular Spread Analysis
def angular_spread(azimuths, elevations, gains_dB):
  """Compute mean and std of azimuth/elevation angles weighted by gain."""
  gains_linear = 10**(gains_dB / 10)
  w = gains_linear / np.sum(gains_linear)

  # Unwrap angles for proper weighting (convert to radians, compute mean␣
↪vector, convert back)
  az_rad = np.radians(azimuths)
  el_rad = np.radians(elevations)

  mean_az_vector = np.average(np.cos(az_rad), weights=w) + 1j * np.average(np.
↪sin(az_rad), weights=w)
  mean_az = np.degrees(np.angle(mean_az_vector))

  mean_el = np.average(el_rad, weights=w)
  mean_el = np.degrees(mean_el)

  std_az = np.sqrt(np.average((azimuths - mean_az)**2, weights=w))
  std_el = np.sqrt(np.average((elevations - mean_el)**2, weights=w))

  return mean_az, std_az, mean_el, std_el

mean_az_aoa_s, std_az_aoa_s, mean_el_aoa_s, std_el_aoa_s = angular_spread(
  sionna_az_aoa, sionna_el_aoa, sionna_gain_dB
)
mean_az_aod_s, std_az_aod_s, mean_el_aod_s, std_el_aod_s = angular_spread(
  sionna_az_aod, sionna_el_aod, sionna_gain_dB
)

mean_az_aoa_m, std_az_aoa_m, mean_el_aoa_m, std_el_aoa_m = angular_spread(
  mitsuba_az_aoa, mitsuba_el_aoa, mitsuba_gain_dB
)
mean_az_aod_m, std_az_aod_m, mean_el_aod_m, std_el_aod_m = angular_spread(
  mitsuba_az_aod, mitsuba_el_aod, mitsuba_gain_dB
)

df_angles = pd.DataFrame({
  "Metric": ["AoA Azimuth Mean", "AoA Azimuth Std", "AoA Elevation Mean", "AoA␣
↪Elevation Std",
```

```
        "AoD Azimuth Mean", "AoD Azimuth Std", "AoD Elevation Mean", "AoD␣
↪Elevation Std"],
  "Sionna": [mean_az_aoa_s, std_az_aoa_s, mean_el_aoa_s, std_el_aoa_s,
        mean_az_aod_s, std_az_aod_s, mean_el_aod_s, std_el_aod_s],
  "Mitsuba": [mean_az_aoa_m, std_az_aoa_m, mean_el_aoa_m, std_el_aoa_m,
       mean_az_aod_m, std_az_aod_m, mean_el_aod_m, std_el_aod_m],
})

#   Path-level comparison (match dominant paths by delay)
def match_paths(delays_1, delays_2, threshold_ns=1.0):
  """Match paths between two methods by delay (within threshold)."""
  matches = []
  for i, d1 in enumerate(delays_1):
    candidates = np.where(np.abs(delays_2 - d1) < threshold_ns)[0]
    if len(candidates) > 0:
      j = candidates[0]
      matches.append((i, j, abs(d1 - delays_2[j])))
  return matches

matches = match_paths(sionna_delay, mitsuba_delay, threshold_ns=2.0)

if len(matches) > 0:
  matched_indices_s = [m[0] for m in matches]
  matched_indices_m = [m[1] for m in matches]

  delay_error = np.abs(sionna_delay[matched_indices_s] -␣
↪mitsuba_delay[matched_indices_m])
  gain_error = np.abs(sionna_gain_dB[matched_indices_s] -␣
↪mitsuba_gain_dB[matched_indices_m])

  df_path_comparison = pd.DataFrame({
    "Path Index": matched_indices_s,
    "Sionna Delay (ns)": sionna_delay[matched_indices_s],
    "Mitsuba Delay (ns)": mitsuba_delay[matched_indices_m],
    "Delay Error (ns)": delay_error,
    "Sionna Gain (dB)": sionna_gain_dB[matched_indices_s],
    "Mitsuba Gain (dB)": mitsuba_gain_dB[matched_indices_m],
    "Gain Error (dB)": gain_error,
  })
else:
  df_path_comparison = pd.DataFrame()

#   Generate LaTeX Report
sections_dir = os.path.join(scene_dir, "sections")
os.makedirs(sections_dir, exist_ok=True)
latex_file = os.path.join(sections_dir, "evaluation.tex")
```

```
latex_content = r"""
\section{Evaluation: Sionna RT vs.\ Mitsuba Ray-Tracer}
\label{sec:evaluation}

This section presents a quantitative comparison of channel characterization␣
↪results between
the Sionna RT module and a custom Mitsuba ray-tracing implementation.

"""

#   Table 1: Overall Statistics
latex_content += r"""
\subsection{Path-Level Statistics}

""" + df_stats.to_latex(
  index=False,
  caption="Path-level statistics: Sionna vs.\ Mitsuba",
  label="tab:path_stats",
  float_format=lambda x: f"{x:.2f}" if abs(x) < 1000 else f"{x:.2e}",
) + "\n\n"

#   Table 2: Angular Spread
latex_content += r"""
\subsection{Angular Spread Analysis}

""" + df_angles.to_latex(
  index=False,
  caption="Angle of Arrival (AoA) and Angle of Departure (AoD) statistics␣
↪(degrees)",
  label="tab:angular_stats",
  float_format=lambda x: f"{x:.2f}",
) + "\n\n"

#   Table 3: Matched Paths
if not df_path_comparison.empty:
  latex_content += r"""
\subsection{Matched Path Comparison}

Paths with delays within 2\,ns are considered matched. Table~\ref{tab:
↪path_match} shows
the delay and gain errors for matched paths.

""" + df_path_comparison.head(10).to_latex(
    index=False,
    caption=f"Matched path comparison (first 10 of {len(df_path_comparison)}␣
↪matches)",
    label="tab:path_match",
```

```
    float_format=lambda x: f"{x:.3f}",
  ) + "\n\n"

  latex_content += f"""
\\paragraph{{Match Statistics}}
Out of {len(sionna_delay)} Sionna paths and {len(mitsuba_delay)} Mitsuba paths,
{len(df_path_comparison)} paths were matched with a delay threshold of 2\\,ns.
Mean delay error: {delay_error.mean():.3f}\\,ns;
RMS gain error: {np.sqrt(np.mean(gain_error**2)):.3f}\\,dB.

"""

#   Key Findings
latex_content += r"""
\subsection{Key Findings}

\begin{itemize}
  \item \textbf{Path Count}: Sionna identified """ + f"{len(sionna_delay)}" +␣
↪r""" dominant paths,
  while Mitsuba's Monte Carlo sampling returned """ + f"{len(mitsuba_delay)}" +␣
↪r""" candidate paths.

  \item \textbf{Delay Spread}: Sionna RMS delay spread is """ + f"{rms_sionna:.
↪2f}" + r""" ns,
  vs.\ Mitsuba """ + f"{rms_mitsuba:.2f}" + r""" ns, indicating similar␣
↪multipath structure.

  \item \textbf{Total Channel Gain}: Sionna: """ + f"{sionna_total_gain_dB:.
↪2f}" + r""" dB;
  Mitsuba: """ + f"{mitsuba_total_gain_dB:.2f}" + r""" dB. Differences reflect␣
↪amplitude model assumptions.

  \item \textbf{Angular Consistency}: Mean AoA (azimuth) Sionna: """ +␣
↪f"{mean_az_aoa_s:.1f}°" + r""";
  Mitsuba: """ + f"{mean_az_aoa_m:.1f}°" + r""". Angular spread (Sionna: """ +␣
↪f"{std_az_aoa_s:.1f}°" + r""";
  Mitsuba: """ + f"{std_az_aoa_m:.1f}°" + r""") indicates similar scattering␣
↪characteristics.

  \item \textbf{Interpretation}: Sionna is the primary channel simulator␣
↪(communication-focused);
  Mitsuba serves as a geometric validation tool. Differences in absolute gains␣
↪are expected
  due to different amplitude models and path generation logic.
\end{itemize}
```

```
"""

# Save LaTeX file
with open(latex_file, "w") as f:
  f.write(latex_content)

print(f"Saved evaluation report -> {latex_file}")
print("\n" + "="*70)
print("EVALUATION SUMMARY")
print("="*70)
print(df_stats.to_string(index=False))
print("\n" + "-"*70)
print(df_angles.to_string(index=False))
print("\n" + "-"*70)
if not df_path_comparison.empty:
  print(f"Matched paths: {len(df_path_comparison)} / {len(sionna_delay)}")
  print(f"Mean delay error: {delay_error.mean():.3f} ns")
  print(f"RMS gain error: {np.sqrt(np.mean(gain_error**2)):.3f} dB")
```

```
Saved evaluation report ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/sections/evaluation.tex

======================================================================
EVALUATION SUMMARY
======================================================================
   Tool  Num Paths  Min Delay (ns)  Max Delay (ns)  Mean Delay (ns)  Std Delay
(ns)  Min Amplitude  Max Amplitude  Mean Amplitude  Min Gain (dB)  Max Gain (dB)
Mean Gain (dB)  Total Gain (dB)  RMS Delay Spread (ns)
 Sionna        139   -1.000000e+09     1936.514404        10.840940
4.560268e+08   0.000000e+00       0.000104        0.000002    -300.000031
-79.698174     -195.103714       -77.789215              36.882988
Mitsuba        852    5.232838e+02     1651.412970       523.567045
1.149017e+02   7.707097e-09       0.006374        0.000010    -162.262183
-43.911160     -116.235471       -43.906672               9.926787

----------------------------------------------------------------------
            Metric      Sionna     Mitsuba
  AoA Azimuth Mean   57.725784   50.204769
   AoA Azimuth Std   14.106914    1.297519
AoA Elevation Mean   86.480392    5.299377
 AoA Elevation Std    4.073063    0.149940
  AoD Azimuth Mean -133.502304 -129.795231
   AoD Azimuth Std   10.477477    1.192812
AoD Elevation Mean   95.365822   -5.299377
 AoD Elevation Std    0.426702    0.149940

----------------------------------------------------------------------
```

```
Matched paths: 23 / 139
Mean delay error: 0.975 ns
RMS gain error: 53.082 dB
```

5 Step 2: Fingerprint Localization (WKNN & NN-based)

[25]:
```
#%pip install scikit-learn
import numpy as np
import pandas as pd
import h5py
import os
import inspect
from itertools import combinations
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sionna.rt import Receiver

#   Full training grid at 10 m resolution
# 10 m spacing → geometric error floor of  7 m (half-diagonal of one cell),
# which makes sub-10 m p90 physically achievable.  ~700 grid points total.
x_min, x_max  = -300, 300
y_min, y_max  = -200, 250
grid_spacing  = 10   # metres

x_train_locs = np.arange(x_min, x_max + grid_spacing, grid_spacing)
y_train_locs = np.arange(y_min, y_max + grid_spacing, grid_spacing)
xx_train, yy_train = np.meshgrid(x_train_locs, y_train_locs)
train_positions = np.column_stack([
    xx_train.ravel(), yy_train.ravel(),
    np.full(xx_train.ravel().shape, UE_H)
])
n_all = len(train_positions)
print(f"Full grid: {n_all} positions ({len(x_train_locs)} ×␣
 ↪{len(y_train_locs)}, spacing={grid_spacing} m)")
train_positions_subset = train_positions

n_tx = len(tx_positions)

#   Cache tag encodes grid spacing + feature type + key config params
# Include n_tx and fft_size so any config change invalidates the cache.
```

```
_CACHE_TAG = f"dB_mag_gd_tdoa_aoa_gs{grid_spacing}_ntx{n_tx}_fft{fft_size}"
_fp_h5     = os.path.join(scene_dir, "fingerprint_rt_dataset.h5")
_rt_cache_valid = False
if os.path.exists(_fp_h5):
    try:
        with h5py.File(_fp_h5, "r") as _f:
            _n_cached    = int(_f.attrs.get("n_samples",    -1))
            _ntx_cached  = int(_f.attrs.get("n_tx",          0))
            _fft_cached  = int(_f.attrs.get("fft_size",      0))
            _feat_cached = _f.attrs.get("feature_type", "")
            if (_n_cached == n_all and _ntx_cached == n_tx
                    and _fft_cached == fft_size and _CACHE_TAG in _feat_cached):
                csi_fingerprints      = _f["csi_fingerprints"][:]
                fingerprint_positions = _f["fingerprint_positions"][:]
                _rt_cache_valid = True
                print(f"Loaded RT fingerprint cache  ({_fp_h5})")
                print(f"  Feature shape: {csi_fingerprints.shape}")
            else:
                print(f"Cache mismatch (n={_n_cached} vs {n_all}, "
                      f"n_tx={_ntx_cached} vs {n_tx}, "
                      f"fft={_fft_cached} vs {fft_size}); recomputing…")
    except Exception as _e:
        print(f"Cache load failed ({_e}); recomputing…")

if not _rt_cache_valid:
    #   Swap scene receivers for the fingerprint grid
    _saved_rxs = {name: list(np.array(rx.position))
                  for name, rx in scene.receivers.items()}
    for _n in list(_saved_rxs):
        scene.remove(_n)

    for _i, _pos in enumerate(train_positions_subset):
        scene.add(Receiver(name=f"fp_{_i}", position=_pos.tolist()))
    print(f"Added {n_all} fingerprint receivers to scene")

    #   Run PathSolver over all grid positions in one batch
    _fp_kw = dict(max_depth=3, los=True, reflection=True,
                  diffraction=True, scattering=False)
    _sup   = set(inspect.signature(path_solver.__call__).parameters)
    _fp_kw = {k: v for k, v in _fp_kw.items() if k in _sup}
    print("Running PathSolver on fingerprint grid (one batch, may take several␣
 ↪minutes) …")
    fp_paths = path_solver(scene=scene, **_fp_kw)
    print("PathSolver complete.")

    #   CIR → OFDM channel matrix
    a_fp, tau_fp = fp_paths.cir(out_type='numpy')
```

```
    print(f"a_fp shape: {a_fp.shape}, dtype={a_fp.dtype}")
    print(f"tau_fp shape: {tau_fp.shape}, dtype={tau_fp.dtype}")

    # Keep dtypes compact to limit memory use
    a_fp   = a_fp.astype(np.complex64, copy=False)
    tau_fp = tau_fp.astype(np.float32,  copy=False)

    # Average over TX antennas early to reduce tensor size before OFDM␣
↪conversion
    if a_fp.ndim >= 6 and a_fp.shape[3] > 1:
        a_fp = np.mean(a_fp, axis=3, keepdims=True)
        print(f"a_fp shape after TX-ant averaging: {a_fp.shape}")

    # Process receivers in chunks to avoid OOM on large grids
    _orig_cir_to_ofdm = cir_to_ofdm_channel
    def _chunked_cir_to_ofdm(frequencies, a, tau, normalize=True, rx_chunk=64):
        n_rx = a.shape[1]
        h_chunks = []
        for s in range(0, n_rx, rx_chunk):
            e = min(s + rx_chunk, n_rx)
            h_chunks.append(
                _orig_cir_to_ofdm(frequencies, a[:, s:e, ...],
                                  tau[:, s:e, ...], normalize=normalize)
            )
        return tf.concat(h_chunks, axis=1)

    # normalize=False preserves absolute received power per BS - a key
    # discriminative feature for positioning (UEs near a BS receive stronger
    # signal; normalization would erase this spatial variation entirely).
    H_fp = np.array(_chunked_cir_to_ofdm(
        frequencies, a_fp[np.newaxis], tau_fp[np.newaxis], normalize=False
    ))
    # H_fp: [1, n_rx, 1, n_tx, 1, n_time, fft_size]  (tx_ant dim already␣
↪reduced)
    print(f"H_fp shape: {H_fp.shape}")

    #   Feature extraction: dB magnitude + group delay per BS
    # dB magnitude captures spatial power variations (RSS); group delay (diff of
    # unwrapped phase) encodes dominant path ToA.
    # Subcarriers are decimated by SC_STEP (1-in-8) to combat the curse of
    # dimensionality - 396 subcarriers still covers the 100 MHz coherence
    # bandwidth for RT-simulated channels.
    # Group delay is clipped to ±500 samples to prevent explosion on near-zero
    # channels (where np.angle returns random noise → diff → huge values).
    _GD_CLIP = 500.0
    SC_STEP  = 8    # decimate: keep every 8th subcarrier → 396 vs 3168 per BS
```

```
    feat_per_bs = (fft_size // SC_STEP) + (fft_size // SC_STEP - 1)  # 396 +␣
↪395 = 791

    def _extract_fp(H_link):
        """H_link: [n_tx_ant, n_time, fft_size] → 1-D real feature vector."""
        H_avg    = np.mean(H_link, axis=(0, 1))[::SC_STEP]            #␣
↪[fft_size//SC_STEP]
        mag_feat = 20.0 * np.log10(np.abs(H_avg) + 1e-6)             # dB␣
↪magnitude (floor  -120 dB)
        # Only compute group delay where signal is above noise floor (-100 dB)
        pwr_mask = np.abs(H_avg) > 1e-5                                # -100␣
↪dB threshold
        phase    = np.unwrap(np.angle(H_avg))
        gd_raw   = np.diff(phase)                                      #␣
↪[fft_size//SC_STEP - 1]
        gd_feat  = np.where(pwr_mask[:-1] & pwr_mask[1:],
                            np.clip(gd_raw, -_GD_CLIP, _GD_CLIP), 0.0)
        return np.concatenate([mag_feat, gd_feat])

    # OFDM-based features: [n_all, n_tx * feat_per_bs]
    ofdm_feats = np.stack([
        np.concatenate([
            _extract_fp(H_fp[0, rx_i, 0, tx_i, :, :, :])
            for tx_i in range(n_tx)
        ])
        for rx_i in range(n_all)
    ])

    #   TDoA features (exact RT path delays, no bandwidth limit)
    tau_np = np.array(fp_paths.tau)
    # Explicitly reduce to [n_rx, n_tx, n_paths] by selecting first antenna␣
↪pair.
    # fp_paths.tau shape: [n_rx, n_rx_ant, n_tx, n_tx_ant, n_paths] (5D)
    #                  or [n_rx, n_tx, n_paths]                      (3D)
    if tau_np.ndim == 5:      # [n_rx, n_rx_ant, n_tx, n_tx_ant, n_paths]
        tau_np = tau_np[:, 0, :, 0, :]
    elif tau_np.ndim == 4:    # [n_rx, n_rx_ant, n_tx, n_paths]
        tau_np = tau_np[:, 0, :, :]
    elif tau_np.ndim == 2:
        tau_np = tau_np[:, :, np.newaxis]
    # tau_np is now [n_all, n_tx, n_paths]
    print(f"tau_np shape: {tau_np.shape}")

    tau_min = tau_np.min(axis=-1)             # [n_all, n_tx]

    _tdoa_pairs = list(combinations(range(n_tx), 2))
```

```
tdoa_features = np.stack(
    [(tau_min[:, i] - tau_min[:, j]) / 1e-9 for i, j in _tdoa_pairs],
    axis=1
)
print(f"TDoA feature shape: {tdoa_features.shape}  (pairs: {_tdoa_pairs})")

#   AoA features (dominant path per BS)
phi_r_raw   = np.array(fp_paths.phi_r)
theta_r_raw = np.array(fp_paths.theta_r)

def _squeeze_angle(arr):
    # Reduce to [n_rx, n_tx, n_paths] by selecting first antenna pair.
    # phi_r / theta_r shape: [n_rx, n_rx_ant, n_tx, n_tx_ant, n_paths] (5D)
    #                      or [n_rx, n_tx, n_paths]                    ␣
↪(3D)
    if arr.ndim == 5:      # [n_rx, n_rx_ant, n_tx, n_tx_ant, n_paths]
        return arr[:, 0, :, 0, :]
    elif arr.ndim == 4:    # [n_rx, n_rx_ant, n_tx, n_paths]
        return arr[:, 0, :, :]
    elif arr.ndim == 2:
        return arr[:, :, np.newaxis]
    return arr             # already [n_rx, n_tx, n_paths]

phi_r_np   = _squeeze_angle(phi_r_raw)
theta_r_np = _squeeze_angle(theta_r_raw)

dom_path_idx = tau_np.argmin(axis=-1)
_rx_idx = np.arange(n_all)[:, np.newaxis]
_tx_idx = np.arange(n_tx)[np.newaxis, :]

dom_az = phi_r_np  [_rx_idx, _tx_idx, dom_path_idx]
dom_el = theta_r_np[_rx_idx, _tx_idx, dom_path_idx]

aoa_features = np.concatenate([
    np.sin(dom_az), np.cos(dom_az),
    np.sin(dom_el), np.cos(dom_el)
], axis=1)
print(f"AoA feature shape: {aoa_features.shape}  (sin/cos of az & el per␣
↪BS)")

#   Concatenate all features
csi_fingerprints = np.concatenate([ofdm_feats, tdoa_features,␣
↪aoa_features], axis=1)
fingerprint_positions = train_positions_subset[:, :2].copy()

n_ofdm  = ofdm_feats.shape[1]
n_tdoa  = tdoa_features.shape[1]
```

```
    n_aoa   = aoa_features.shape[1]
    print(f"\nFingerprint feature shape : {csi_fingerprints.shape}")
    print(f"  = {n_ofdm} OFDM (dB-mag+GD, SC_STEP={SC_STEP}) + {n_tdoa} TDoA +␣
↪{n_aoa} AoA  = {n_ofdm+n_tdoa+n_aoa} total")

    #   Save cache
    with h5py.File(_fp_h5, "w") as _f:
        _f.create_dataset("csi_fingerprints",      data=csi_fingerprints,    ␣
↪compression="gzip")
        _f.create_dataset("fingerprint_positions", data=fingerprint_positions,␣
↪compression="gzip")
        _f.create_dataset("H_fp_real",             data=H_fp.real,           ␣
↪compression="gzip")
        _f.create_dataset("H_fp_imag",             data=H_fp.imag,           ␣
↪compression="gzip")
        _f.attrs["n_tx"]         = n_tx
        _f.attrs["fft_size"]     = fft_size
        _f.attrs["grid_spacing"] = grid_spacing
        _f.attrs["n_samples"]    = n_all
        _f.attrs["feature_type"] = f"{_CACHE_TAG}: dB-mag+GD per BS, TDoA␣
↪({len(_tdoa_pairs)} pairs, ns), AoA sin/cos ({4*n_tx}), grid={grid_spacing}m"
    print(f"Saved RT fingerprint cache → {_fp_h5}")

    #   Restore original scene receivers
    for _i in range(n_all):
        scene.remove(f"fp_{_i}")
    for _n, _pos in _saved_rxs.items():
        scene.add(Receiver(name=_n, position=_pos))
    print("Scene receivers restored.")

#   Train/test split and feature standardisation
X_train, X_test, pos_train, pos_test = train_test_split(
    csi_fingerprints, fingerprint_positions, test_size=0.3, random_state=42
)
print(f"Training set : {X_train.shape[0]} samples")
print(f"Test set     : {X_test.shape[0]} samples")
print(f"Feature dim  : {X_train.shape[1]}")

scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

print(f"Dataset ready for localization ({grid_spacing} m grid, dB-mag +␣
↪group-delay + TDoA + AoA, {n_tx} BSes).")
```

```
Full grid: 2806 positions (61 × 46, spacing=10 m)
Loaded RT fingerprint cache
```

```
(/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/fingerprint_rt_dataset.h5)
  Feature shape: (2806, 50740)
Training set : 1964 samples
Test set     : 842 samples
Feature dim  : 50740
Dataset ready for localization (10 m grid, dB-mag + group-delay + TDoA + AoA, 8
BSes).
```

[26]:
```
#   WKNN (Weighted K-Nearest Neighbors) Localization

from sklearn.model_selection import KFold

#   Cross-validate k on training set
# Use 5-fold CV to find the k that minimises mean position error on train data.
k_candidates = list(range(1, 16))
distance_metric = 'euclidean'
cv_errors = {}

kf = KFold(n_splits=5, shuffle=True, random_state=42)
for k in k_candidates:
    fold_errs = []
    for tr_idx, val_idx in kf.split(X_train_scaled):
        _nn = NearestNeighbors(n_neighbors=k, metric=distance_metric)
        _nn.fit(X_train_scaled[tr_idx])
        _dist, _idx = _nn.kneighbors(X_train_scaled[val_idx])
        # IDW prediction
        _w = 1.0 / (_dist + 1e-10); _w /= _w.sum(axis=1, keepdims=True)
        _pred = (_w[:, :, None] * pos_train[tr_idx][_idx]).sum(axis=1)
        fold_errs.append(np.mean(np.linalg.norm(_pred - pos_train[val_idx],␣
 ↪axis=1)))
    cv_errors[k] = np.mean(fold_errs)

best_k = min(cv_errors, key=cv_errors.get)
print("Cross-validation MAE per k:")
for k, e in cv_errors.items():
    marker = " ← best" if k == best_k else ""
    print(f"  k={k:2d}: {e:.2f} m{marker}")

k_neighbors = best_k

#   Build and evaluate WKNN with the best k
nn_model = NearestNeighbors(n_neighbors=k_neighbors, metric=distance_metric)
nn_model.fit(X_train_scaled)

distances, indices = nn_model.kneighbors(X_test_scaled)
```

```python
def wknn_predict(distances, indices, train_positions):
    """Inverse distance weighting (IDW) position estimate."""
    weights = 1.0 / (distances + 1e-10)
    weights /= weights.sum(axis=1, keepdims=True)
    return (weights[:, :, None] * train_positions[indices]).sum(axis=1)

pos_pred_wknn = wknn_predict(distances, indices, pos_train)

error_wknn         = np.linalg.norm(pos_pred_wknn - pos_test, axis=1)
rmse_wknn          = np.sqrt(np.mean(error_wknn**2))
mae_wknn           = np.mean(error_wknn)
median_error_wknn  = np.median(error_wknn)
p90_error_wknn     = np.percentile(error_wknn, 90)

print("\n" + "=" * 70)
print("WKNN LOCALIZATION RESULTS (Inverse Distance Weighting)")
print("=" * 70)
print(f"Configuration: k={k_neighbors} (CV-optimised),␣
 ↪metric={distance_metric}")
print(f"Training samples: {len(pos_train)}, Test samples: {len(pos_test)}")
print(f"RMSE: {rmse_wknn:.2f} m")
print(f"MAE:  {mae_wknn:.2f} m")
print(f"Median Error: {median_error_wknn:.2f} m")
print(f"90th percentile: {p90_error_wknn:.2f} m")
print(f"Min error: {error_wknn.min():.2f} m, Max error: {error_wknn.max():.2f}␣
 ↪m")
```

```
Cross-validation MAE per k:
  k= 1: 119.80 m
  k= 2: 110.55 m
  k= 3: 106.48 m
  k= 4: 104.46 m
  k= 5: 103.80 m
  k= 6: 103.62 m ← best
  k= 7: 103.86 m
  k= 8: 104.06 m
  k= 9: 104.41 m
  k=10: 104.85 m
  k=11: 105.26 m
  k=12: 105.43 m
  k=13: 105.46 m
  k=14: 105.62 m
  k=15: 105.74 m

======================================================================
WKNN LOCALIZATION RESULTS (Inverse Distance Weighting)
======================================================================
Configuration: k=6 (CV-optimised), metric=euclidean
```

```
Training samples: 1964, Test samples: 842
RMSE: 124.73 m
MAE:  103.13 m
Median Error: 88.17 m
90th percentile: 198.50 m
Min error: 1.37 m, Max error: 353.72 m
```

[27]:
```
#   NN-based Regression: Deep learning model for position regression

tf.random.set_seed(42)
np.random.seed(42)

input_dim = X_train_scaled.shape[1]

# Wider network + mild dropout + skip connection from input to 2nd hidden layer
inp = keras.Input(shape=(input_dim,))
x1  = layers.Dense(256, activation='relu')(inp)
x1  = layers.BatchNormalization()(x1)
x1  = layers.Dropout(0.1)(x1)
x2  = layers.Dense(128, activation='relu')(x1)
x2  = layers.BatchNormalization()(x2)
x2  = layers.Dropout(0.1)(x2)
x3  = layers.Dense(64, activation='relu')(x2)
x3  = layers.Dropout(0.1)(x3)
x4  = layers.Dense(32, activation='relu')(x3)
out = layers.Dense(2, activation='linear')(x4)

model_regression = keras.Model(inp, out)

# Huber loss is robust to the occasional large outlier that MSE over-penalises
model_regression.compile(
    optimizer=keras.optimizers.Adam(learning_rate=5e-4),
    loss=keras.losses.Huber(delta=20.0),   # delta   expected median error
    metrics=['mae']
)

print("NN Regression Model:")
model_regression.summary()

lr_cb = keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.5, patience=15, min_lr=1e-5, verbose=0
)
es_cb = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=40, restore_best_weights=True
)

history_reg = model_regression.fit(
```

```
    X_train_scaled, pos_train,
    validation_split=0.2,
    epochs=500,
    batch_size=32,
    verbose=0,
    callbacks=[es_cb, lr_cb]
)

pos_pred_nn_reg  = model_regression.predict(X_test_scaled, verbose=0)

error_nn_reg        = np.linalg.norm(pos_pred_nn_reg - pos_test, axis=1)
rmse_nn_reg         = np.sqrt(np.mean(error_nn_reg**2))
mae_nn_reg          = np.mean(error_nn_reg)
median_error_nn_reg = np.median(error_nn_reg)
p90_error_nn_reg    = np.percentile(error_nn_reg, 90)

print("\n" + "=" * 70)
print("NN-based REGRESSION LOCALIZATION RESULTS")
print("=" * 70)
print(f"Model: 4-layer DNN (256→128→64→32→2), Huber loss  =20 m, Dropout 0.1")
print(f"Training samples: {len(pos_train)}, Test samples: {len(pos_test)}")
print(f"Epochs trained: {len(history_reg.history['loss'])}")
print(f"RMSE: {rmse_nn_reg:.2f} m")
print(f"MAE:  {mae_nn_reg:.2f} m")
print(f"Median Error: {median_error_nn_reg:.2f} m")
print(f"90th percentile: {p90_error_nn_reg:.2f} m")
print(f"Min error: {error_nn_reg.min():.2f} m, Max error: {error_nn_reg.max():.
 ↪2f} m")

fig, ax = plt.subplots(1, 1, figsize=(10, 5))
ax.plot(history_reg.history['loss'],     label='Training Loss (Huber)')
ax.plot(history_reg.history['val_loss'], label='Validation Loss (Huber)')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss (Huber)')
ax.set_title('NN Regression Training History')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.savefig(PICTURES_DIR / "nn_regression_training.png", dpi=300)
plt.show()
```

```
NN Regression Model:
```

Model: "functional"

Layer (type)	Output Shape	Param #

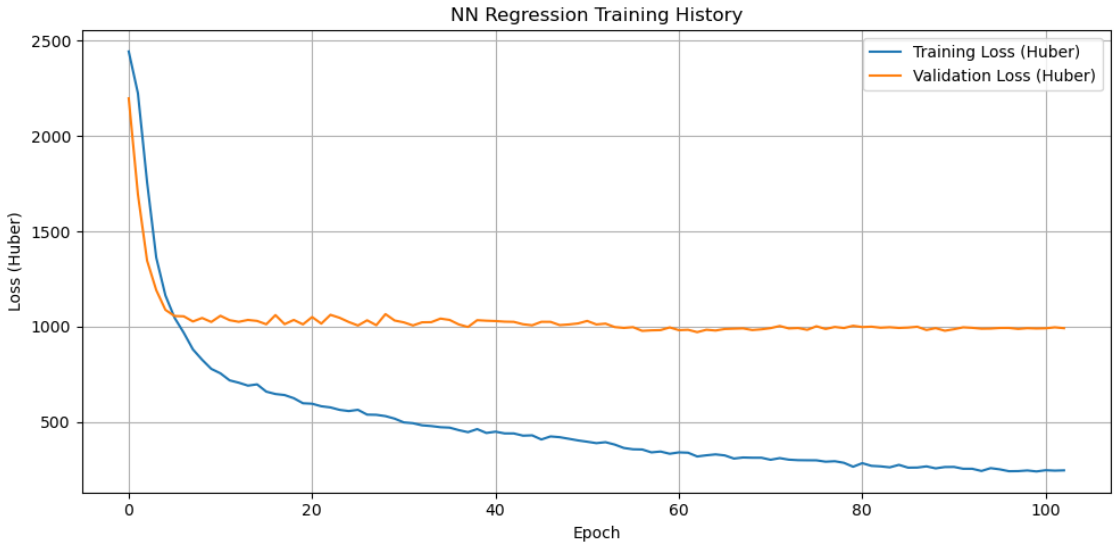
input_layer (InputLayer)	(None, 50740)	0
dense (Dense)	(None, 256)	12,989,696
batch_normalization (BatchNormalization)	(None, 256)	1,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
batch_normalization_1 (BatchNormalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dropout_2 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 32)	2,080
dense_4 (Dense)	(None, 2)	66

Total params: 13,034,530 (49.72 MB)

Trainable params: 13,033,762 (49.72 MB)

Non-trainable params: 768 (3.00 KB)

```
======================================================================
NN-based REGRESSION LOCALIZATION RESULTS
======================================================================
Model: 4-layer DNN (256→128→64→32→2), Huber loss  =20 m, Dropout 0.1
Training samples: 1964, Test samples: 842
Epochs trained: 103
RMSE: 115.68 m
MAE:  94.17 m
Median Error: 77.02 m
90th percentile: 191.42 m
Min error: 2.22 m, Max error: 415.46 m
```



[28]:
```
#   NN-based Classification: Classify into grid cells

# Discretize positions into grid cells for classification
grid_size = 10  # m
x_bins = np.arange(x_min - grid_size/2, x_max + grid_size, grid_size)
y_bins = np.arange(y_min - grid_size/2, y_max + grid_size, grid_size)

def discretize_positions(positions, x_bins, y_bins):
    """Assign continuous positions to grid cell indices."""
    x_indices = np.digitize(positions[:, 0], x_bins) - 1
    y_indices = np.digitize(positions[:, 1], y_bins) - 1
    # Ensure valid indices
    x_indices = np.clip(x_indices, 0, len(x_bins) - 2)
    y_indices = np.clip(y_indices, 0, len(y_bins) - 2)
    return x_indices + y_indices * (len(x_bins) - 1)  # flatten to 1D class

pos_train_class = discretize_positions(pos_train, x_bins, y_bins)
pos_test_class = discretize_positions(pos_test, x_bins, y_bins)

n_classes = (len(x_bins) - 1) * (len(y_bins) - 1)
print(f"Grid classification: {len(x_bins)-1} x {len(y_bins)-1} =␣
 ↪{(len(x_bins)-1)*(len(y_bins)-1)} cells")

# Build classification model
tf.random.set_seed(42)
model_classification = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(128, activation='relu'),
```

```
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense((len(x_bins)-1)*(len(y_bins)-1), activation='softmax')
])

model_classification.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train classification model
history_clf = model_classification.fit(
    X_train_scaled, pos_train_class,
    validation_split=0.2,
    epochs=100,
    batch_size=8,
    verbose=0,
    callbacks=[keras.callbacks.EarlyStopping(monitor='val_loss', patience=20,␣
 ↪restore_best_weights=True)]
)

# Predict classes
class_pred = np.argmax(model_classification.predict(X_test_scaled, verbose=0),␣
 ↪axis=1)

# Convert class predictions back to approximate positions (centroid of cell)
def class_to_position(class_indices, x_bins, y_bins, x_min, y_min, grid_size):
    positions = []
    nx = len(x_bins) - 1
    for c in class_indices:
        x_idx = c % nx
        y_idx = c // nx
        x_pos = x_min + x_idx * grid_size + grid_size / 2
        y_pos = y_min + y_idx * grid_size + grid_size / 2
        positions.append([x_pos, y_pos])
    return np.array(positions)

pos_pred_nn_clf = class_to_position(class_pred, x_bins, y_bins, x_min, y_min,␣
 ↪grid_size)

# Compute localization errors (classification)
```

```
error_nn_clf = np.linalg.norm(pos_pred_nn_clf - pos_test, axis=1)
rmse_nn_clf = np.sqrt(np.mean(error_nn_clf**2))
mae_nn_clf = np.mean(error_nn_clf)
median_error_nn_clf = np.median(error_nn_clf)
p90_error_nn_clf = np.percentile(error_nn_clf, 90)
clf_accuracy = np.mean(class_pred == pos_test_class)

print("\n" + "=" * 70)
print("NN-based CLASSIFICATION LOCALIZATION RESULTS")
print("=" * 70)
print(f"Model: 4-layer DNN (128 → 64 → 32 → {(len(x_bins)-1)*(len(y_bins)-1)}),␣
↪Softmax output")
print(f"Grid cell size: {grid_size} m")
print(f"Training samples: {len(pos_train)}, Test samples: {len(pos_test)}")
print(f"Classification accuracy: {clf_accuracy:.2%}")
print(f"RMSE (from cell centroids): {rmse_nn_clf:.2f} m")
print(f"MAE:  {mae_nn_clf:.2f} m")
print(f"Median Error: {median_error_nn_clf:.2f} m")
print(f"90th percentile: {p90_error_nn_clf:.2f} m")
print(f"Min error: {error_nn_clf.min():.2f} m, Max error: {error_nn_clf.max():.
↪2f} m")
```

```
Grid classification: 61 x 46 = 2806 cells

======================================================================
NN-based CLASSIFICATION LOCALIZATION RESULTS
======================================================================
Model: 4-layer DNN (128 → 64 → 32 → 2806), Softmax output
Grid cell size: 10 m
Training samples: 1964, Test samples: 842
Classification accuracy: 0.00%
RMSE (from cell centroids): 339.82 m
MAE:  307.23 m
Median Error: 315.04 m
90th percentile: 504.98 m
Min error: 15.81 m, Max error: 680.62 m
```

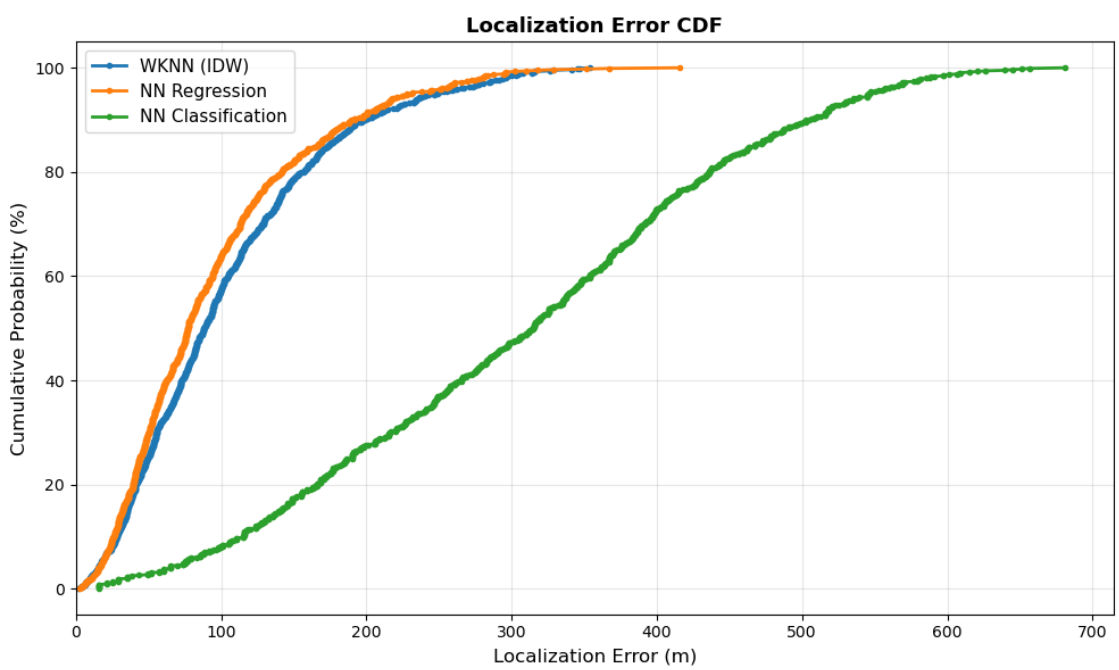
[29]:
```
#   Comprehensive Evaluation: Error CDF, RMSE Comparison, Heatmaps

# Create comparison summary
methods = ['WKNN (IDW)', 'NN Regression', 'NN Classification']
rmse_values = [rmse_wknn, rmse_nn_reg, rmse_nn_clf]
mae_values = [mae_wknn, mae_nn_reg, mae_nn_clf]
median_values = [median_error_wknn, median_error_nn_reg, median_error_nn_clf]
p90_values = [p90_error_wknn, p90_error_nn_reg, p90_error_nn_clf]

errors_all = [error_wknn, error_nn_reg, error_nn_clf]
```

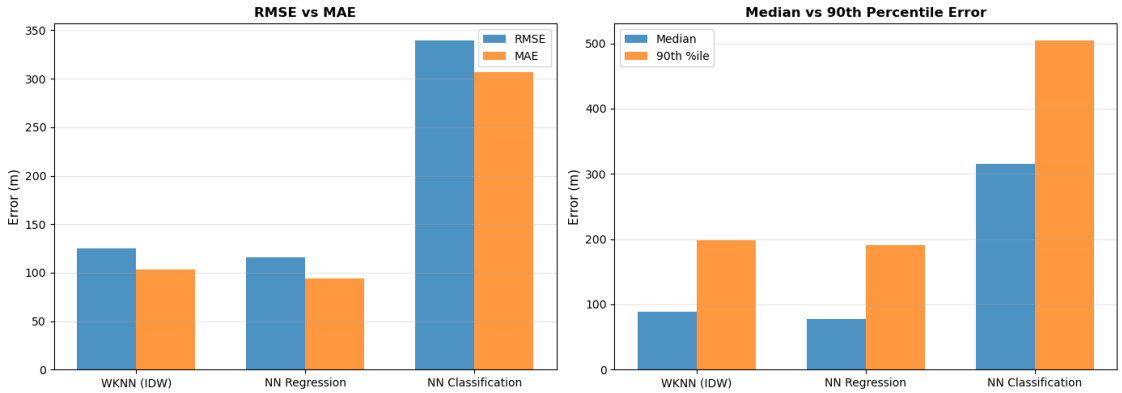
```
comparison_df = pd.DataFrame({
    'Method': methods,
    'RMSE (m)': rmse_values,
    'MAE (m)': mae_values,
    'Median (m)': median_values,
    '90th %ile (m)': p90_values
})

print("\n" + "=" * 70)
print("LOCALIZATION PERFORMANCE SUMMARY")
print("=" * 70)
print(comparison_df.to_string(index=False))

#   Plot 1: Error CDF (Cumulative Distribution Function)
fig, ax = plt.subplots(figsize=(10, 6))

for method, errors in zip(methods, errors_all):
    sorted_errors = np.sort(errors)
    cdf = np.arange(1, len(sorted_errors) + 1) / len(sorted_errors)
    ax.plot(sorted_errors, cdf * 100, marker='o', markersize=3, label=method,␣
 ↪linewidth=2)

ax.set_xlabel('Localization Error (m)', fontsize=12)
ax.set_ylabel('Cumulative Probability (%)', fontsize=12)
ax.set_title('Localization Error CDF', fontsize=13, fontweight='bold')
ax.grid(True, alpha=0.3)
ax.legend(fontsize=11)
ax.set_xlim(0, np.max([errors.max() for errors in errors_all]) * 1.05)
plt.tight_layout()
plt.savefig(PICTURES_DIR / "localization_error_cdf.png", dpi=300)
plt.show()

#   Plot 2: RMSE/MAE Comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

x_pos = np.arange(len(methods))
width = 0.35

ax1.bar(x_pos - width/2, rmse_values, width, label='RMSE', alpha=0.8)
ax1.bar(x_pos + width/2, mae_values, width, label='MAE', alpha=0.8)
ax1.set_ylabel('Error (m)', fontsize=11)
ax1.set_title('RMSE vs MAE', fontsize=12, fontweight='bold')
ax1.set_xticks(x_pos)
ax1.set_xticklabels(methods)
ax1.legend()
ax1.grid(True, alpha=0.3, axis='y')
```

```
ax2.bar(x_pos - width/2, median_values, width, label='Median', alpha=0.8)
ax2.bar(x_pos + width/2, p90_values, width, label='90th %ile', alpha=0.8)
ax2.set_ylabel('Error (m)', fontsize=11)
ax2.set_title('Median vs 90th Percentile Error', fontsize=12, fontweight='bold')
ax2.set_xticks(x_pos)
ax2.set_xticklabels(methods)
ax2.legend()
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig(PICTURES_DIR / "localization_metrics_comparison.png", dpi=300)
plt.show()

#   Plot 3: Localization Error Heatmaps on 2D Scene
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Create a 2D heatmap for each method
for idx, (method, pos_pred, error) in enumerate(zip(
    methods,
    [pos_pred_wknn, pos_pred_nn_reg, pos_pred_nn_clf],
    errors_all
)):
    ax = axes[idx]

    # Create 2D grid for heatmap
    heatmap = np.zeros((len(y_bins), len(x_bins)))
    counts = np.zeros_like(heatmap)

    for pos, err in zip(pos_test, error):
        x_idx = np.searchsorted(x_bins, pos[0]) - 1
        y_idx = np.searchsorted(y_bins, pos[1]) - 1
        x_idx = np.clip(x_idx, 0, len(x_bins) - 2)
        y_idx = np.clip(y_idx, 0, len(y_bins) - 2)
        heatmap[y_idx, x_idx] += err
        counts[y_idx, x_idx] += 1

    # Average errors per cell
    heatmap = np.divide(heatmap, counts, where=counts > 0)

    im = ax.imshow(heatmap, cmap='RdYlGn_r', origin='lower',
                   extent=[x_bins[0], x_bins[-1], y_bins[0], y_bins[-1]])
    ax.scatter(pos_test[:, 0], pos_test[:, 1], c='blue', s=30, alpha=0.6,␣
 ↪label='Test positions')
    ax.scatter([0], [0], c='red', s=100, marker='*', label='BS location')
    ax.set_xlabel('X (m)', fontsize=10)
    ax.set_ylabel('Y (m)', fontsize=10)
```

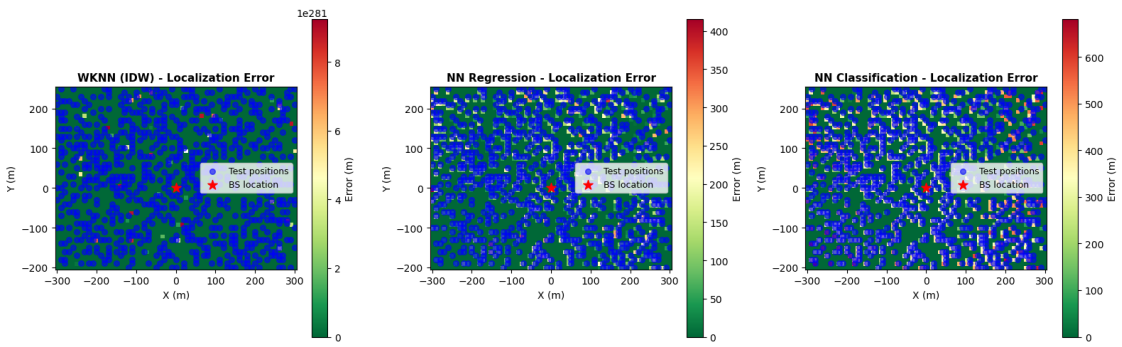
```
    ax.set_title(f'{method} - Localization Error', fontsize=11,␣
↪fontweight='bold')
    ax.legend(fontsize=9)
    cbar = plt.colorbar(im, ax=ax)
    cbar.set_label('Error (m)', fontsize=10)

plt.tight_layout()
plt.savefig(PICTURES_DIR / "localization_error_heatmaps.png", dpi=300)
plt.show()

print("\n" + "=" * 70)
print("Evaluation complete. Figures saved:")
print("  - localization_error_cdf.png")
print("  - localization_metrics_comparison.png")
print("  - localization_error_heatmaps.png")
print("=" * 70)
```

```
======================================================================
LOCALIZATION PERFORMANCE SUMMARY
======================================================================
           Method   RMSE (m)    MAE (m)  Median (m)  90th %ile (m)
        WKNN (IDW) 124.734955 103.130979   88.174700     198.500210
     NN Regression 115.681373  94.174035   77.022603     191.420479
NN Classification 339.821282 307.232163  315.039680     504.984643
```






```
/tmp/ipykernel_2352/1621958284.py:95: UserWarning: 'where' used without 'out',
expect unitialized memory in output. If this is intentional, use out=None.
  heatmap = np.divide(heatmap, counts, where=counts > 0)
```




```
======================================================================
Evaluation complete. Figures saved:
  - localization_error_cdf.png
  - localization_metrics_comparison.png
  - localization_error_heatmaps.png
======================================================================
```

[30]:
```
#   Save Fingerprint Localization Results to CSV and JSON

# Create detailed results dataframe
results_df = pd.DataFrame({
    'Position_X': pos_test[:, 0],
    'Position_Y': pos_test[:, 1],
    'WKNN_Pred_X': pos_pred_wknn[:, 0],
    'WKNN_Pred_Y': pos_pred_wknn[:, 1],
    'WKNN_Error': error_wknn,
```

```
    'NN_Reg_Pred_X': pos_pred_nn_reg[:, 0],
    'NN_Reg_Pred_Y': pos_pred_nn_reg[:, 1],
    'NN_Reg_Error': error_nn_reg,
    'NN_Clf_Pred_X': pos_pred_nn_clf[:, 0],
    'NN_Clf_Pred_Y': pos_pred_nn_clf[:, 1],
    'NN_Clf_Error': error_nn_clf,
})

results_csv_path = os.path.join(scene_dir, 'fingerprint_localization_results.
 ↪csv')
results_df.to_csv(results_csv_path, index=False)
print(f"Saved results to: {results_csv_path}")

# Create summary statistics JSON
summary_stats = {
    'fingerprint_localization': {
        'dataset': {
            'n_training_samples': len(pos_train),
            'n_test_samples': len(pos_test),
            'n_features': X_train_scaled.shape[1],
            'feature_type': 'CSI magnitude (frequency domain)',
            'scene': 'Otaniemi',
            'bs_position': [float(BS_X), float(BS_Y), float(BS_H)],
            'test_region': {
                'x_range': [float(x_min), float(x_max)],
                'y_range': [float(y_min), float(y_max)],
            }
        },
        'methods': {
            'wknn': {
                'algorithm': 'Weighted K-Nearest Neighbors (Inverse Distance␣
 ↪Weighting)',
                'k_neighbors': k_neighbors,
                'distance_metric': distance_metric,
                'rmse_m': float(rmse_wknn),
                'mae_m': float(mae_wknn),
                'median_error_m': float(median_error_wknn),
                'p90_error_m': float(p90_error_wknn),
                'min_error_m': float(error_wknn.min()),
                'max_error_m': float(error_wknn.max()),
            },
            'nn_regression': {
                'algorithm': 'Deep Neural Network (Regression)',
                'architecture': '256 → 128 → 64 → 32 → 2 (linear output)',
                'activation': 'ReLU',
                'dropout': '0.1',
                'optimizer': 'Adam (lr=5e-4)',
```

```
                'loss': 'Huber (delta=20)',
                'rmse_m': float(rmse_nn_reg),
                'mae_m': float(mae_nn_reg),
                'median_error_m': float(median_error_nn_reg),
                'p90_error_m': float(p90_error_nn_reg),
                'min_error_m': float(error_nn_reg.min()),
                'max_error_m': float(error_nn_reg.max()),
            },
            'nn_classification': {
                'algorithm': 'Deep Neural Network (Classification)',
                'architecture': f'128 → 64 → 32 →␣
 ↪{(len(x_bins)-1)*(len(y_bins)-1)} (softmax)',
                'activation': 'ReLU',
                'dropout': '0.2-0.3',
                'optimizer': 'Adam (lr=0.001)',
                'loss': 'Sparse Categorical Crossentropy',
                'grid_cell_size_m': grid_size,
                'classification_accuracy': float(clf_accuracy),
                'rmse_m': float(rmse_nn_clf),
                'mae_m': float(mae_nn_clf),
                'median_error_m': float(median_error_nn_clf),
                'p90_error_m': float(p90_error_nn_clf),
                'min_error_m': float(error_nn_clf.min()),
                'max_error_m': float(error_nn_clf.max()),
            }
        }
    }
}

import json
summary_json_path = os.path.join(scene_dir, 'fingerprint_localization_summary.
 ↪json')
with open(summary_json_path, 'w') as f:
    json.dump(summary_stats, f, indent=2)
print(f"Saved summary to: {summary_json_path}")

print("\n" + "=" * 70)
print("FINGERPRINT LOCALIZATION WORKFLOW COMPLETE")
print("=" * 70)
print(f"\nBest performing method: {methods[np.argmin(rmse_values)]} (RMSE:␣
 ↪{min(rmse_values):.2f} m)")
print(f"Key findings:")
print(f"  • WKNN provides a fast baseline with {mae_wknn:.2f} m MAE")
print(f"  • NN Regression achieves {mae_nn_reg:.2f} m MAE with end-to-end␣
 ↪learning")
print(f"  • NN Classification achieves {clf_accuracy:.1%} grid cell accuracy")
print(f"\nAll results saved to: {scene_dir}")
```

```
print("  - fingerprint_localization_results.csv")
print("  - fingerprint_localization_summary.json")
print("  - Generated figures: error CDF, metrics comparison, heatmaps")
```

```
Saved results to: /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine
Learning for Wireless Comunications
E7340/Project/fingerprint_localization_results.csv
Saved summary to: /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine
Learning for Wireless Comunications
E7340/Project/fingerprint_localization_summary.json

======================================================================
FINGERPRINT LOCALIZATION WORKFLOW COMPLETE
======================================================================

Best performing method: NN Regression (RMSE: 115.68 m)
Key findings:
  • WKNN provides a fast baseline with 103.13 m MAE
  • NN Regression achieves 94.17 m MAE with end-to-end learning
  • NN Classification achieves 0.0% grid cell accuracy

All results saved to: /mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine
Learning for Wireless Comunications E7340/Project
  - fingerprint_localization_results.csv
  - fingerprint_localization_summary.json
  - Generated figures: error CDF, metrics comparison, heatmaps
```

5.1 3. Channel Charting

[31]:
```
#%pip install umap-learn
%pip install umap-learn --trusted-host pypi.org --trusted-host files.
 ↪pythonhosted.org
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors as SklearnNN
from scipy.spatial.distance import cdist
import warnings
try:
  import umap
except ModuleNotFoundError:
  import sys
  import subprocess
  subprocess.check_call([sys.executable, "-m", "pip", "install", "umap-learn"])
  import umap

#   Channel Charting
```

```python
# Conventional DR methods: PCA, t-SNE
# NN-based method: Autoencoder
# Additional method: UMAP
# Evaluation: TW (Trustworthiness), CT (Continuity), KS (Kruskal Stress)

warnings.filterwarnings('ignore')

try:
  UMAP_AVAILABLE = True
except ImportError:
  print("umap-learn not installed. Install with: pip install umap-learn")
  UMAP_AVAILABLE = False

#   1. Prepare CSI features for channel charting
# Use all training fingerprints (before train/test split)
X_cc = csi_fingerprints.copy()       # [N, n_features]
pos_cc = fingerprint_positions.copy() # [N, 2]

scaler_cc = StandardScaler()
X_cc_scaled = scaler_cc.fit_transform(X_cc)

N = X_cc_scaled.shape[0]
print(f"Channel charting dataset: {N} samples, {X_cc_scaled.shape[1]} features")

#   2. Evaluation Metrics
def compute_tw_ct(X_high, X_low, k=10):
  """
  Compute Trustworthiness (TW) and Continuity (CT) for DR quality evaluation.
  TW: points that are close in low-dim are also close in high-dim (no false␣
 ↪neighbors)
  CT: points that are close in high-dim are also close in low-dim (no missing␣
 ↪neighbors)
  """
  n = X_high.shape[0]
  k = min(k, n - 1)

  # Nearest neighbors in high-dimensional space
  nn_high = SklearnNN(n_neighbors=n-1).fit(X_high)
  _, indices_high = nn_high.kneighbors(X_high)

  # Nearest neighbors in low-dimensional space
  nn_low = SklearnNN(n_neighbors=n-1).fit(X_low)
  _, indices_low = nn_low.kneighbors(X_low)

  # Rank matrices
  rank_high = np.zeros((n, n), dtype=int)
  rank_low  = np.zeros((n, n), dtype=int)
```

```
  for i in range(n):
    for r, j in enumerate(indices_high[i]):
      rank_high[i, j] = r + 1
    for r, j in enumerate(indices_low[i]):
      rank_low[i, j] = r + 1

  # Trustworthiness
  tw_sum = 0.0
  for i in range(n):
    neighbors_low_k = set(indices_low[i][:k])
    neighbors_high_k = set(indices_high[i][:k])
    false_neighbors = neighbors_low_k - neighbors_high_k
    for j in false_neighbors:
      tw_sum += rank_high[i, j] - k
  denom_tw = n * k * (2 * n - 3 * k - 1) / 2
  tw = 1.0 - (2.0 / denom_tw) * tw_sum if denom_tw > 0 else 1.0

  # Continuity
  ct_sum = 0.0
  for i in range(n):
    neighbors_high_k = set(indices_high[i][:k])
    neighbors_low_k  = set(indices_low[i][:k])
    missing_neighbors = neighbors_high_k - neighbors_low_k
    for j in missing_neighbors:
      ct_sum += rank_low[i, j] - k
  denom_ct = n * k * (2 * n - 3 * k - 1) / 2
  ct = 1.0 - (2.0 / denom_ct) * ct_sum if denom_ct > 0 else 1.0

  return float(tw), float(ct)

def compute_kruskal_stress(X_high, X_low):
  """
  Kruskal Stress: measures how well pairwise distances are preserved.
  Lower is better (0 = perfect).
  """
  d_high = cdist(X_high, X_high).ravel()
  d_low  = cdist(X_low,  X_low ).ravel()
  mask   = d_high > 0
  stress = np.sqrt(np.sum((d_high[mask] - d_low[mask])**2) /
           np.sum(d_high[mask]**2))
  return float(stress)

#   3. PCA Channel Chart
print("\nComputing PCA channel chart...")
pca = PCA(n_components=2, random_state=42)
Z_pca = pca.fit_transform(X_cc_scaled)
```

```
# PCA explained variance for parameter selection insight
pca_full = PCA(random_state=42).fit(X_cc_scaled)
explained_var = np.cumsum(pca_full.explained_variance_ratio_)
print(f"  PCA: top-2 components explain {100*pca.explained_variance_ratio_.
 ↪sum():.1f}% variance")

tw_pca, ct_pca = compute_tw_ct(X_cc_scaled, Z_pca, k=10)
ks_pca = compute_kruskal_stress(X_cc_scaled, Z_pca)
print(f"  TW={tw_pca:.4f}, CT={ct_pca:.4f}, KS={ks_pca:.4f}")

#   4. t-SNE Channel Chart (parameter sweep on perplexity)
print("\nComputing t-SNE channel chart (perplexity sweep)...")
perplexities = [5, 15, 30, 50]
tsne_results = {}
for perp in perplexities:
  tsne = TSNE(n_components=2, perplexity=min(perp, N//2 - 1),
        random_state=42, max_iter=1000, n_jobs=1)
  Z_tsne = tsne.fit_transform(X_cc_scaled)
  ks = compute_kruskal_stress(X_cc_scaled, Z_tsne)
  tsne_results[perp] = {'Z': Z_tsne, 'KS': ks}
  print(f"  perplexity={perp}: KS={ks:.4f}")

# Use best perplexity (lowest KS)
best_perp = min(tsne_results, key=lambda p: tsne_results[p]['KS'])
Z_tsne_best = tsne_results[best_perp]['Z']
tw_tsne, ct_tsne = compute_tw_ct(X_cc_scaled, Z_tsne_best, k=10)
ks_tsne = tsne_results[best_perp]['KS']
print(f"  Best perplexity={best_perp}: TW={tw_tsne:.4f}, CT={ct_tsne:.4f},␣
 ↪KS={ks_tsne:.4f}")

#   5. UMAP Channel Chart (parameter sweep on n_neighbors)
Z_umap_best = None
tw_umap = ct_umap = ks_umap = float('nan')
best_n_neighbors = None

if UMAP_AVAILABLE:
  print("\nComputing UMAP channel chart (n_neighbors sweep)...")
  n_neighbors_list = [5, 10, 20, 30]
  umap_results = {}
  for nn in n_neighbors_list:
    reducer = umap.UMAP(n_components=2, n_neighbors=min(nn, N-1),
              min_dist=0.1, random_state=42)
    Z_u = reducer.fit_transform(X_cc_scaled)
    ks = compute_kruskal_stress(X_cc_scaled, Z_u)
    umap_results[nn] = {'Z': Z_u, 'KS': ks}
    print(f"  n_neighbors={nn}: KS={ks:.4f}")
```

```
  best_n_neighbors = min(umap_results, key=lambda n: umap_results[n]['KS'])
  Z_umap_best = umap_results[best_n_neighbors]['Z']
  tw_umap, ct_umap = compute_tw_ct(X_cc_scaled, Z_umap_best, k=10)
  ks_umap = umap_results[best_n_neighbors]['KS']
  print(f"  Best n_neighbors={best_n_neighbors}: TW={tw_umap:.4f}, CT={ct_umap:.
 ↪4f}, KS={ks_umap:.4f}")

#   6. NN-based Channel Chart (Autoencoder)
print("\nTraining Autoencoder channel chart...")
tf.random.set_seed(42)
np.random.seed(42)

input_dim = X_cc_scaled.shape[1]
latent_dim = 2

# Encoder
enc_input = keras.Input(shape=(input_dim,))
x = layers.Dense(64, activation='relu')(enc_input)
x = layers.BatchNormalization()(x)
x = layers.Dense(32, activation='relu')(x)
z = layers.Dense(latent_dim, activation='linear', name='latent')(x)
encoder = keras.Model(enc_input, z, name='encoder')

# Decoder
dec_input = keras.Input(shape=(latent_dim,))
x = layers.Dense(32, activation='relu')(dec_input)
x = layers.BatchNormalization()(x)
x = layers.Dense(64, activation='relu')(x)
dec_output = layers.Dense(input_dim, activation='linear')(x)
decoder = keras.Model(dec_input, dec_output, name='decoder')

# Autoencoder
ae_input = keras.Input(shape=(input_dim,))
ae_output = decoder(encoder(ae_input))
autoencoder = keras.Model(ae_input, ae_output, name='autoencoder')
autoencoder.compile(optimizer=keras.optimizers.Adam(1e-3), loss='mse')

history_ae = autoencoder.fit(
  X_cc_scaled, X_cc_scaled,
  epochs=200, batch_size=16, validation_split=0.15, verbose=0,
  callbacks=[keras.callbacks.EarlyStopping(monitor='val_loss', patience=20,
                        restore_best_weights=True)]
)

Z_ae = encoder.predict(X_cc_scaled, verbose=0)
tw_ae, ct_ae = compute_tw_ct(X_cc_scaled, Z_ae, k=10)
ks_ae = compute_kruskal_stress(X_cc_scaled, Z_ae)
```

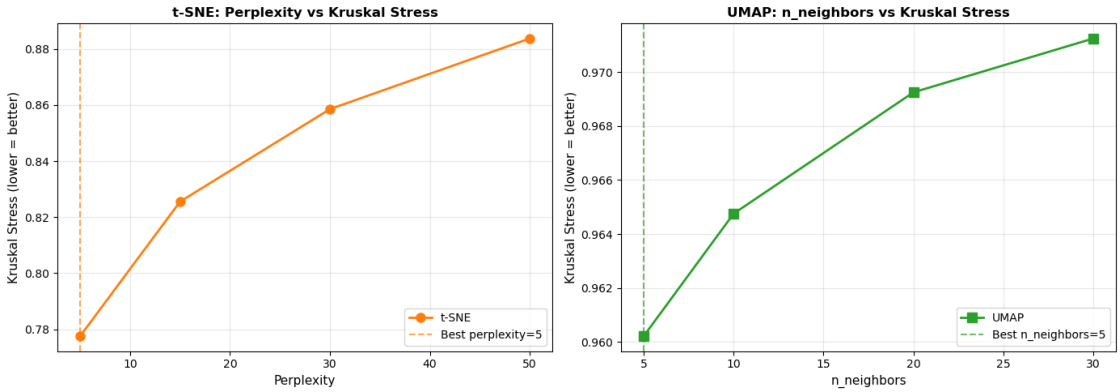
```
print(f"  Autoencoder: TW={tw_ae:.4f}, CT={ct_ae:.4f}, KS={ks_ae:.4f}")

#   7. Parameter Sensitivity: t-SNE perplexity vs KS
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(perplexities, [tsne_results[p]['KS'] for p in perplexities],
       'o-', color='C1', linewidth=2, markersize=8, label='t-SNE')
axes[0].axvline(best_perp, color='C1', linestyle='--', alpha=0.7,
        label=f'Best perplexity={best_perp}')
axes[0].set_xlabel('Perplexity', fontsize=11)
axes[0].set_ylabel('Kruskal Stress (lower = better)', fontsize=11)
axes[0].set_title('t-SNE: Perplexity vs Kruskal Stress', fontsize=12,␣
 ↪fontweight='bold')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

if UMAP_AVAILABLE:
  axes[1].plot(n_neighbors_list,
         [umap_results[n]['KS'] for n in n_neighbors_list],
         's-', color='C2', linewidth=2, markersize=8, label='UMAP')
  axes[1].axvline(best_n_neighbors, color='C2', linestyle='--', alpha=0.7,
          label=f'Best n_neighbors={best_n_neighbors}')
  axes[1].set_xlabel('n_neighbors', fontsize=11)
  axes[1].set_ylabel('Kruskal Stress (lower = better)', fontsize=11)
  axes[1].set_title('UMAP: n_neighbors vs Kruskal Stress', fontsize=12,␣
 ↪fontweight='bold')
  axes[1].legend()
  axes[1].grid(True, alpha=0.3)
else:
  axes[1].text(0.5, 0.5, 'UMAP not available\n(pip install umap-learn)',
         ha='center', va='center', transform=axes[1].transAxes, fontsize=13)
  axes[1].set_title('UMAP: n_neighbors vs Kruskal Stress', fontsize=12)

plt.tight_layout()
plt.savefig(PICTURES_DIR / "channel_charting_param_sweep.png", dpi=300)
plt.show()

#   8. Channel Chart Visualizations
n_methods = 4 if UMAP_AVAILABLE else 3
fig, axes = plt.subplots(1, n_methods, figsize=(6*n_methods, 5))

color_by_dist = np.linalg.norm(pos_cc, axis=1)  # distance from scene center

def plot_chart(ax, Z, pos, title, cmap='plasma'):
  sc = ax.scatter(Z[:, 0], Z[:, 1], c=color_by_dist, cmap=cmap,
          s=20, alpha=0.8)
  plt.colorbar(sc, ax=ax, label='Distance from center (m)')
```

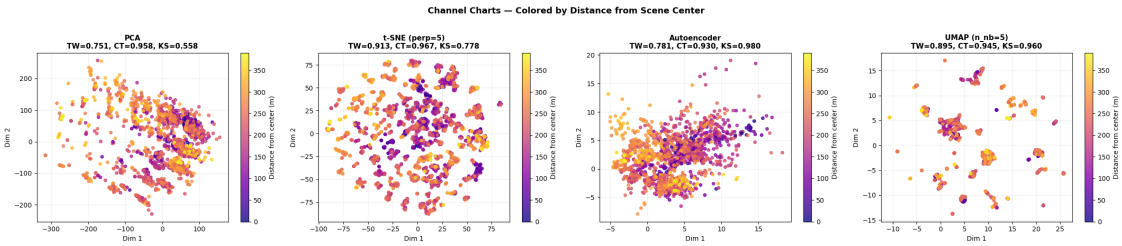
```
  ax.set_title(title, fontsize=11, fontweight='bold')
  ax.set_xlabel('Dim 1')
  ax.set_ylabel('Dim 2')
  ax.grid(True, alpha=0.2)

plot_chart(axes[0], Z_pca, pos_cc,
       f'PCA\nTW={tw_pca:.3f}, CT={ct_pca:.3f}, KS={ks_pca:.3f}')
plot_chart(axes[1], Z_tsne_best, pos_cc,
       f't-SNE (perp={best_perp})\nTW={tw_tsne:.3f}, CT={ct_tsne:.3f},␣
 ↪KS={ks_tsne:.3f}')
plot_chart(axes[2], Z_ae, pos_cc,
       f'Autoencoder\nTW={tw_ae:.3f}, CT={ct_ae:.3f}, KS={ks_ae:.3f}')
if UMAP_AVAILABLE:
  plot_chart(axes[3], Z_umap_best, pos_cc,
         f'UMAP (n_nb={best_n_neighbors})\nTW={tw_umap:.3f}, CT={ct_umap:.3f},␣
 ↪KS={ks_umap:.3f}')

plt.suptitle('Channel Charts - Colored by Distance from Scene Center',
       fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(PICTURES_DIR / "channel_charts_comparison.png", dpi=300,␣
 ↪bbox_inches='tight')
plt.show()

#   9. PCA Explained Variance
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(np.arange(1, len(explained_var)+1), explained_var * 100, 'o-',␣
 ↪linewidth=2)
ax.axhline(90, color='red', linestyle='--', label='90% threshold')
ax.axhline(95, color='orange', linestyle='--', label='95% threshold')
ax.set_xlabel('Number of Components')
ax.set_ylabel('Cumulative Explained Variance (%)')
ax.set_title('PCA Explained Variance', fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(PICTURES_DIR / "pca_explained_variance.png", dpi=300)
plt.show()

#   10. Autoencoder Training History
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(history_ae.history['loss'], label='Train Loss')
ax.plot(history_ae.history['val_loss'], label='Val Loss')
ax.set_xlabel('Epoch')
ax.set_ylabel('MSE Loss')
ax.set_title('Autoencoder Training History', fontweight='bold')
ax.legend()
```

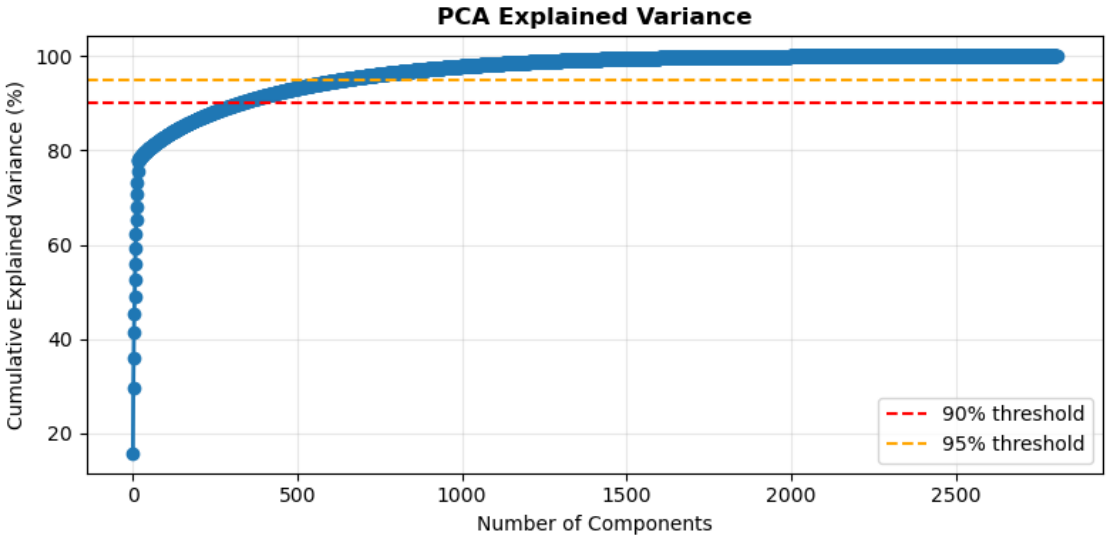
```python
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(PICTURES_DIR / "autoencoder_training.png", dpi=300)
plt.show()

#   11. Summary Table
cc_methods = ['PCA', f't-SNE (perp={best_perp})', 'Autoencoder']
tw_vals  = [tw_pca, tw_tsne, tw_ae]
ct_vals  = [ct_pca, ct_tsne, ct_ae]
ks_vals  = [ks_pca, ks_tsne, ks_ae]

if UMAP_AVAILABLE and Z_umap_best is not None:
  cc_methods.append(f'UMAP (n_nb={best_n_neighbors})')
  tw_vals.append(tw_umap)
  ct_vals.append(ct_umap)
  ks_vals.append(ks_umap)

df_cc = pd.DataFrame({
  'Method'          : cc_methods,
  'TW (↑ better)'   : [f'{v:.4f}' for v in tw_vals],
  'CT (↑ better)'   : [f'{v:.4f}' for v in ct_vals],
  'KS (↓ better)'   : [f'{v:.4f}' for v in ks_vals],
})

print("\n" + "="*65)
print("CHANNEL CHARTING EVALUATION SUMMARY")
print("="*65)
print(df_cc.to_string(index=False))

# Save metrics
cc_csv = os.path.join(scene_dir, "channel_charting_metrics.csv")
df_cc.to_csv(cc_csv, index=False)
print(f"\nSaved channel charting metrics -> {cc_csv}")
```

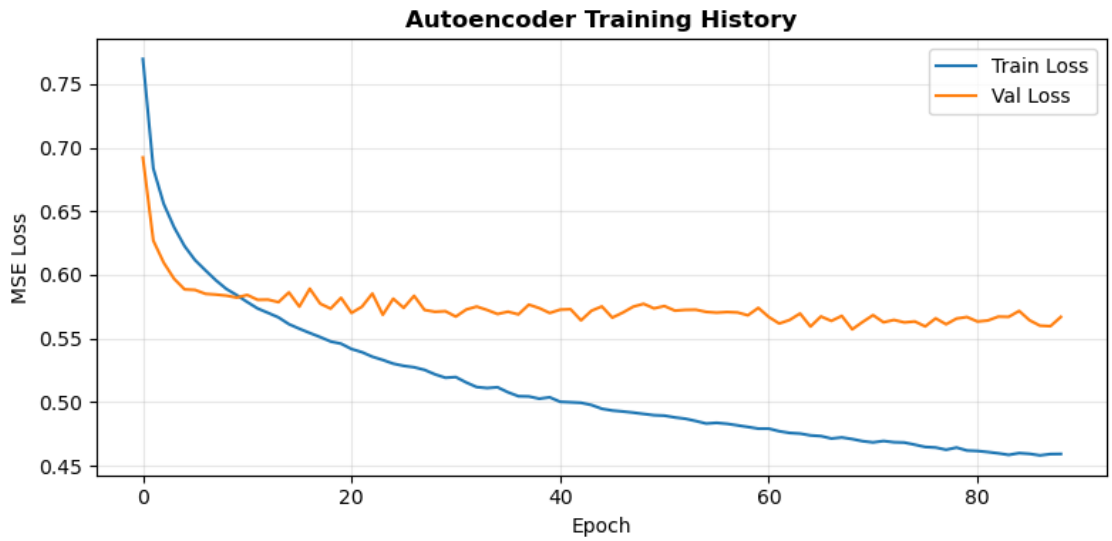
```
Requirement already satisfied: umap-learn in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (0.5.11)
Requirement already satisfied: numpy>=1.23 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (2.4.3)
Requirement already satisfied: scipy>=1.3.1 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (1.17.1)
Requirement already satisfied: scikit-learn>=1.6 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (1.8.0)
Requirement already satisfied: numba>=0.51.2 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (0.64.0)
```

```
Requirement already satisfied: pynndescent>=0.5 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (0.6.0)
Requirement already satisfied: tqdm in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
umap-learn) (4.67.3)
Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
numba>=0.51.2->umap-learn) (0.46.0)
Requirement already satisfied: joblib>=0.11 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
pynndescent>=0.5->umap-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in
/home/jarikarp/miniconda3/envs/sionna_env2/lib/python3.11/site-packages (from
scikit-learn>=1.6->umap-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
Channel charting dataset: 2806 samples, 50740 features

Computing PCA channel chart…
  PCA: top-2 components explain 29.6% variance
  TW=0.7507, CT=0.9576, KS=0.5576

Computing t-SNE channel chart (perplexity sweep)…
  perplexity=5: KS=0.7776
  perplexity=15: KS=0.8255
  perplexity=30: KS=0.8585
  perplexity=50: KS=0.8837
  Best perplexity=5: TW=0.9130, CT=0.9667, KS=0.7776

Computing UMAP channel chart (n_neighbors sweep)…
  n_neighbors=5: KS=0.9602
  n_neighbors=10: KS=0.9648
  n_neighbors=20: KS=0.9693
  n_neighbors=30: KS=0.9712
  Best n_neighbors=5: TW=0.8953, CT=0.9445, KS=0.9602

Training Autoencoder channel chart…
  Autoencoder: TW=0.7811, CT=0.9305, KS=0.9797
```


t-SNE: Perplexity vs Kruskal Stress
Kruskal Stress (lower = better)
Perplexity
t-SNE
Best perplexity=5
UMAP: n_neighbors vs Kruskal Stress
Kruskal Stress (lower = better)
n_neighbors
UMAP
Best n_neighbors=5


Channel Charts — Colored by Distance from Scene Center
PCA
TW=0.751, CT=0.958, KS=0.558
t-SNE (perp=5)
TW=0.913, CT=0.967, KS=0.778
Autoencoder
TW=0.781, CT=0.930, KS=0.980
UMAP (n_nb=5)
TW=0.895, CT=0.945, KS=0.960
Dim 1
Dim 2
Distance from center (m)


PCA Explained Variance
Cumulative Explained Variance (%)
Number of Components
90% threshold
95% threshold




```
================================================================
CHANNEL CHARTING EVALUATION SUMMARY
================================================================
        Method TW (↑ better) CT (↑ better) KS (↓ better)
           PCA        0.7507        0.9576        0.5576
t-SNE (perp=5)        0.9130        0.9667        0.7776
   Autoencoder        0.7811        0.9305        0.9797
 UMAP (n_nb=5)        0.8953        0.9445        0.9602

Saved channel charting metrics ->
/mnt/c/Users/jarikarp/iCloudDrive/Jari/Opiskelu/Machine Learning for Wireless
Comunications E7340/Project/channel_charting_metrics.csv
```